

Algoritma Analizi

Algoritma

- Algoritma bir problemi çözmek için izlenen komutlar kümesidir.
 - Verilen bir problemin birden fazla çözümü dolayısıyla birden fazla algoritması olabilir.
 - Bir algoritma farklı platformlarda farklı programlama dilleri kullanılarak işletilebilir.
- Algoritma tam olarak doğru olmalı ve problemi doğru bir biçimde çözmelidir.
- Elimizde bir problemi çözen bir algoritma olduğunda hemen bu algoritmanın etkinliğini de belirlemeliyiz.

Algorithmik Performans

Algoritmik performansın iki yönü vardır:

- Zaman (Time)
 - Komutlar zaman alır.
 - Algoritmanın çalışması nasıl hızlanır?
 - Çalışma zamanının etkileyen unsurlar nelerdir?
- Bellek (Space)
 - Kullanılan Veri yapısı bellek kullanımını etkiler
 - Kullanılan veri yapısı algoritma çalışma zamanına etki eder.
 - Kullanılan veri yapısı algoritma tasarım fikrini etkiler
- Genel olarak «zaman» üzerine odaklanılır
 - Bir algoritma için gerekli zaman nasıl tahmin edilebilir?
 - Gereken bu zaman nasıl azaltılabilir?

Algoritma Analizi

- **Algoritma analizi**, çözümlerin değişik yöntemlerini analiz etmeye yarayan bir bilgisayar bilimleri alanıdır.
- Aynı problemi çözen iki algoritmanın etkinliği nasıl karşılaştırılabilir?

Naïve yaklaşım: Bir programlama dilinde algoritma çalıştırılır, ve zaman gereksinimleri karşılaştırılır. Fakat programların karşılaştırılmasının zorlukları vardır. Algoritmaları karşılaştırmak daha gerçekçidir.

- *Her programcı farklı bir kodlama tekniği kullanabilir.*
 - Bu durumda karşılaştırma gerçekçi olmaz.
 - Programlama dilinin hassasiyetleri farklı olabilir.
- *Kullanılan bilgisayar farklı olabilir. Belirli bir bilgisayara göre çalışma zamanını karşılaştıramayız.*
- *Programda kullanılan veri yapısı farklı olabilir.*
 - Oysa analiz özel ve belirli bir veri yapısından bağımsız olmalıdır.

Algoritma Analizi

- Bir algoritma analiz edileceği zaman, kullanılan bilgisayardan, programlama dilinden, programcıdan ve veri yapısından bağımsız bir matematiksel teknik kullanılmalıdır.
- Algoritma analizi için:
 - Öncelikle çözümün etkinliğini belirlemek için etkili önemli işlemlerin sayısını hesaplarız.
 - Daha sonra algoritmanın verimini büyüme fonksiyonu olarak ifade ederiz.

Algoritmanın işletim Zamanı:

- Algoritmadaki her işlemin bir maliyeti vardır.
→ Her işlem belirli bir zaman alır.

`count = count + 1;` → belirli bir miktar zaman alır ama sabittir.

Bir dizi işlem:

`count = count + 1;`

maliyet: c_1

`sum = sum + count;`

maliyet: c_2

→ toplam maliyet = $c_1 + c_2$

Algoritmanın işletim Zamanı:

Örnek:: Basit If-Deyimi:

	<u>maliyet</u>	<u>Tekrar</u>
if (n < 0)	c1	1
absval = -n	c2	1
else		
absval = n;	c3	1

Toplam maliyet $\leq c1 + \max(c2, c3)$

Algoritmanın işletim Zamanı:

Örnek: Temel Döngü

	<u>maliyet</u>	<u>Tekrar</u>
<code>i = 1;</code>	c1	1
<code>sum = 0;</code>	c2	1
<code>while (i <= n) {</code>	c3	n+1
<code>i = i + 1;</code>	c4	n
<code>sum = sum + i;</code>	c5	n
<code>}</code>		

Toplam maliyet = $c1 + c2 + (n+1)*c3 + n*c4 + n*c5$

➔ Bu algoritma için gerekli zaman n veri sayısı ile orantılıdır.

Algoritmanın işletim Zamanı:

Örnek: İç içe Döngü

	<u>maliyet</u>	<u>Tekrar</u>
i=1;	c1	1
sum = 0;	c2	1
while (i <= n) {	c3	n+1
j=1;	c4	n
while (j <= n) {	c5	n* (n+1)
sum = sum + i;	c6	n*n
j = j + 1;	c7	n*n
}		
i = i +1;	c8	n
}		

Toplam maliyet=c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5+n*n*c6+n*n*c7+n*c8

➔ Bu algoritma için gerekli zaman n^2 ile orantılıdır.

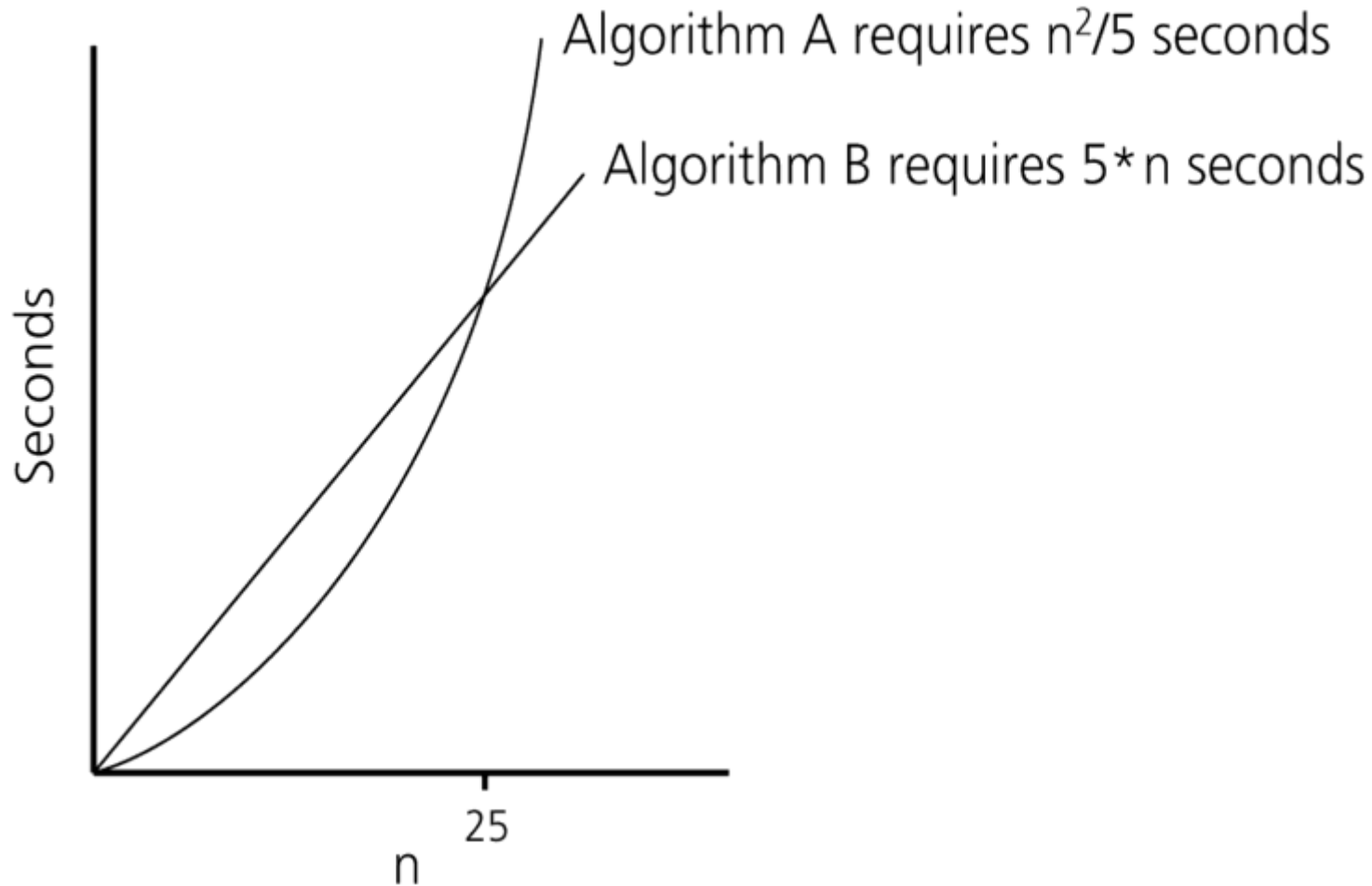
Çalışma zamanı Tahmini için genel kurallar

- **Döngüler:** Bir döngünün çalışma zamanı, en fazla döngü içindeki ifadelerin tekrar sayısı kadardır.
- **Yuvalanmış Döngüler:** En içteki döngüye kadar ifade ve işlem içeren yuvalanmış döngülerin çalışma zamanı, bütün iç döngülerin boyutlarının çarpımı kadardır.
- **Ardışık Deyimler:** Bu ardışık deyimlerin çalışma zamanı toplam çalışma zamanına eklenir.
- **If/Else:** Test sonucuna göre işletilecek olan S1 ve S2 deyimlerinden çalışma zamanı daha büyük olandır.

Algoritma Büyüme Hızları

- Algoritmanın çalışma zamanı *problem girdi boyutunun* bir fonksiyonu olarak ölçülür.
 - Problem girdi boyutu uygulamaya göre değişir. Sıralama algoritması için eleman sayısı, hanoi kuleleri için disk sayısı olur.
- Örnek olarak problem girdi boyutu n olsun
 - A Algoritması $5 \cdot n^2$ zaman gerektirsin.
 - B Algoritması $7 \cdot n$ zaman gerektirsin.
- Giriş veri sayısına bağlı olarak A algoritması n^2 ile B algoritması ise n ile orantılı olarak büyümektedir.
- Bir algoritmanın oransal zaman gereksinimine büyüme hızı (*growth rate*).
- Algoritmaları büyüme hızları ile karşılaştırabiliriz.

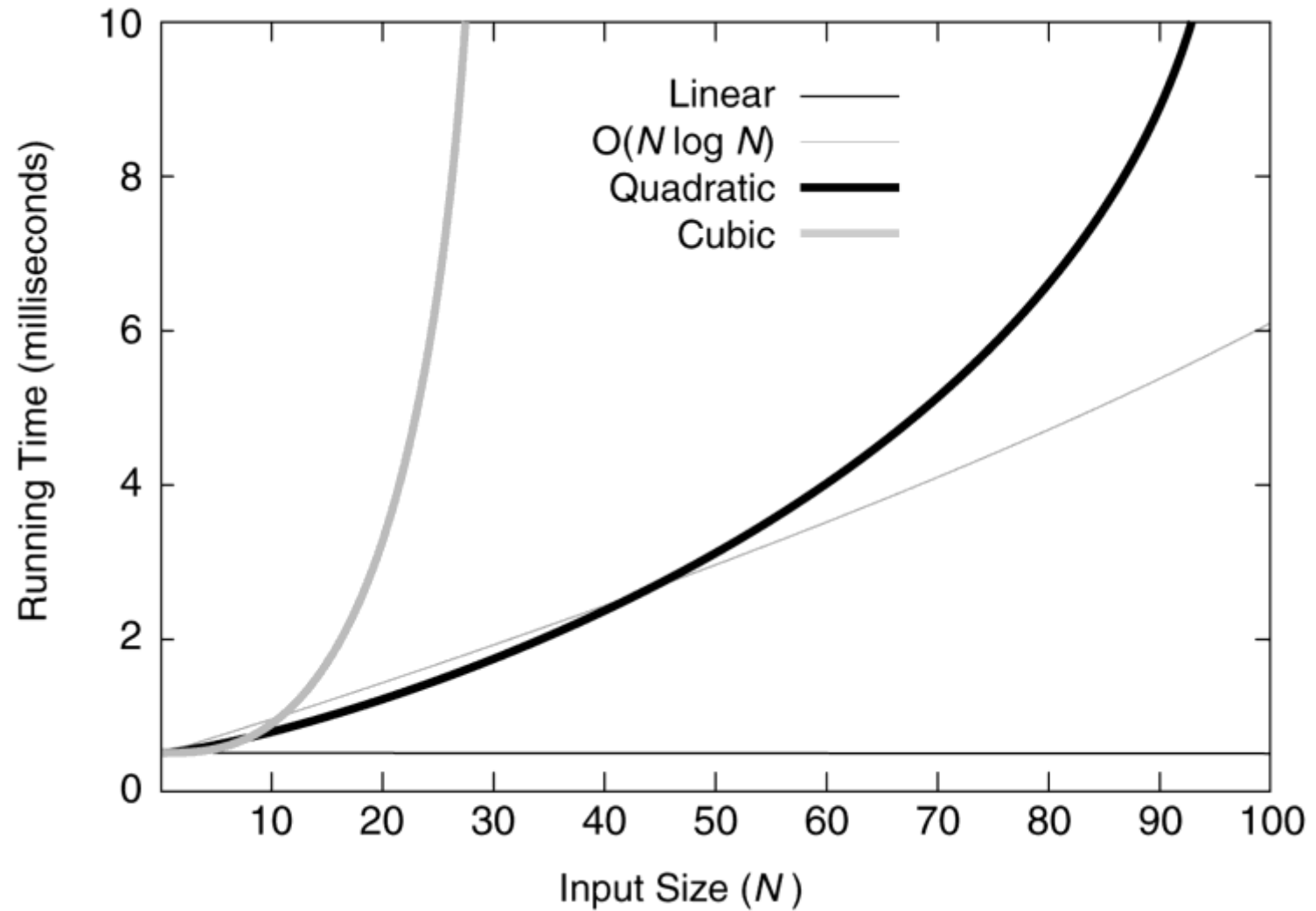
Algoritma Büyüme hızları



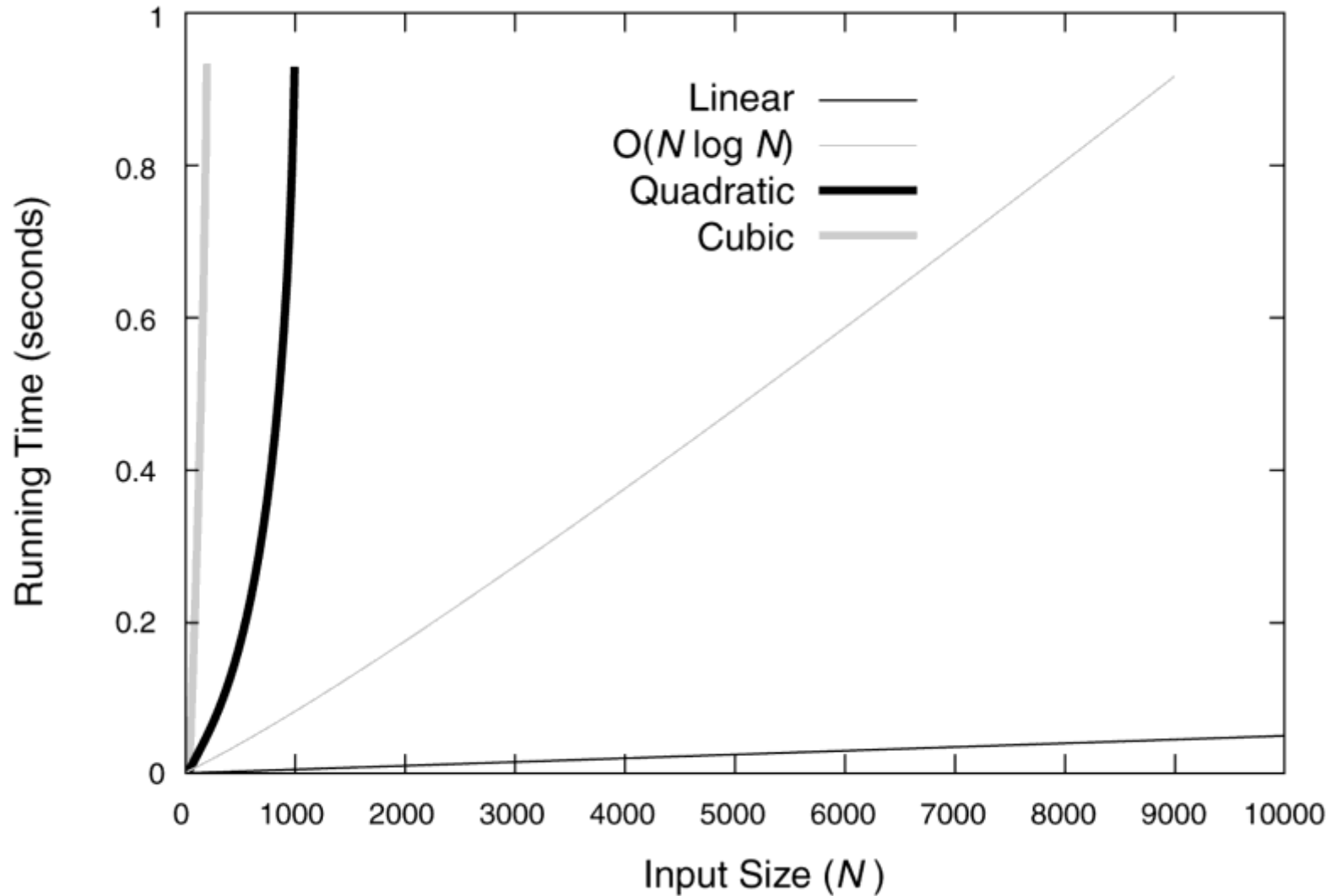
Yaygın Büyüme hızları

Fonksiyon	Büyüme hızı adı
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Küçük veriler için çalışma zamanları



Ortalama veriler için çalışma zamanları



Big O Gösterimi

- Bir A algoritması $f(n)$ ile orantılı zaman gerektiriyorsa A Algoritmasına $f(n)$ mertebesinde denilir ve $O(f(n))$ ile gösterilir.
- $f(n)$ 'e algoritmanın **growth-rate fonksiyonu** denir.
- Bu gösterime **Big O notation** adı verilir.
- A algoritması n^2 , ile orantılı zaman gerektiriyorsa **$O(n^2)$** .
- A algoritması n ile orantılı zaman gerektiriyorsa **$O(n)$** ile ifade edilir.

Algoritma derecesinin tanımı:

Tanım:

Öyle bir k ve n_0 sabitleri vardır ki A algoritması $n \geq n_0$ boyutunda bir problemi çözmek için $k \cdot f(n)$ den daha fazla zamana ihtiyaç duymaz ise A algoritmasının mertebesi $O(f(n))$ ile gösterilir.

Algoritmanın mertebesi

- Eğer bir algoritma n elemanlı bir problem için $n^2 - 3*n + 10$ saniye gerektiriyorsa ve öyle bir k ve n_0 değerleri vardır ki;

$$\text{bütün } n \geq n_0 \text{ için } k*n^2 > n^2 - 3*n + 10$$

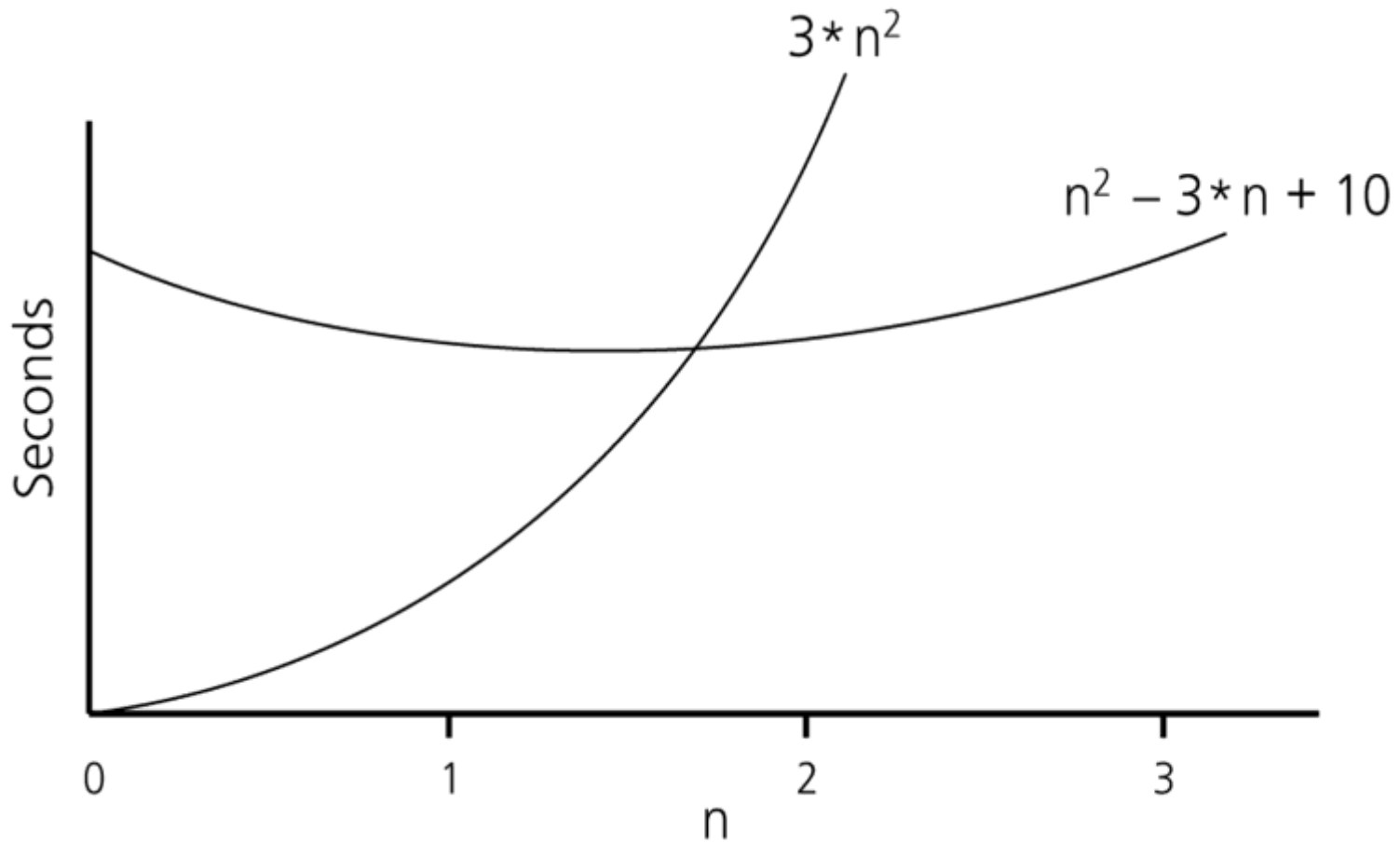
ve algoritmanın mertebesi n^2 olur. (Gerçekten $k=3$ ve $n_0=2$)

$$\text{Bütün } n \geq 2 \text{ için } 3*n^2 > n^2 - 3*n + 10 \text{ olur.}$$

Yani algoritma ($n \geq n_0$) için, $k*n^2$ den daha fazla zamana ihtiyaç duymaz.

ve böylece **$O(n^2)$** ile ifade edilir.

Algoritmanın Mertebesi (derecesi)



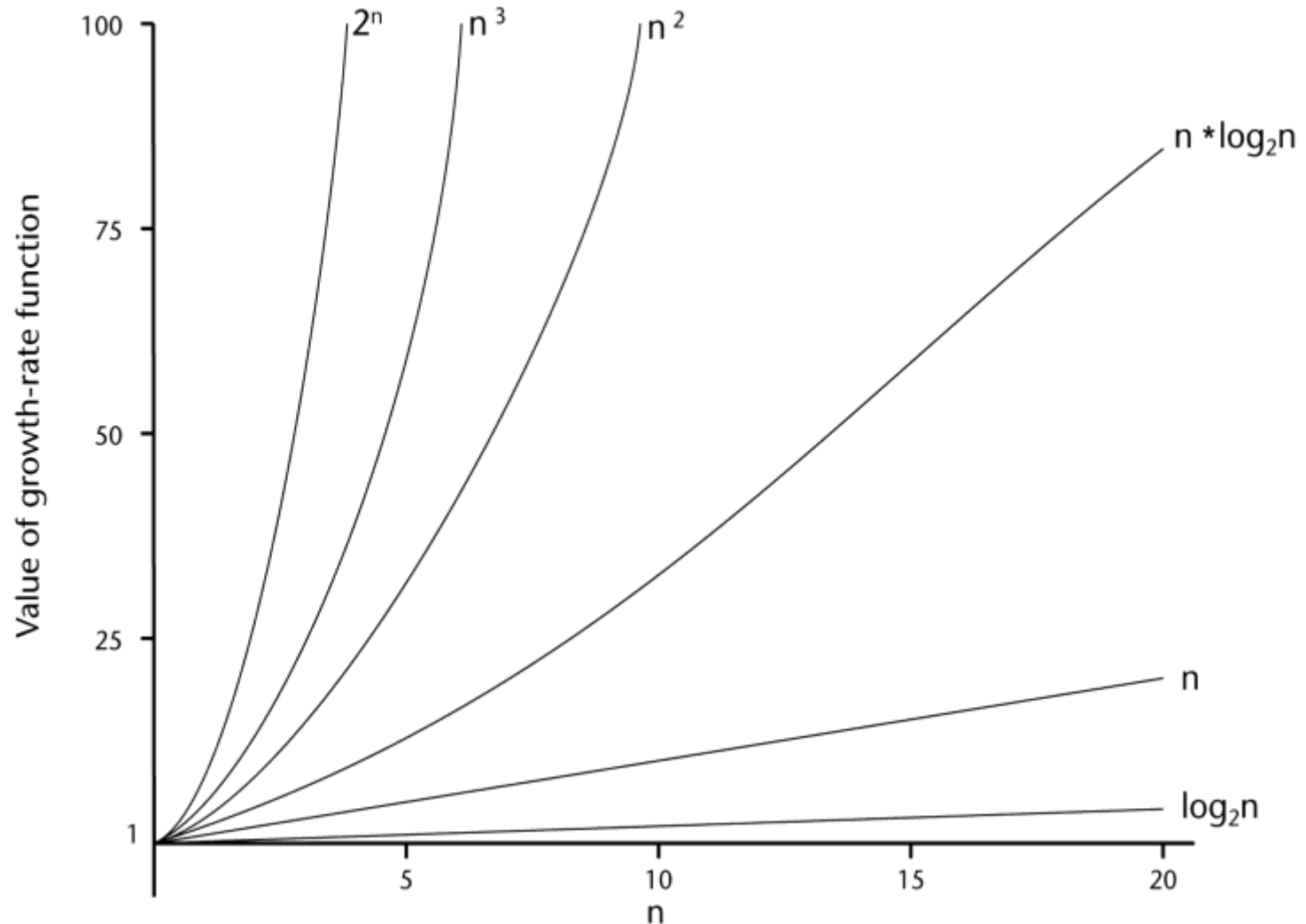
Büyüme hızı fonksiyonlarının karşılaştırılması

(a)

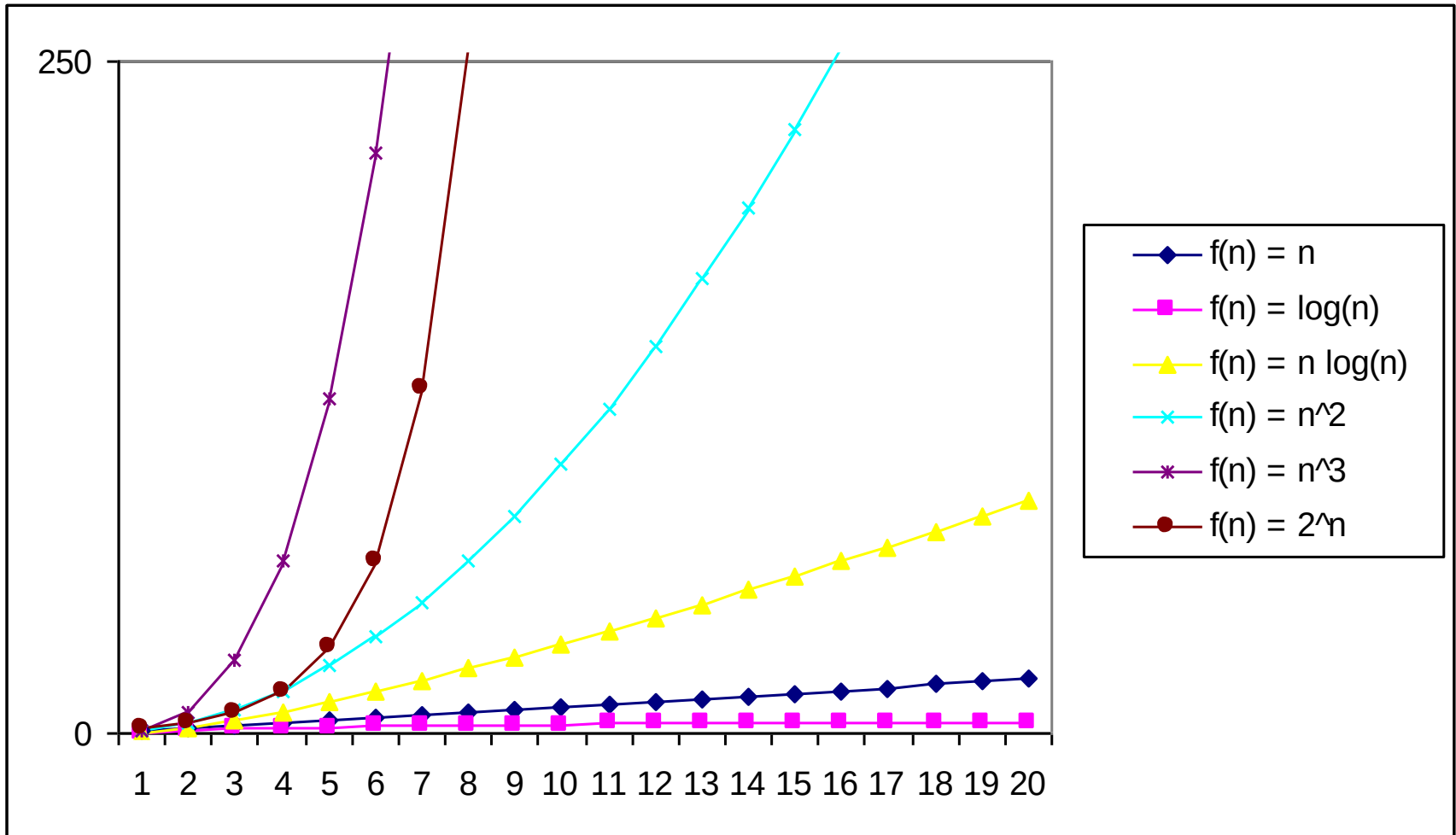
Function	n					
	10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n * \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

Büyüme hızı fonksiyonlarının karşılaştırılması

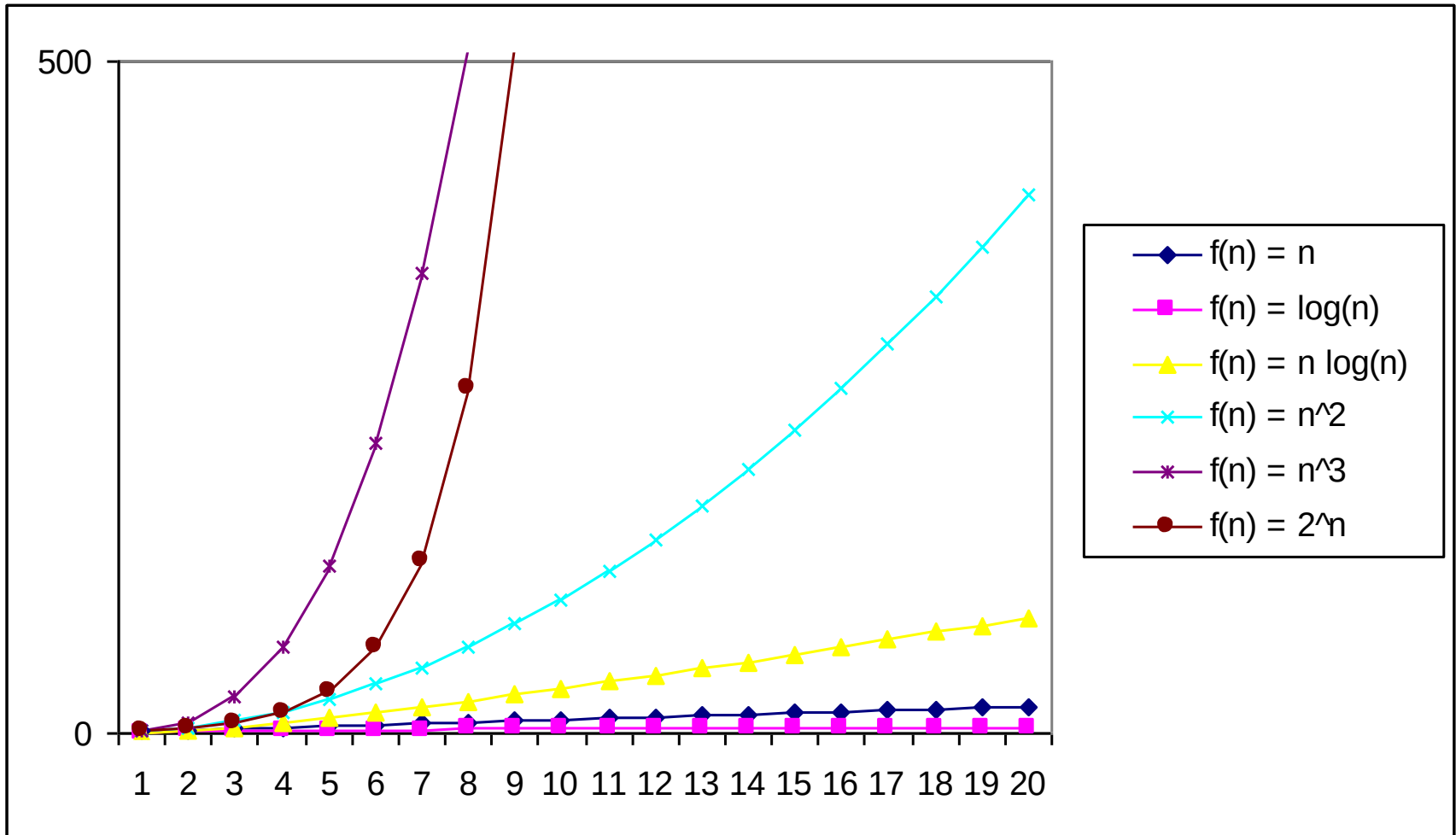
(b)



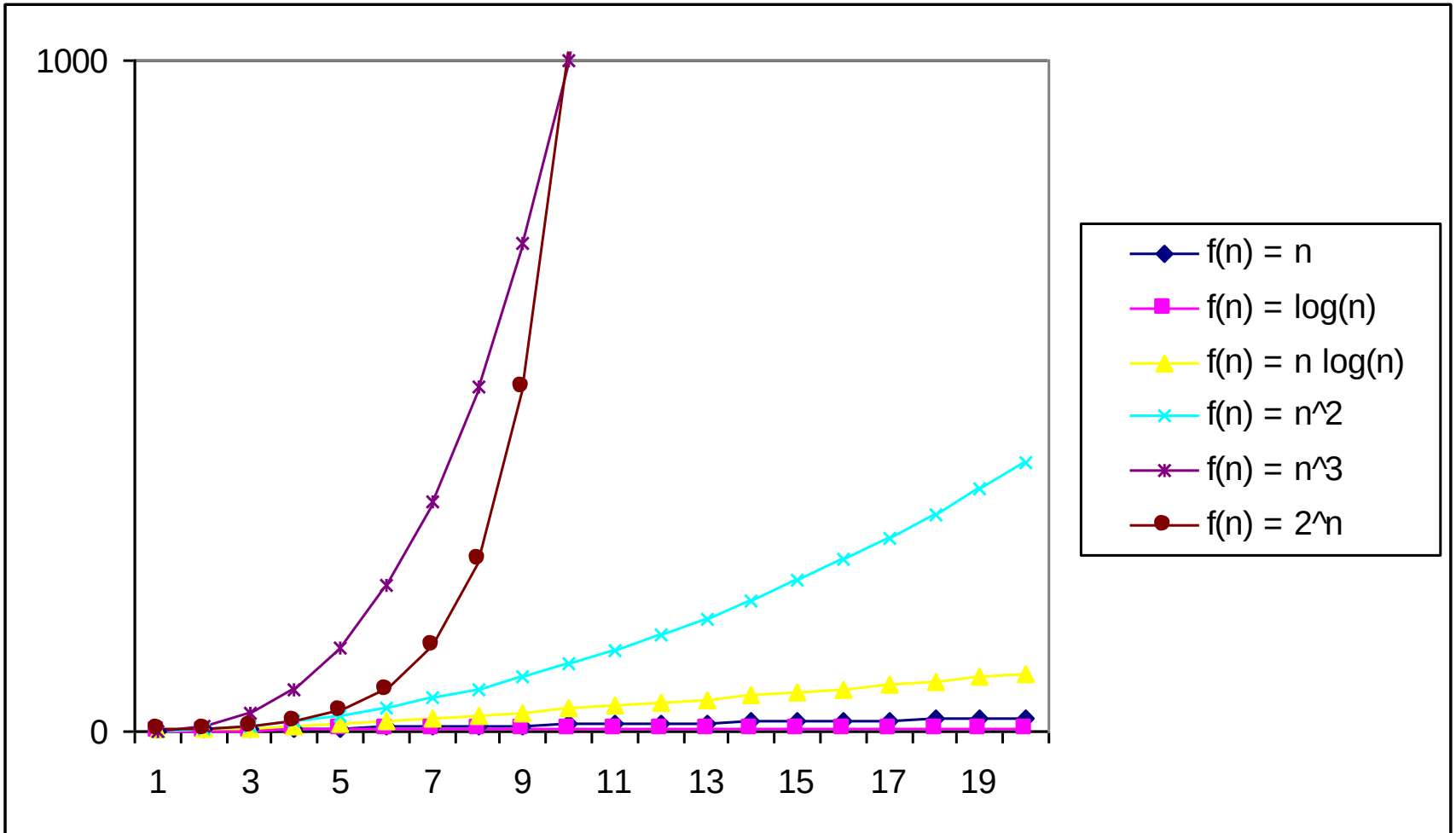
Karmaşıklık



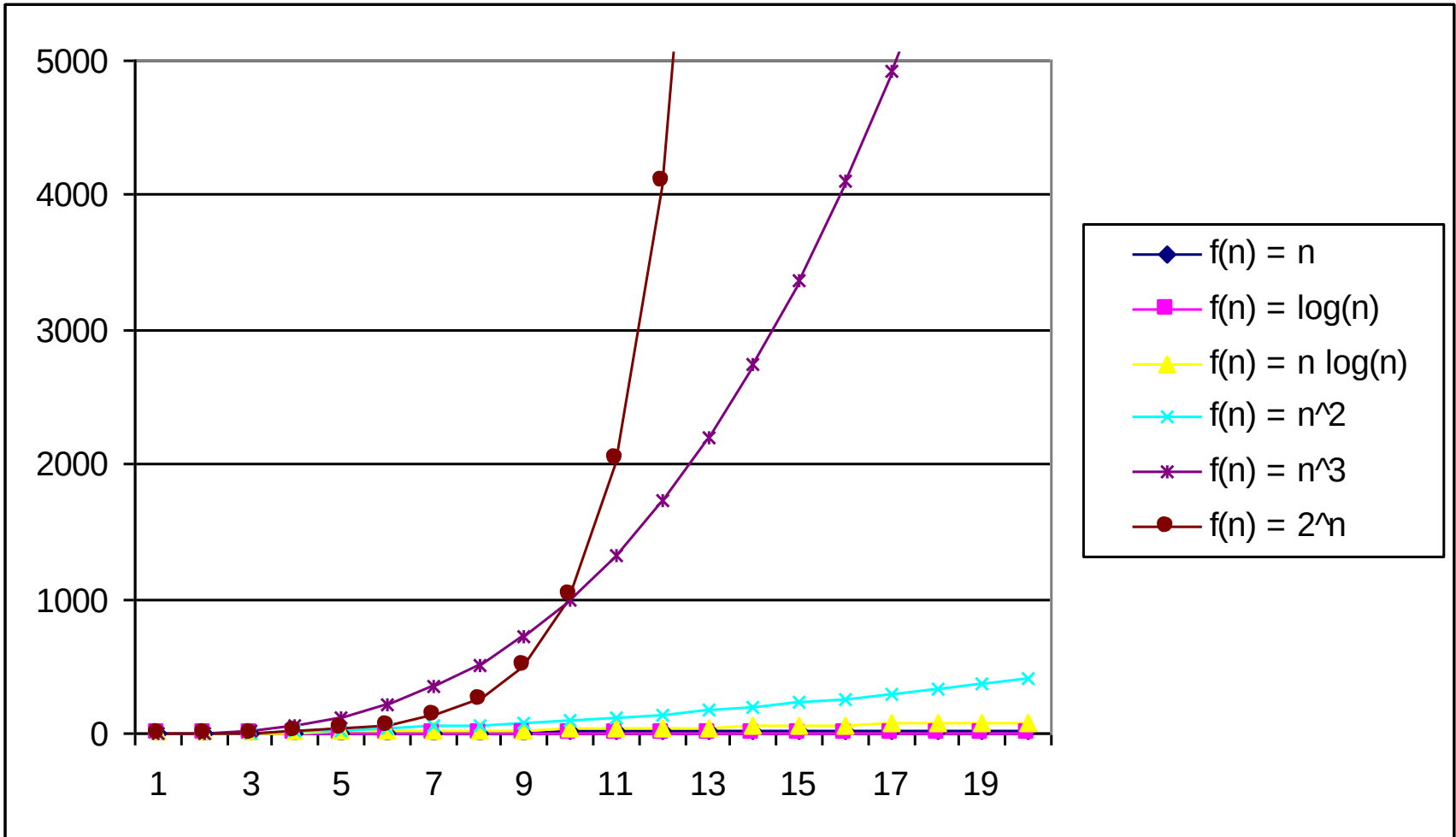
Karmaşıklık



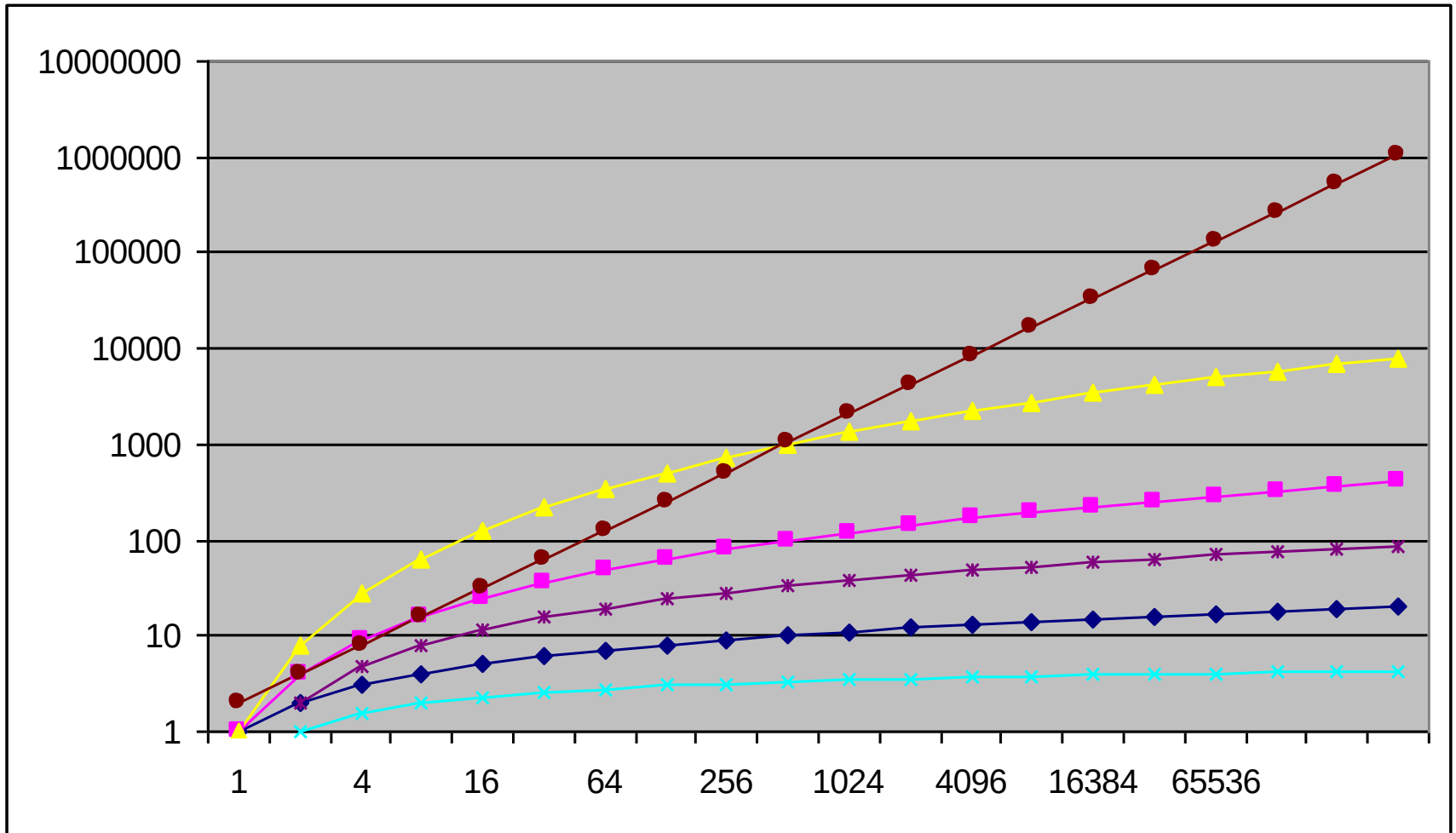
Karmaşıklık



Karmaşıklık



Karmaşıklık



Büyüme hızı fonksiyonları

- $O(1)$** Zaman gereksinimi **sabittir** ve problem boyutundan bağımsızdır.
- $O(\log_2 n)$** Zaman gereksinimi **logaritmiktir** ve problem boyutuna göre yavaş artar.
- $O(n)$** Zaman gereksinimi **doğrusaldır** ve problem girişiyle doğru orantılı artar.
- $O(n \cdot \log_2 n)$** Zaman gereksinimi **$n \cdot \log_2 n$** dir ve **doğrusaldan** daha hızlı artar.
- $O(n^2)$** Zaman gereksinimi **karesel** olup problem boyutuna göre hızlı bir artış gösterir.
- $O(n^3)$** Zaman gereksinimi **cubic** problem boyutuna göre hızlı bir artış gösterir.
- $O(2^n)$** problem girdi boyutu artarken zaman üstel (çok çok hızlı) olarak artar.

Büyüme hızı fonksiyonları

- Bir algoritma 8 elemanlı bir problemi 1 saniyede sonuçlandırıyor, 16 elemanlı bir problem için ne kadar zaman gerekir.
- Algoritmanın mertebesi:

$$O(1) \rightarrow T(n) = 1 \text{ saniye}$$

$$O(\log_2 n) \rightarrow T(n) = (1 * \log_2 16) / \log_2 8 = 4/3 \text{ saniye}$$

$$O(n) \rightarrow T(n) = (1 * 16) / 8 = 2 \text{ saniye}$$

$$O(n * \log_2 n) \rightarrow T(n) = (1 * 16 * \log_2 16) / 8 * \log_2 8 = 8/3 \text{ saniye}$$

$$O(n^2) \rightarrow T(n) = (1 * 16^2) / 8^2 = 4 \text{ saniye}$$

$$O(n^3) \rightarrow T(n) = (1 * 16^3) / 8^3 = 8 \text{ saniye}$$

$$O(2^n) \rightarrow T(n) = (1 * 2^{16}) / 2^8 = 2^8 \text{ saniye} = 256 \text{ saniye}$$

Büyüme hızı fonksiyonlarının Özellikleri

1. *Algoritmanın büyüme hızı fonksiyonundaki düşük dereceli terimleri yok sayabiliriz.*
 - $O(n^3+4n^2+3n)$, aynı zamanda $O(n^3)$ olarak ifade edilebilir.
 - Büyüme hızı fonksiyonu olarak sadece en yüksek derece kullanılabilir.
2. *Büyüme hızı fonksiyonundaki en yüksek dereceli terimin sabit çarpanını yok sayabiliriz.*
 - $O(5n^3)$, aynı zamanda $O(n^3)$ ile ifade edilir.
3. $O(f(n)) + O(g(n)) = O(f(n)+g(n))$
 - Büyüme hızı fonksiyonları birleştirilebilir.

Bazı Eşitlikler

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n * (n + 1)}{2} \approx \frac{n^2}{2}$$

$$\sum_{i=1}^n i^2 = 1 + 4 + \dots + n^2 = \frac{n * (n + 1) * (2n + 1)}{6} \approx \frac{n^3}{3}$$

$$\sum_{i=0}^{n-1} 2^i = 0 + 1 + 2 + \dots + 2^{n-1} = 2^n - 1$$

Örnek1

	<u>maliyet</u>	<u>Tekrar</u>
<code>i = 1;</code>	c1	1
<code>sum = 0;</code>	c2	1
<code>while (i <= n) {</code>	c3	n+1
<code>i = i + 1;</code>	c4	n
<code>sum = sum + i;</code>	c5	n
<code>}</code>		

$$\begin{aligned}T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*c5 \\&= (c3+c4+c5)*n + (c1+c2+c3) \\&= a*n + b\end{aligned}$$

➔ Algoritma büyüme hızı: **O(n)**

Örnek2

	<u>maliyet</u>	<u>Tekrar</u>
i=1;	c1	1
sum = 0;	c2	1
while (i <= n) {	c3	n+1
j=1;	c4	n
while (j <= n) {	c5	n*(n+1)
sum = sum + i;	c6	n*n
j = j + 1;	c7	n*n
}		
i = i + 1;	c8	n
}		

$$\begin{aligned}T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8 \\&= (c5+c6+c7)*n^2 + (c3+c4+c5+c8)*n + (c1+c2+c3) \\&= a*n^2 + b*n + c\end{aligned}$$

➔ Algoritmanın büyüme hızı fonksiyonu: **$O(n^2)$**

Örnek 3

	<u>maliyet</u>	<u>Tekrar</u>
for (i=1; i<=n; i++)	c1	n+1
for (j=1; j<=i; j++)	c2	$\sum_{j=1}^n (j+1)$
for (k=1; k<=j; k++)	c3	$\sum_{j=1}^n \sum_{k=1}^j (k+1)$
x=x+1;	c4	$\sum_{j=1}^n \sum_{k=1}^j k$
$T(n) = c1*(n+1) + c2*(\sum_{j=1}^n (j+1)) + c3*(\sum_{j=1}^n \sum_{k=1}^j (k+1)) + c4*(\sum_{j=1}^n \sum_{k=1}^j k)$		
$= a*n^3 + b*n^2 + c*n + d$		
➔ Algoritmanın büyüme hızı fonksiyonu: O(n³)		

Örnek:Sıralı arama

```
int sequentialSearch(const int a[], int item, int n){  
    for (int i = 0; i < n && a[i] != item; i++);  
    if (i == n)  
        return -1;  
    return i;  
}
```

Aranan eleman bulunamadı: → $O(n)$

Aranan eleman bulundu:

Best-Case: Aranan eleman dizinin ilk elemanı → $O(1)$

Worst-Case: Aranan eleman dizinin son elemanı → $O(n)$

Average-Case: Karşılaştırma sayısı, 1, 2, ..., n

$$\frac{\sum_{i=1}^n i}{n} = \frac{(n^2 + n)/2}{n} \rightarrow O(n)$$

İkili arama - Binary Search

```
int binarySearch(int a[], int size, int x) {
    int low = 0;
    int high = size - 1;
    int mid;           // mid will be the index of
                       // target when it's found.
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] < x)
            low = mid + 1;
        else if (a[mid] > x)
            high = mid - 1;
        else
            return mid;
    }
    return -1;
}
```

İkili Arama Analiz

- Aranan eleman bulunamadı:
 - Döngüdeki adım sayısı: $\lfloor \log_2 n \rfloor + 1$
 $\rightarrow O(\log_2 n)$
- Aranan eleman bulundu:
 - **Best-Case:** tek adımda bulunur. $\rightarrow O(1)$
 - **Worst-Case:** Adım sayısı: $\lfloor \log_2 n \rfloor + 1$ $\rightarrow O(\log_2 n)$
 - **Average-Case:** Adım sayısı $< \log_2 n$ $\rightarrow O(\log_2 n)$

0 1 2 3 4 5 6 7 $\leftarrow$ 8 elemanlı bir dizi

3 2 3 1 3 2 3 4 $\leftarrow$ # adımlar

Ortalama adım sayısı = $21/8 < \log_2 8$

<u>n</u>	<u>$O(\log_2 n)$</u>
16	4
64	6
256	8
1024 (1KB)	10
16,384	14
131,072	17
262,144	18
524,288	19
1,048,576 (1MB)	20
1,073,741,824 (1GB)	30

Yinelemeli (Iterative) Algoritmalar

Analiz

ADD-1(n : nonnegative integer)

```
1   $a \leftarrow 0$   
2  for  $i \leftarrow 1$  to  $n$   
3       $a \leftarrow a + i$   
4  return  $a$ 
```

$$\sum_{i=1}^n 1 = n \quad \Theta(n)$$

ADD-3(n : nonnegative integer)

```
1   $a \leftarrow 0$   
2  for  $i \leftarrow 1$  to  $n$   
3      for  $j \leftarrow 1$  to  $n$   
4           $a \leftarrow a + 1$   
5  return  $a$ 
```

$$\sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n n = n^2 \quad \Theta(n^2)$$

ADD-GENERALISED(n : nonnegative integer)

1 for $i_1 \leftarrow 1$ to n

2 for $i_2 \leftarrow 1$ to n

3 ...

4 for $i_k \leftarrow 1$ to n

5 expression $\Theta(1)$

$\Theta(n^k)$

ADD-4(n : nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      for j ← i to n
4          a ← a + 1
5  return a
```

$$\sum_{i=1}^n \sum_{j=i}^n 1 = \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n 1}_n - \underbrace{\sum_{j=1}^{i-1} 1}_{i-1} \right) = \sum_{i=1}^n (n - i + 1) =$$

$$\sum_{i=1}^n (n + 1) - \sum_{i=1}^n i = n(n + 1) - \frac{n(n + 1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n = \Theta(n^2)$$

ADD-5(*n*: nonnegative integer)

```
1  a ← 0
2  for i ← 1 to n
3      for j ← i + 1 to n
4          a ← a + 1
5  return a
```

$$\sum_{i=1}^n \sum_{j=i+1}^n 1 = \sum_{i=1}^n \left(\underbrace{\sum_{j=1}^n 1}_n - \underbrace{\sum_{j=1}^i 1}_i \right) = \sum_{i=1}^n (n - i) = \sum_{i=1}^n (n) - \sum_{i=1}^n i =$$

$$n^2 - \frac{n(n+1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n = \Theta(n^2)$$

A2(n: positive integer)

```

1  a ← 0
2  for i ← 1 to n-1
3      for j ← i+1 to n
4          for k ← 1 to j
5              a ← a + 1
6  return a

```

$$\begin{aligned}
 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \sum_{k=1}^j 1 &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n j = \sum_{i=1}^{n-1} \left(\sum_{j=1}^n j - \sum_{j=1}^i j \right) \\
 &= \sum_{i=1}^{n-1} \left(\frac{1}{2}n(n+1) - \frac{1}{2}i(i+1) \right) = \sum_{i=1}^{n-1} \left(\frac{1}{2}n(n+1) \right) - \frac{1}{2} \sum_{i=1}^{n-1} (i^2 + i)
 \end{aligned}$$

$$\begin{aligned}
 &= \frac{1}{2}n(n+1)(n-1) - \frac{1}{2} \sum_{i=1}^{n-1} i^2 - \frac{1}{2} \sum_{i=1}^{n-1} i \\
 &= \frac{1}{2}n(n-1)(n+1) - \frac{1}{12}n(n-1)(2n-1) - \frac{1}{4}n(n-1) \\
 &= \frac{1}{2}n(n-1) \left(n+1 - \frac{1}{6}(2n-1) - \frac{1}{2} \right) \\
 &= \frac{1}{12}n(n-1)(6n+3-2n+1) = \frac{1}{12}n(n-1)(4n+4) = \frac{1}{3}n(n-1)(n+1)
 \end{aligned}$$

$\Theta(n^3)$

$$\sum_{i=1}^n i^2 = \frac{1}{6}n(n+1)(2n+1)$$

$$\sum_{i=1}^{n-1} i^2 = \frac{1}{6}(n-1)n(2n-1)$$

$$\sum_{i=1}^{n-1} i = \frac{1}{2}n(n-1)$$

Verilen Algoritmanın çalışma zamanı büyüme hızı nedir?

P 1($a_1, a_2, \dots, a_n$: array of positive integers)

1 $s \leftarrow 0$

2 **for** $i \leftarrow 1$ **to** $n - 4$

3 **for** $j \leftarrow i$ **to** $i + 4$

4 **for** $k \leftarrow i$ **to** j

5 $s \leftarrow s + a_i$

En dıştaki döngü $n-4$ defa çalışır.

Ortadaki döngü tam olarak 5 defa çalışır.

En içteki döngü j 'nin $i, i + 1, i + 2, i + 3$ ve $i + 4$ 'e eşit değerleri için sırasıyla 1, 2, 3, 4 ve 5 kez ve toplam 15 kez yürütülür

$\Theta(n)$

P 1($a_1, a_2, \dots, a_n$: array of positive integers)

1 $s \leftarrow 0$

2 **for** $i \leftarrow 1$ **to** $n - 4$

3 **for** $j \leftarrow i$ **to** $i + 4$

4 **for** $k \leftarrow i$ **to** j

5 $s \leftarrow s + a_i$

$$\sum_{i=1}^{n-4} \sum_{j=1}^{i+4} \sum_{k=i}^j 1$$

$$\sum_{k=i}^j 1 = \begin{cases} j - i + 1, & \text{if } j \geq i \\ 0, & \text{else} \end{cases}$$

$$\sum_{i=1}^{n-4} \left(\underbrace{\sum_{j=1}^{i-1} \sum_{k=i}^j 1}_0 + \sum_{j=i}^{i+4} \sum_{k=i}^j 1 \right) = \sum_{i=1}^{n-4} \sum_{j=i}^{i+4} (j - i + 1) =$$

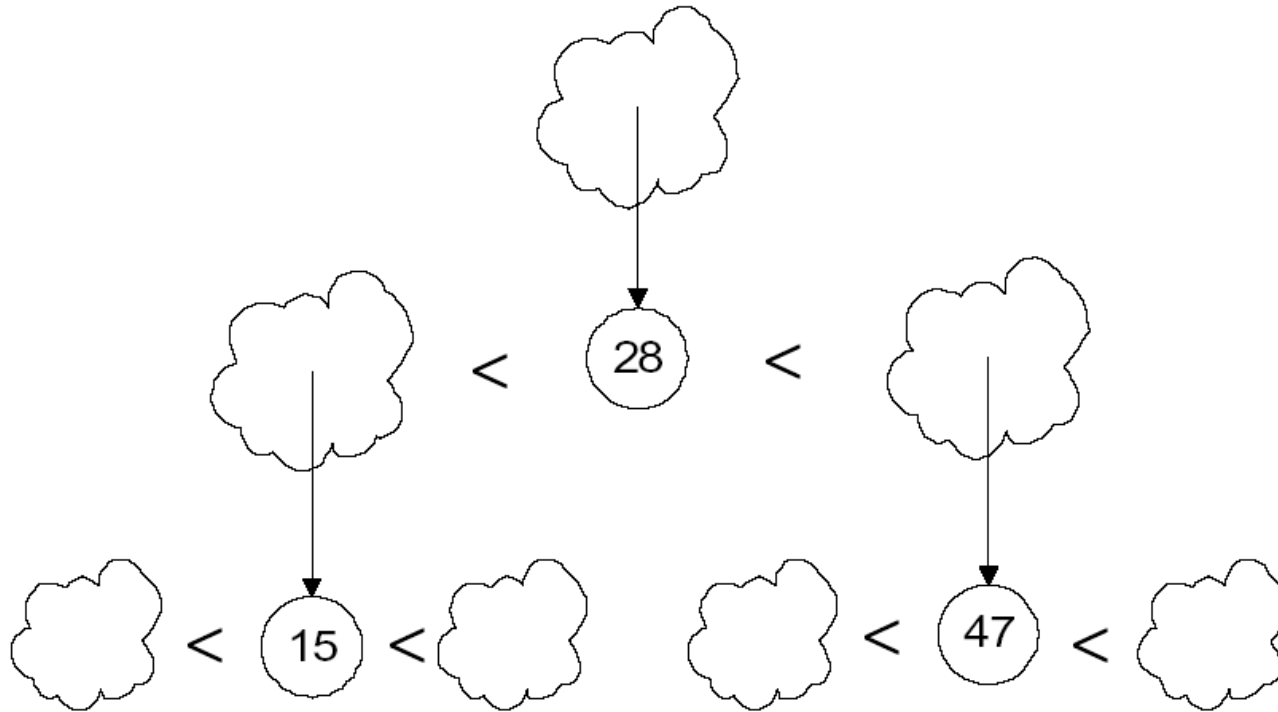
$$= \sum_{i=1}^{n-4} ((i - i + 1) + (i + 1 - i + 1) + (i + 2 - i + 1) + (i + 3 - i + 1) + (i + 4 - i + 1))$$

$$= \sum_{i=1}^{n-4} (1 + 2 + 3 + 4 + 5) = 15(n - 4)$$

Hızlı Sıralama (Quick-Sort)

- Bu algoritma Böl-Yönet yaklaşımına dayanmaktadır.

Hızlı Sıralama



Bir pivot eleman seçilir. Pivottan küçük olanlar ve büyük olanlar olarak dizi ikiye bölünür.
Herbir taraf özyineli olarak sıralanır.

Hızlı sıralama (Quicksort) Algoritması

Tamsayılardan oluşan n elemanlı bir dizi verilmiş olsun.

- If {dizi sadece bir elemandan oluşuyorsa} , return
- Else
 - *pivot* olarak kullanmak için bir eleman seç
 - Diziyi aşağıdaki gibi iki alt dizi halinde böl:
 - *Pivot*'a eşit yada pivottan küçük olanlar
 - *Pivottan* büyük olanlar
 - İki alt diziyi Hızlı sırala
 - Return sonuç dizi

Örnek

Verilen diziyi algoritmaya göre sıralayalım

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Pivot Eleman seç

- Pivot eleman değişik şekillerde seçilebilir. Bu örnekte ilk eleman pivot olarak seçilecektir.

40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

Diziyi bölme

- Pivottan küçük yada eşit olanlardan oluşan bir altdizi($\leq$ pivot)
- Pivottan büyük olan elemanlardan oluşan bir dizi($>$ pivot)
- **Altdiziler orijinal dizide tutulur.**
- **Bölme işlemini diziler parçalanamayacak hale gelene kadar rekürsif olarak tekrarla**

pivot_index = 0

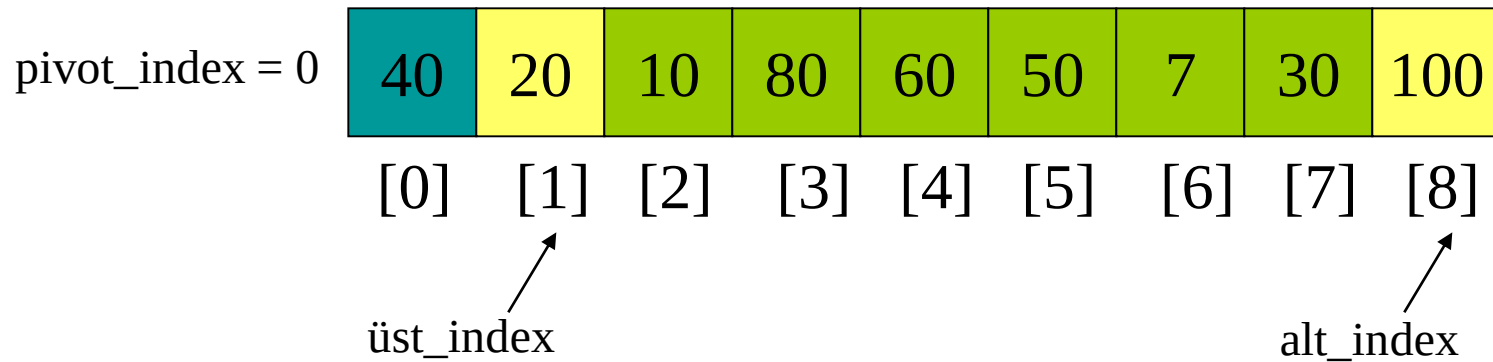
40	20	10	80	60	50	7	30	100
----	----	----	----	----	----	---	----	-----

[0] [1] [2] [3] [4] [5] [6] [7] [8]

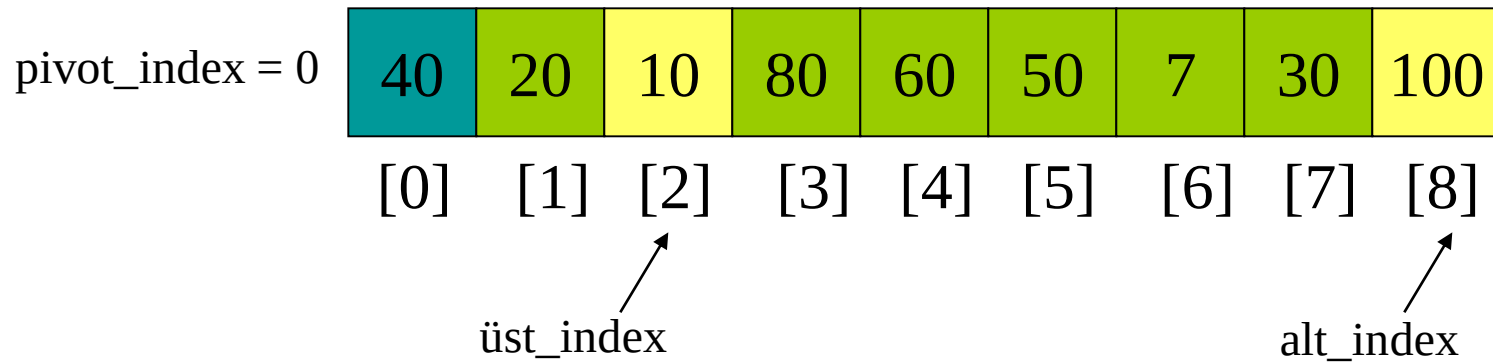
↑
üst_index

↑
alt_index

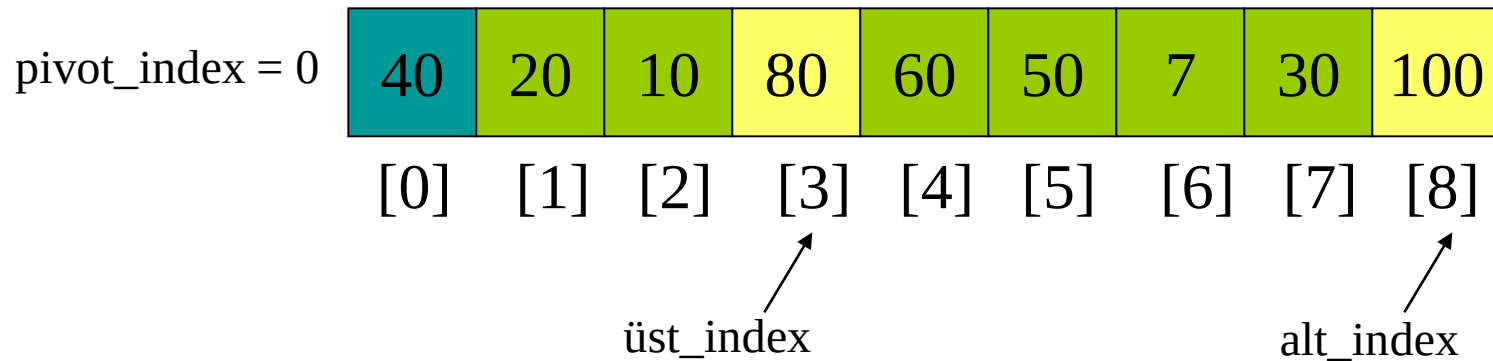
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 $++\text{üst_index}$



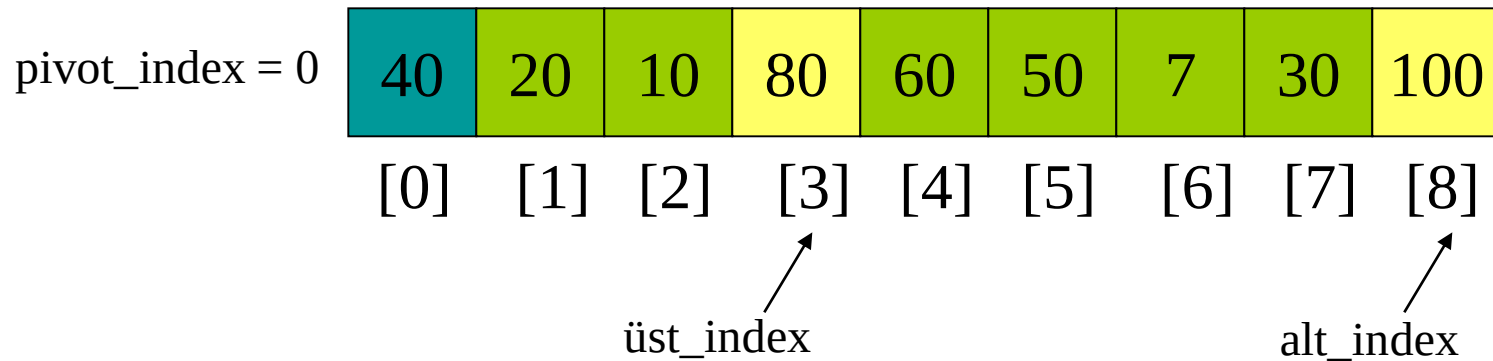
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 $++\text{üst_index}$



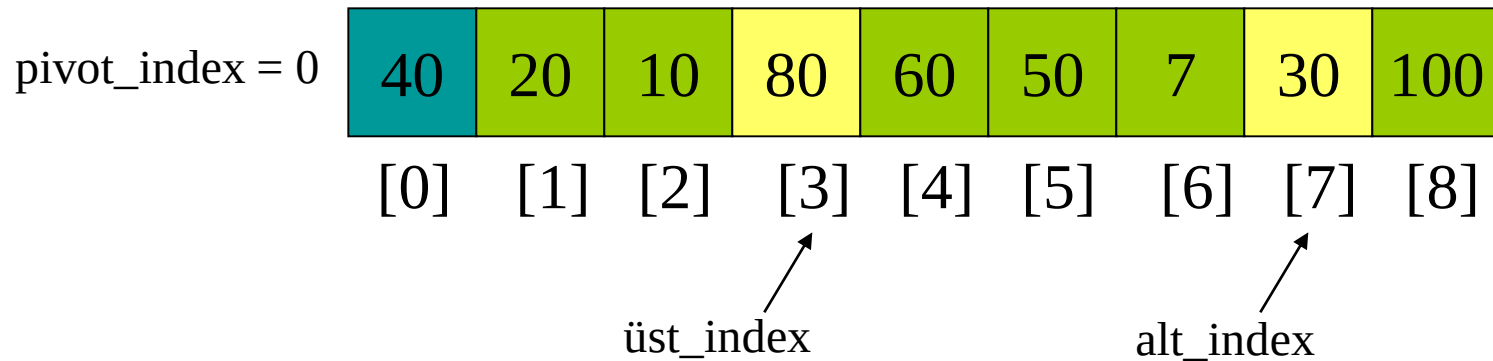
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 $++\text{üst_index}$



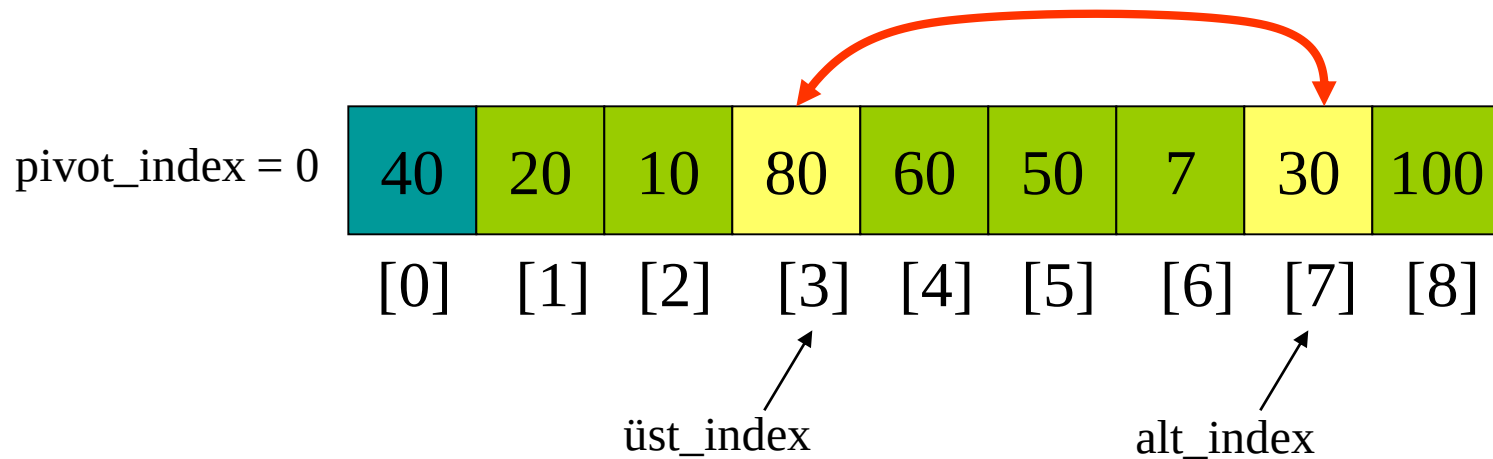
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index



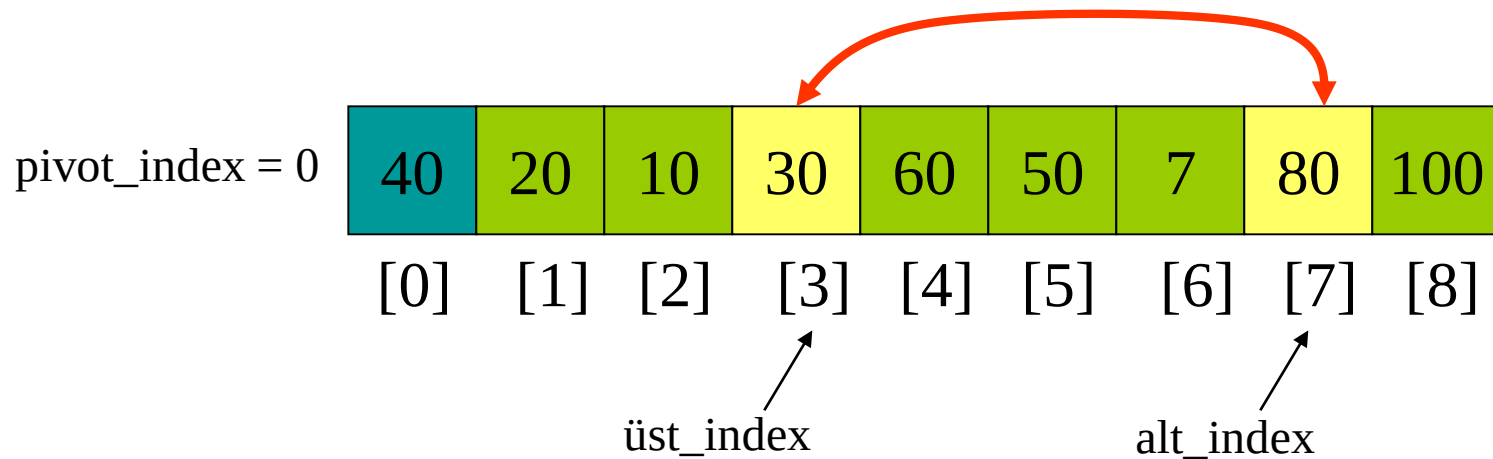
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index



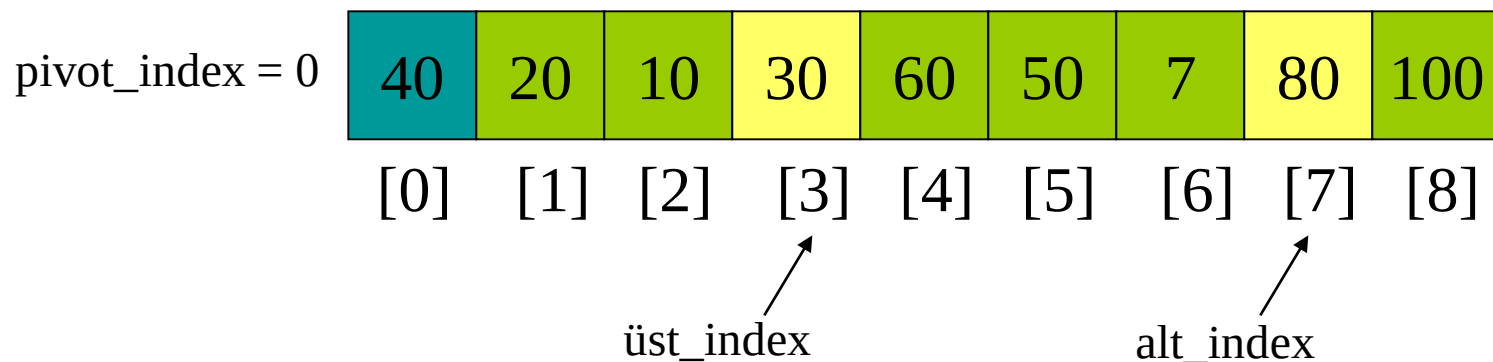
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$



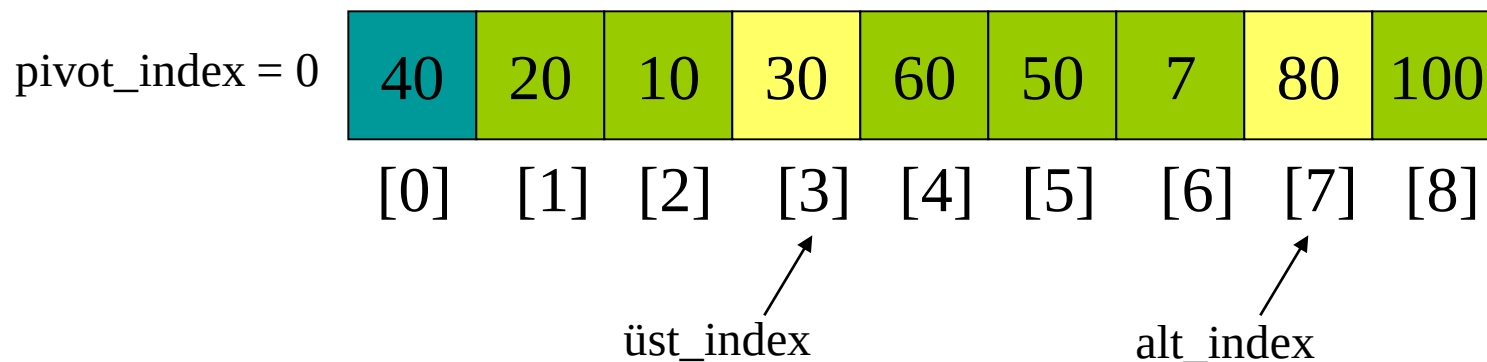
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$



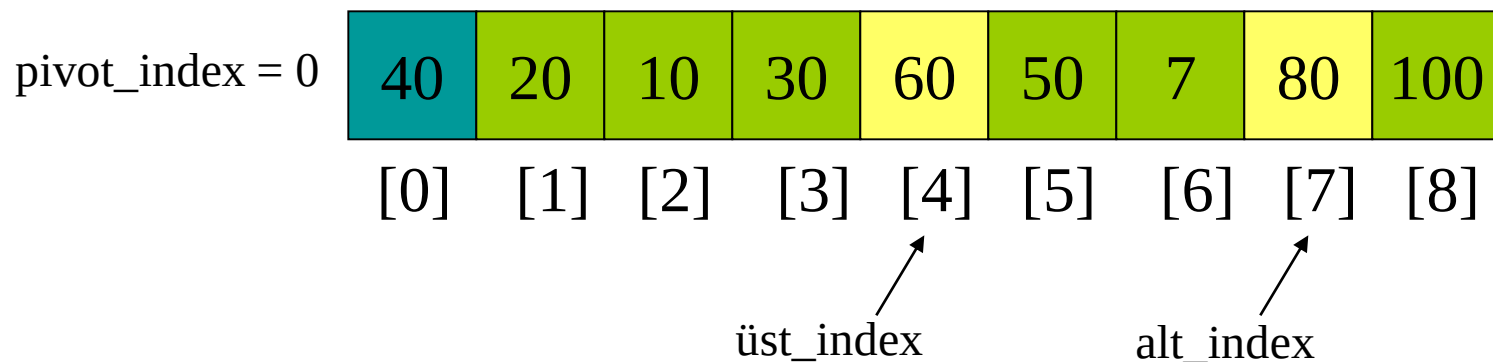
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



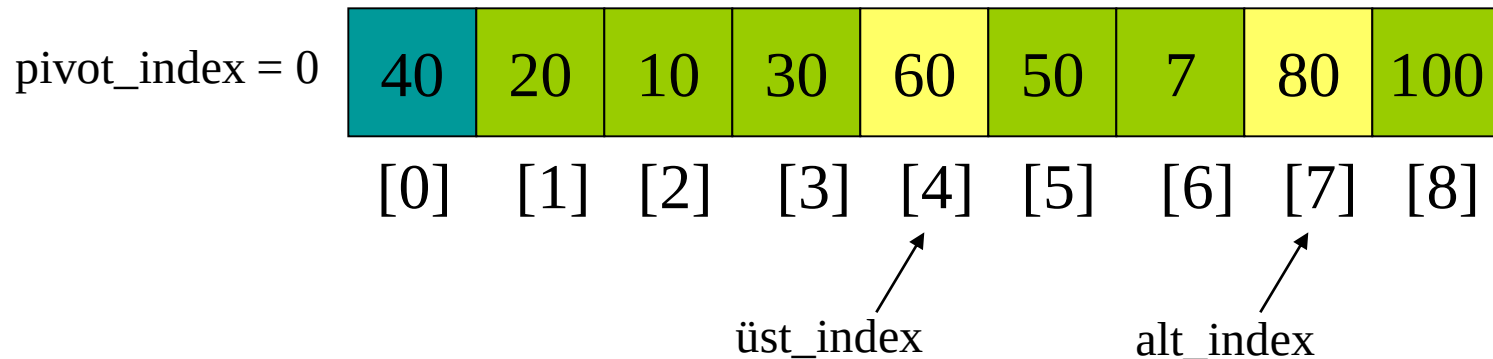
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



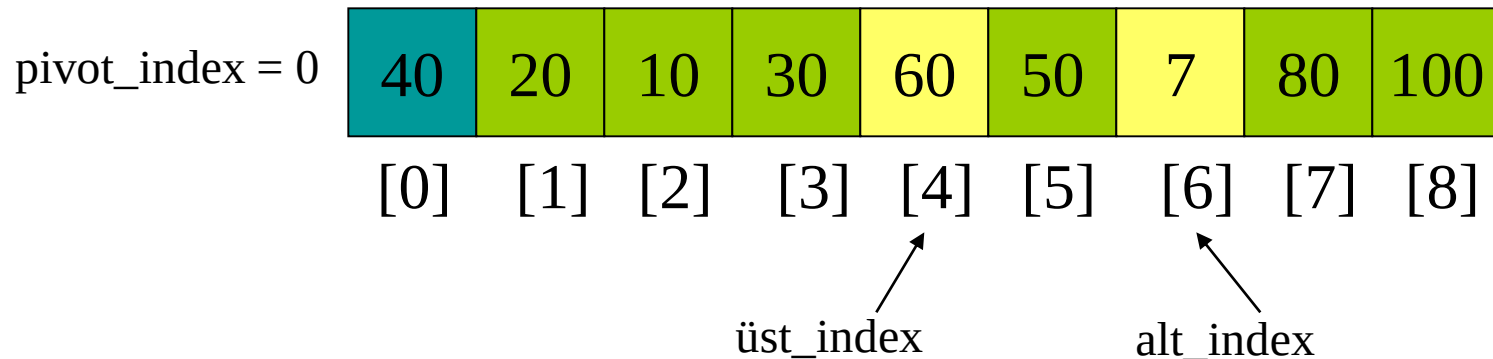
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



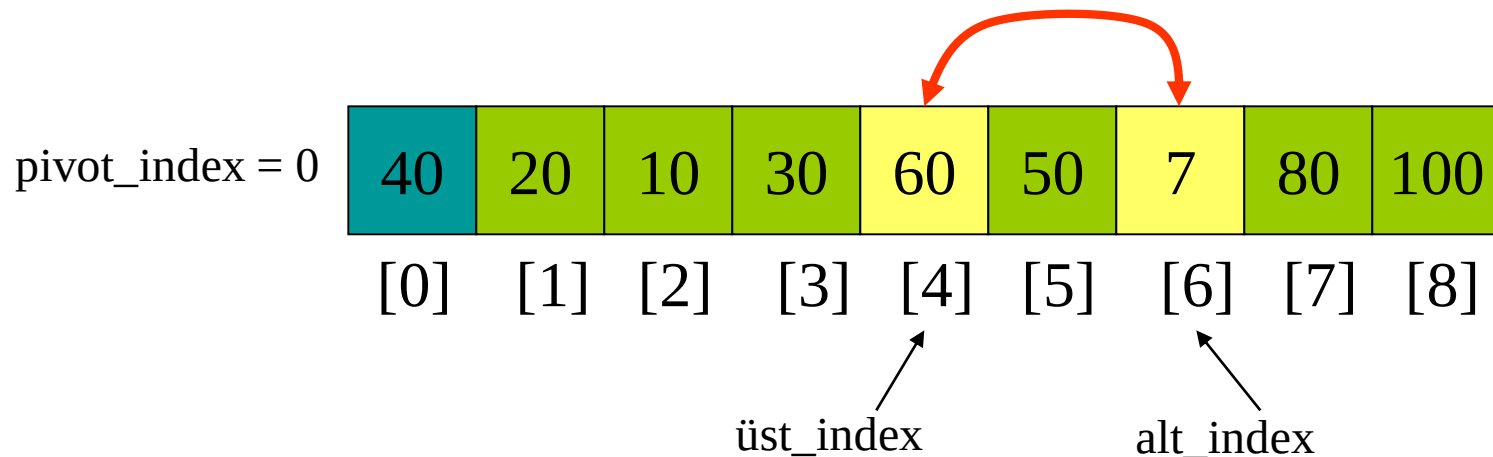
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



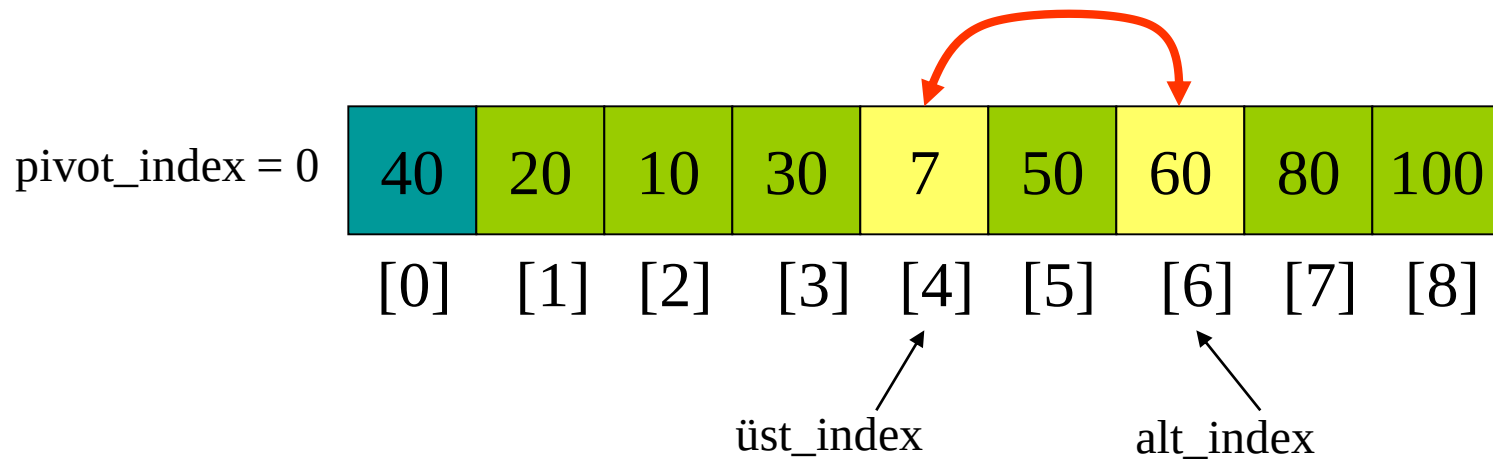
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



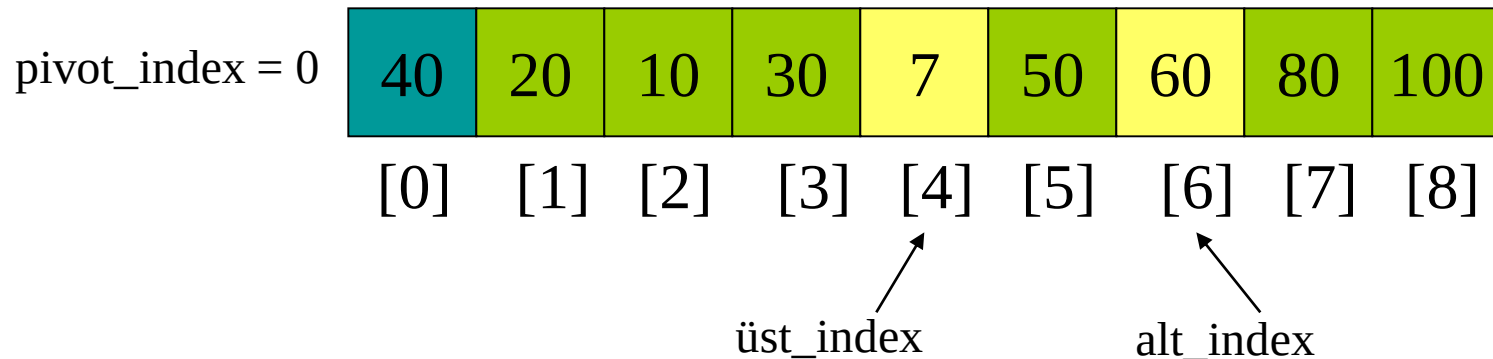
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



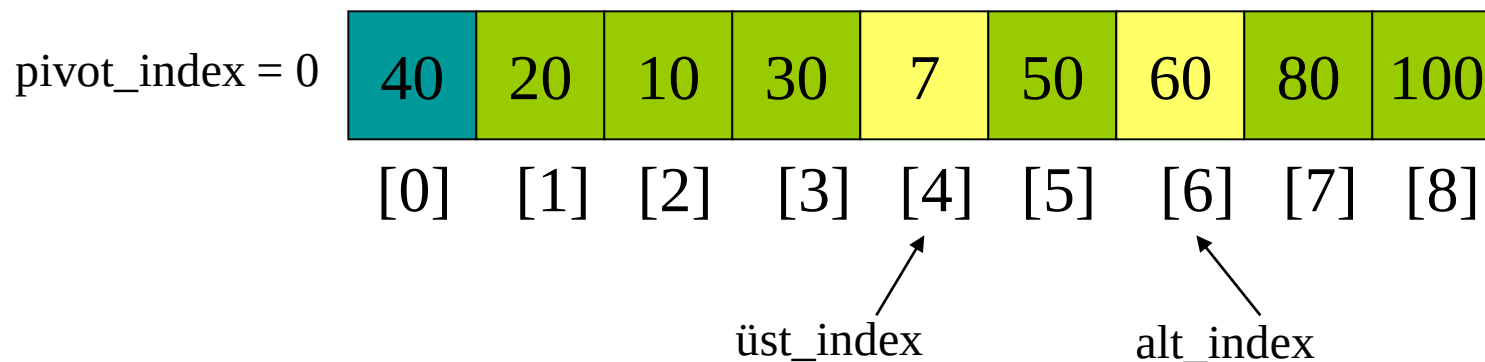
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



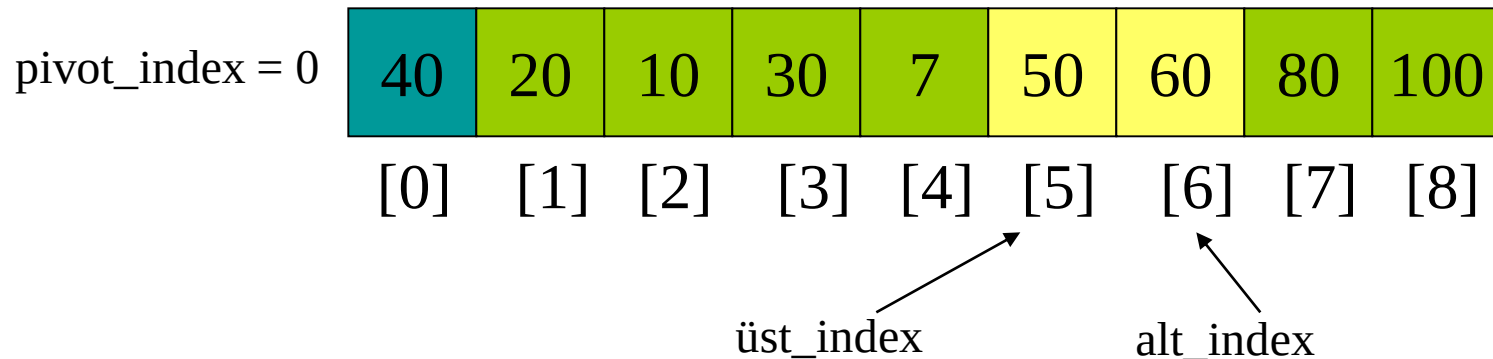
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.



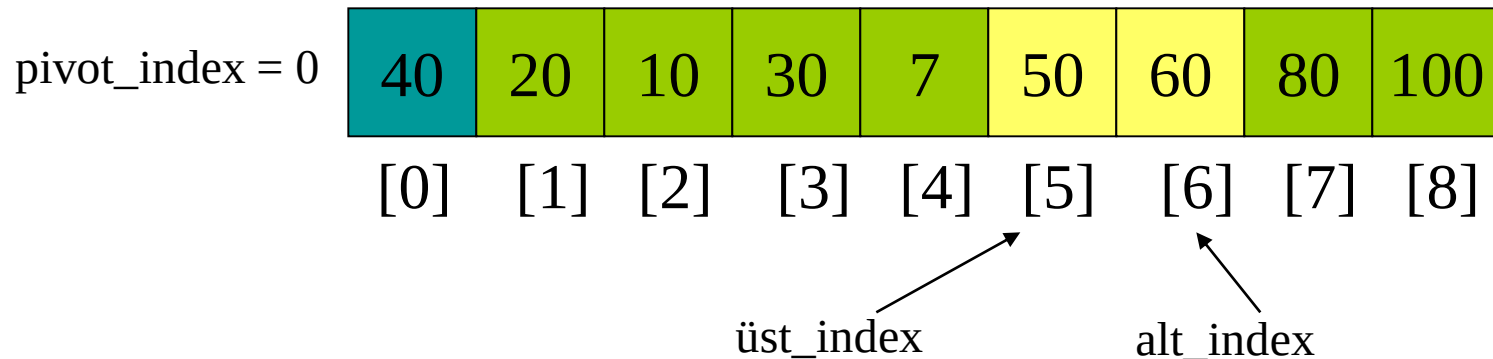
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



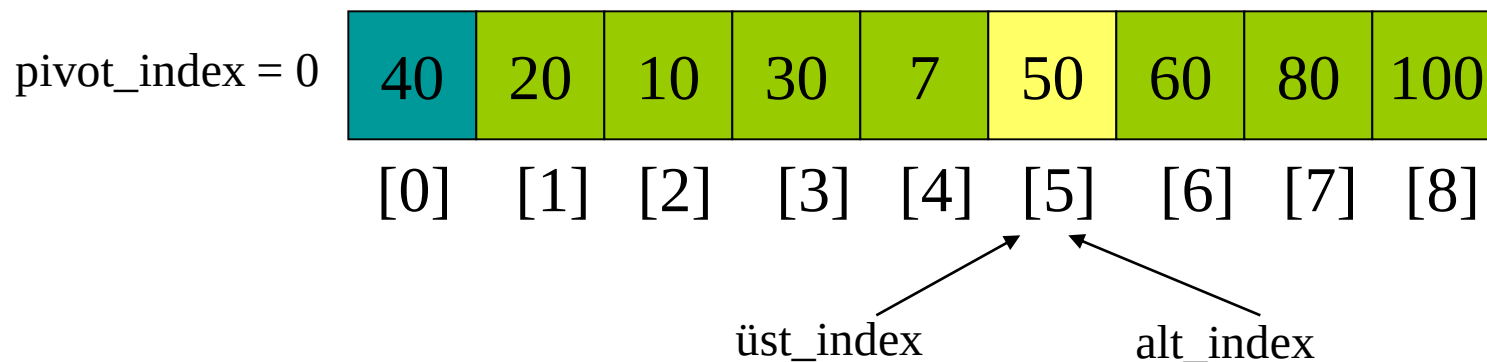
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



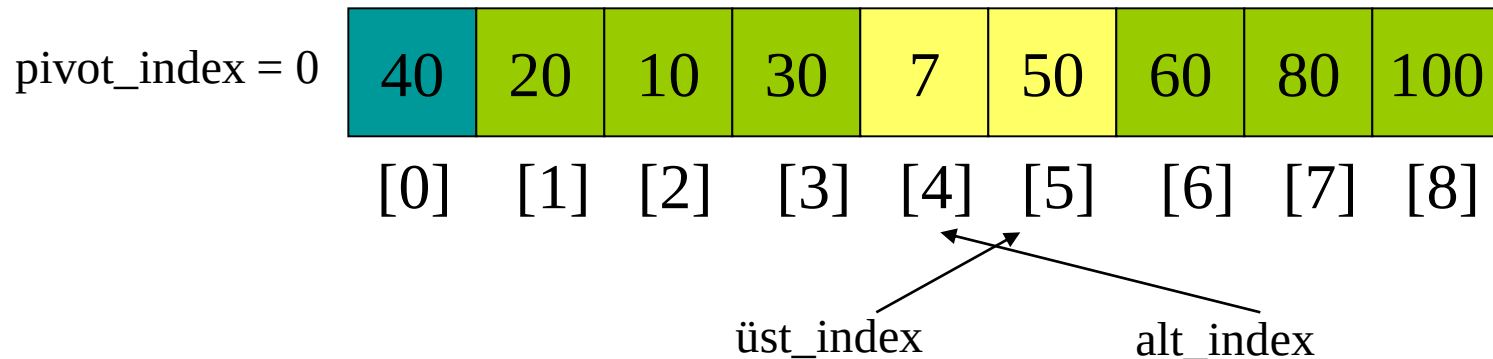
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



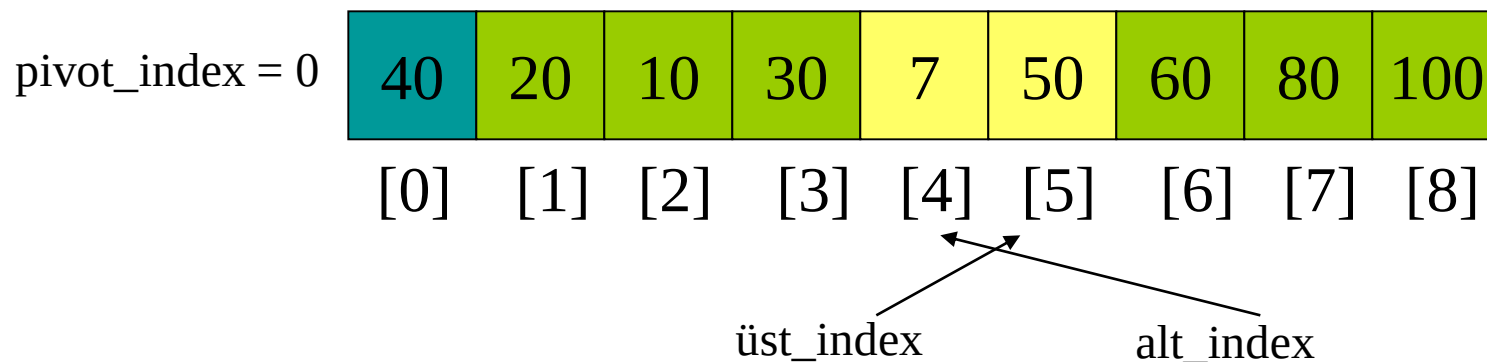
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



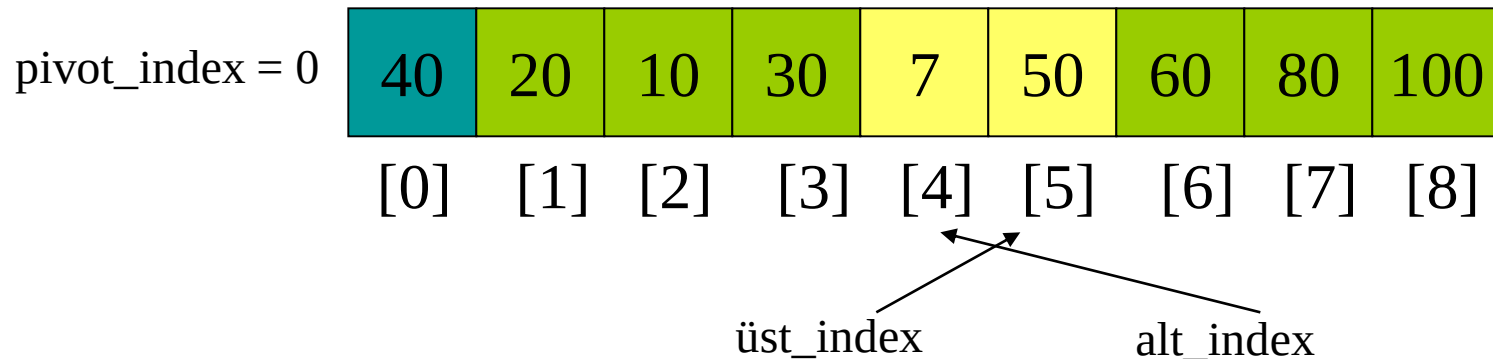
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



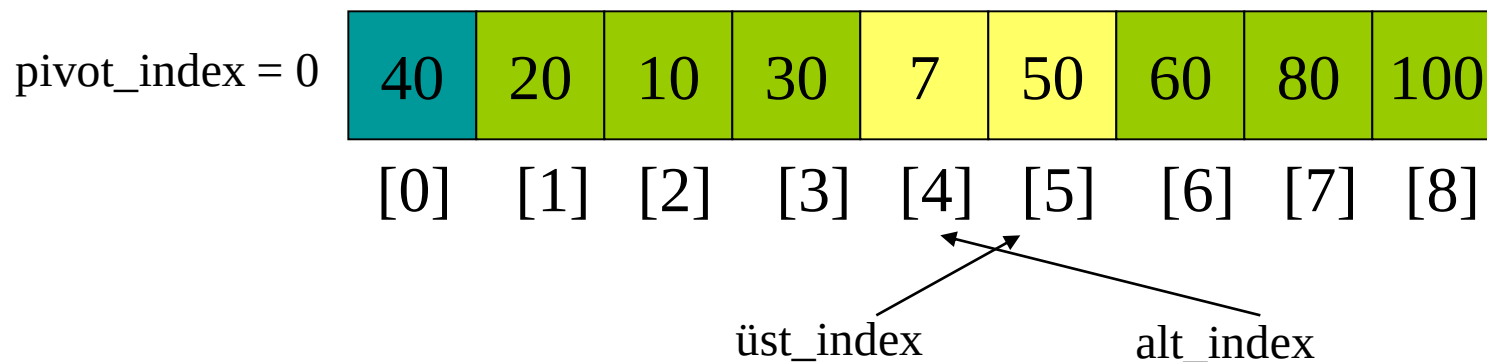
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.

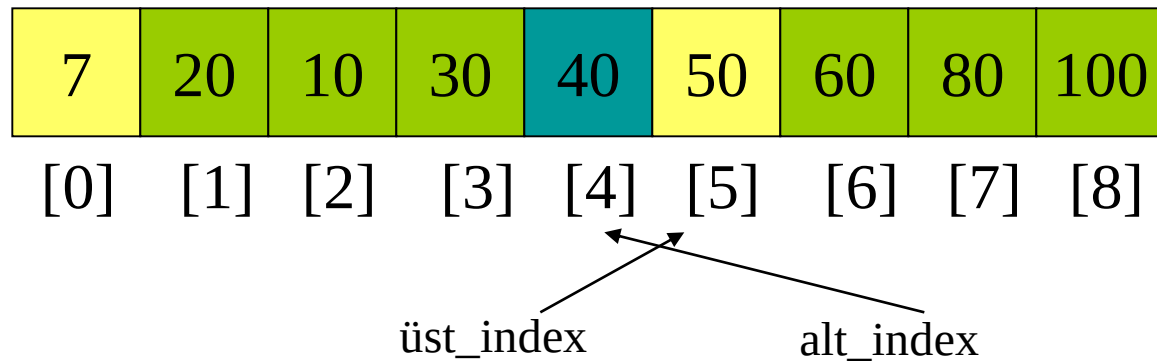


1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$

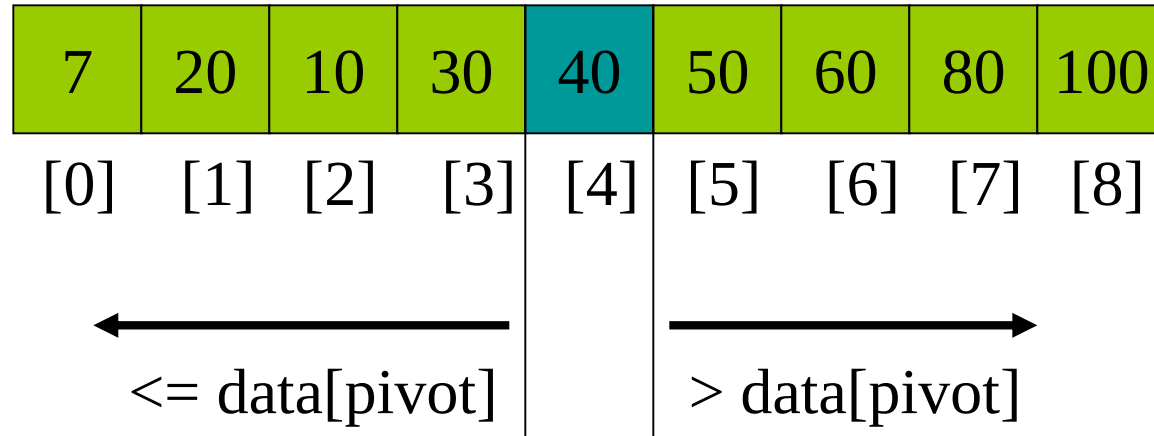


1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$

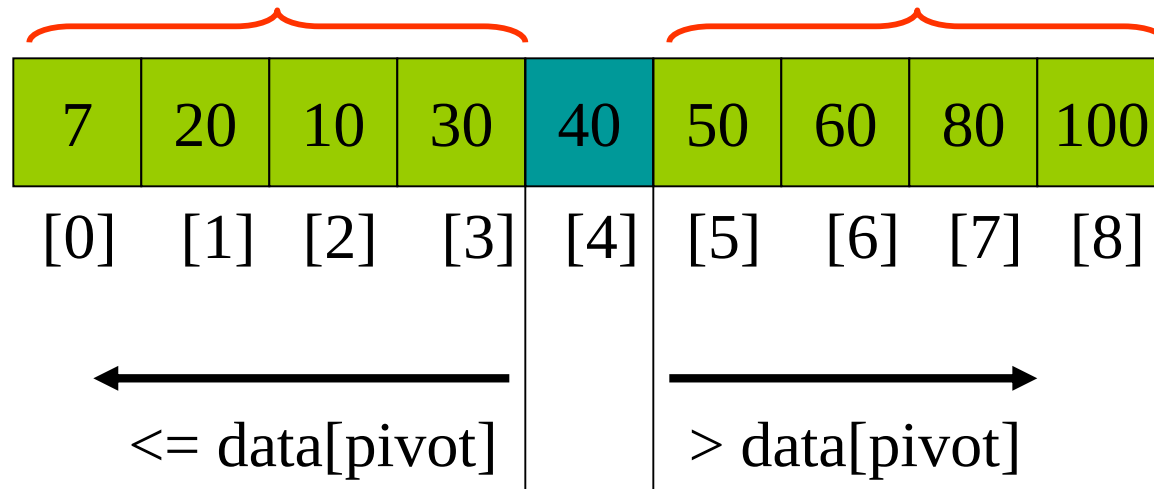
pivot_index = 4



Oluşturulan iki alt dizi

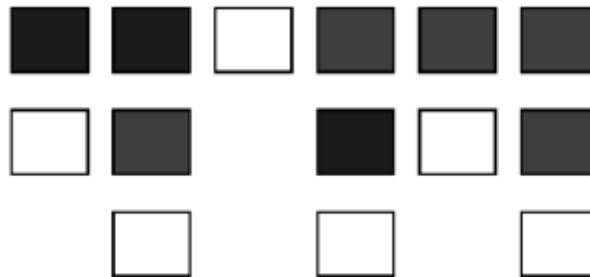


Recursion: Altdizilere Quicksort



Quicksort Algoritmasının Analizi

- Elemanlar düzensiz dağılmışsa,
- **“Best case” çalışma zamanı: $O(n \log_2 n)$**
 - Pivot eleman her zaman ortada
 - Her rekürsif çağrıda dizi iki eş parçaya ayrılmakta.



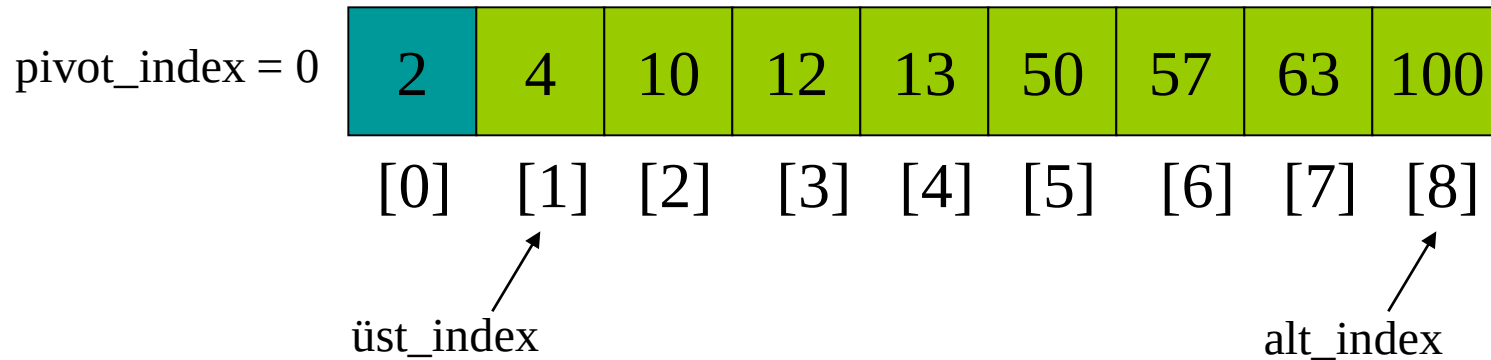
Quicksort Analysis

Worst case: $O(N^2)$

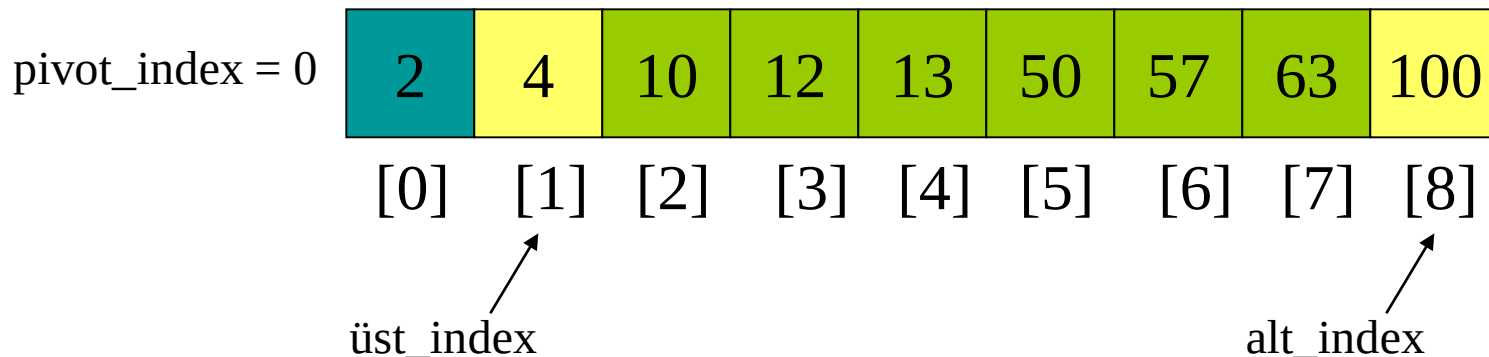
- Her rekürsif çağrıda pivot en küçük elemandır.
- Dizi zaten sıralı olması halidir.

Quicksort: Worst Case

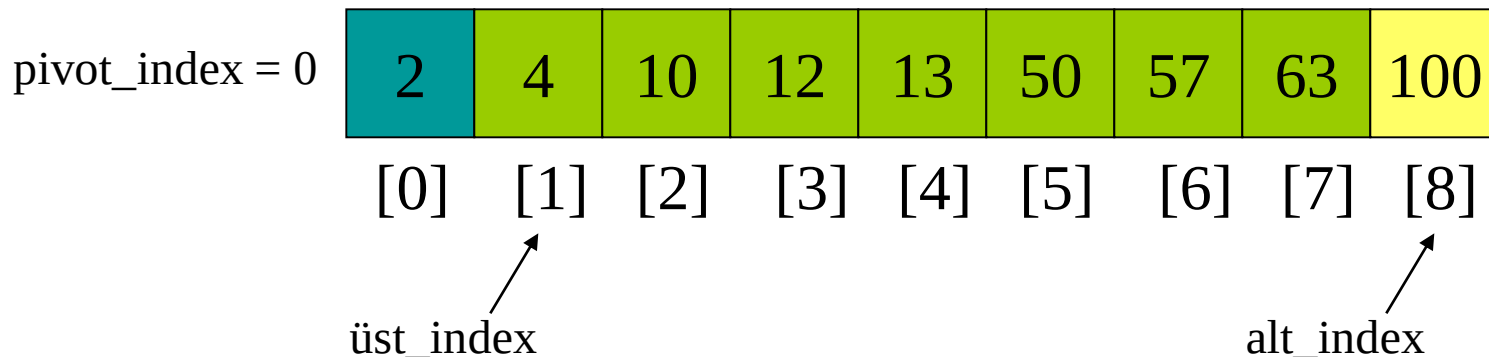
- İlk eleman pivot olarak seçilmiş olsun.
- Dizi zaten sıralı olsun



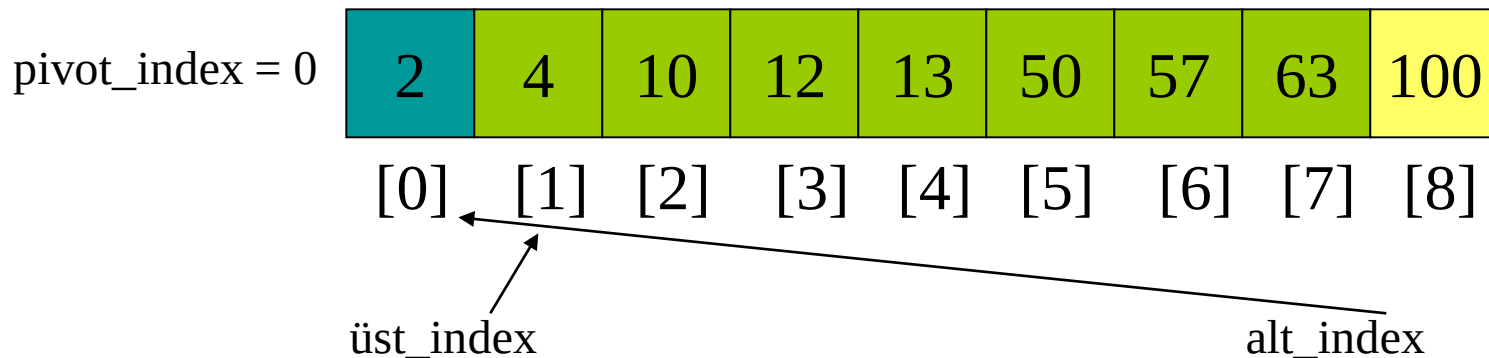
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



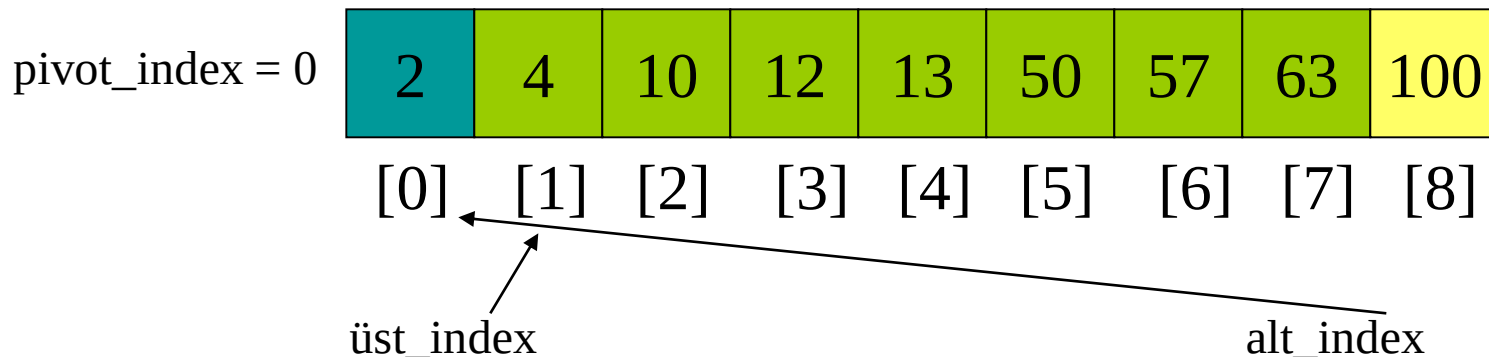
- 1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



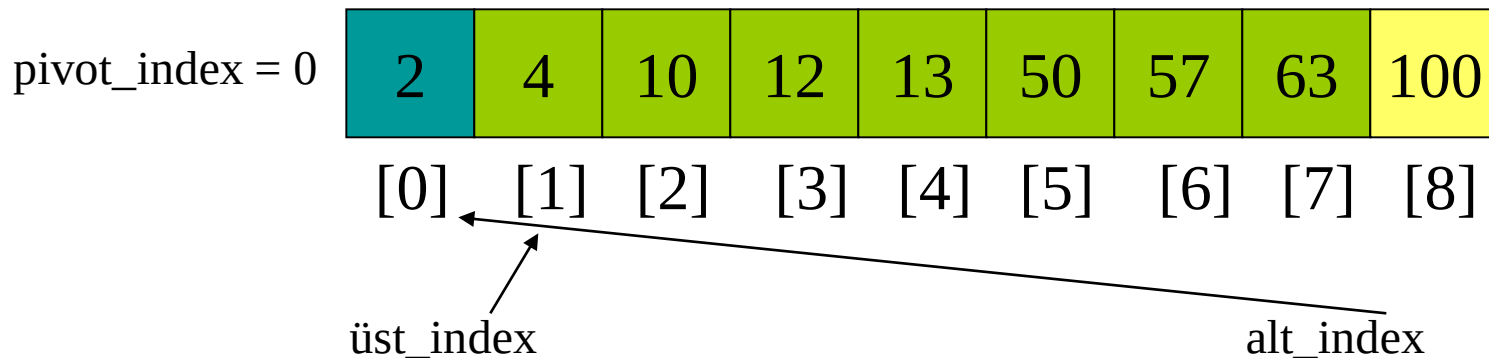
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
- 2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



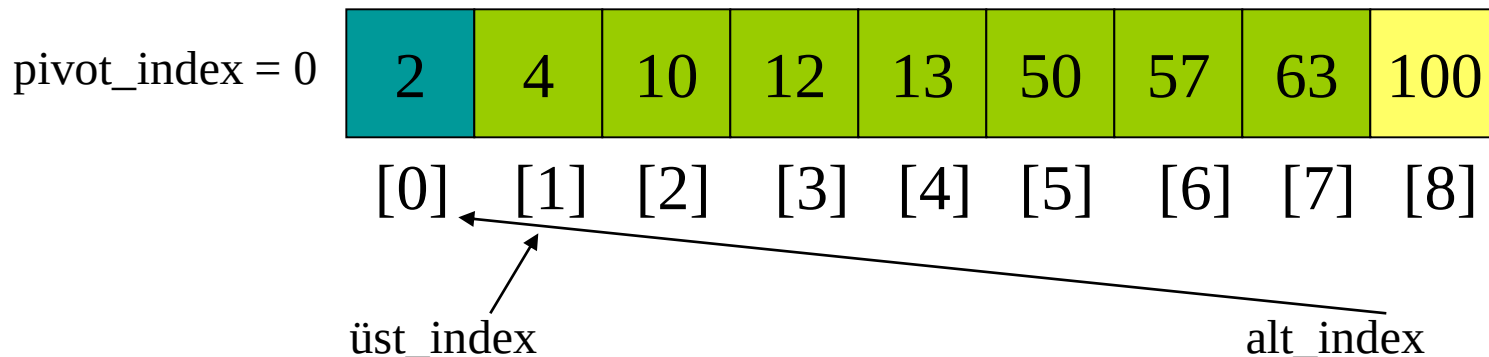
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
- 3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



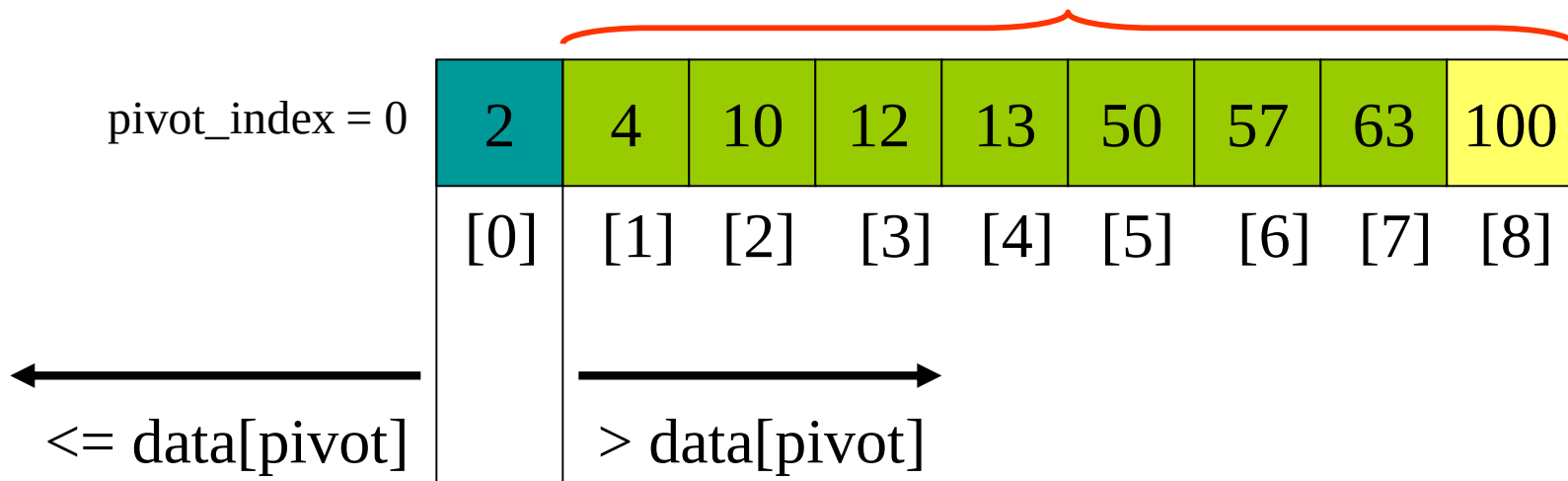
1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
- 4. While $\text{alt_index} > \text{üst_index}$, go to 1.
5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



1. While $\text{data}[\text{üst_index}] \leq \text{data}[\text{pivot}]$
 ++üst_index
2. While $\text{data}[\text{alt_index}] > \text{data}[\text{pivot}]$
 --alt_index
3. If $\text{üst_index} < \text{alt_index}$
 swap $\text{data}[\text{üst_index}]$ and $\text{data}[\text{alt_index}]$
4. While $\text{alt_index} > \text{üst_index}$, go to 1.
- 5. Swap $\text{data}[\text{alt_index}]$ and $\text{data}[\text{pivot_index}]$



$$T(n) = T(k) + T(n - k) + \alpha n$$

En kötü durumda dizinin bölünmesi 1 ve n-1 elemanlı biçiminde olacaktır.

$$T(n) = T(1) + T(n - 1) + \alpha n$$

$$T(n) = T(n - 1) + T(1) + \alpha n = [T(n - 2) + T(1) + \alpha(n - 1)] + T(1) + \alpha n$$

$$= T(n - 2) + 2T(1) + \alpha(n - 1 + n)$$

$$= [T(n - 3) + T(1) + \alpha(n - 2)] + 2T(1) + \alpha(n - 1 + n)$$

$$= T(n - 3) + 3T(1) + \alpha(n - 2 + n - 1 + n)$$

$$= [T(n - 4) + T(1) + \alpha(n - 3)] + 3T(1) + \alpha(n - 2 + n - 1 + n)$$

$$= T(n - 4) + 4T(1) + \alpha(n - 3 + n - 2 + n - 1 + n)$$

$$= T(n - i) + iT(1) + \alpha(n - i + 1 + \dots + n - 2 + n - 1 + n)$$

$$= T(n - i) + i.T(1) + \alpha(\sum_{j=0}^{i-1} (n - j))$$

$$= T(1) + (n-1)T(1) + \alpha(\sum_{j=0}^{n-2} (n - j))$$

$$\begin{matrix} \underline{n-i=1} \\ \underline{i=n-1} \end{matrix}$$

$$\sum_{j=0}^{n-2} j = \sum_{j=1}^{n-2} j = (n-2)(n-1)/2$$

Karesel
çalışma
zamanı

Eniyi Durum Analizi

En iyi durumda dizi her defasında tam olarak ikiye bölünecektir.

$$T(n) = 2T(n/2) + \alpha n = 2(2T(n/4) + \alpha n/2) + \alpha n$$

$$T(n/2) = 2T(n/4) + \alpha n/2$$

$$T(n) = 2T(n/2) + \alpha n$$

$$= 2^2 T(n/4) + 2\alpha n$$

$$= 2^2 (2T(n/8) + \alpha n/4) + 2\alpha n$$

$$= 2^3 T(n/8) + 3\alpha n$$

$$= 2^k T(n/2^k) + k\alpha n$$

$$n = 2^k$$

$$n/2^k < 1$$

$$k = \log n$$

$$T(n) = nT(1) + \alpha n \log n, \text{ which is } O(n \log n).$$

Quicksort Algoritmasının Çalışma Zamanı Analizi

- **Best case** : $O(n \log_2 n)$
- **Worst case** : $O(n^2)$
- **Average case** : $O(n \log_2 n)$

Quicksort

QUICKSORT(A, p, r)

if $p < r$

 then $q \leftarrow \text{PARTITION}(A, p, r)$

 QUICKSORT(A, p, $q-1$)

 QUICKSORT(A, $q+1$, r)

Algoritma rekürsif yapıda olduğundan bilgisayarın yığıtı yoğun bir biçimde kullanılmaktadır.

Yinelemeli (Iterative) Algoritmalar

Analiz

Özyineli Algoritmalar(Recursive Algorithms)

- Analiz

Örnek:

$$T(n) = T(n - 1) + n$$

$$T(0) = 1$$

Çalışma zamanı analizini yapınız?

$$T(n) = T(n - 1) + n$$

$$= T(n - 2) + n - 1 + n$$

$$= T(n - 3) + n - 2 + n - 1 + n$$

...

$$= T(0) + 1 + 2 + 3 + \dots + n - 2 + n - 1 + n =$$

$$= 1 + 1 + 2 + 3 + \dots + n - 2 + n - 1 + n =$$

$$= 1 + \frac{n(n + 1)}{2}$$

$$T(n) = 2T(n-1) + 1$$

$$T(0) = 0$$

$$T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1$$

$$= 4T(n-2) + 2 + 1$$

$$= 8T(n-3) + 4 + 2 + 1$$

$$= 16T(n-4) + 8 + 4 + 2 + 1$$

$$= \dots$$

$$= 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \dots + 8 + 4 + 2 + 1$$

HANOI KULELERİ

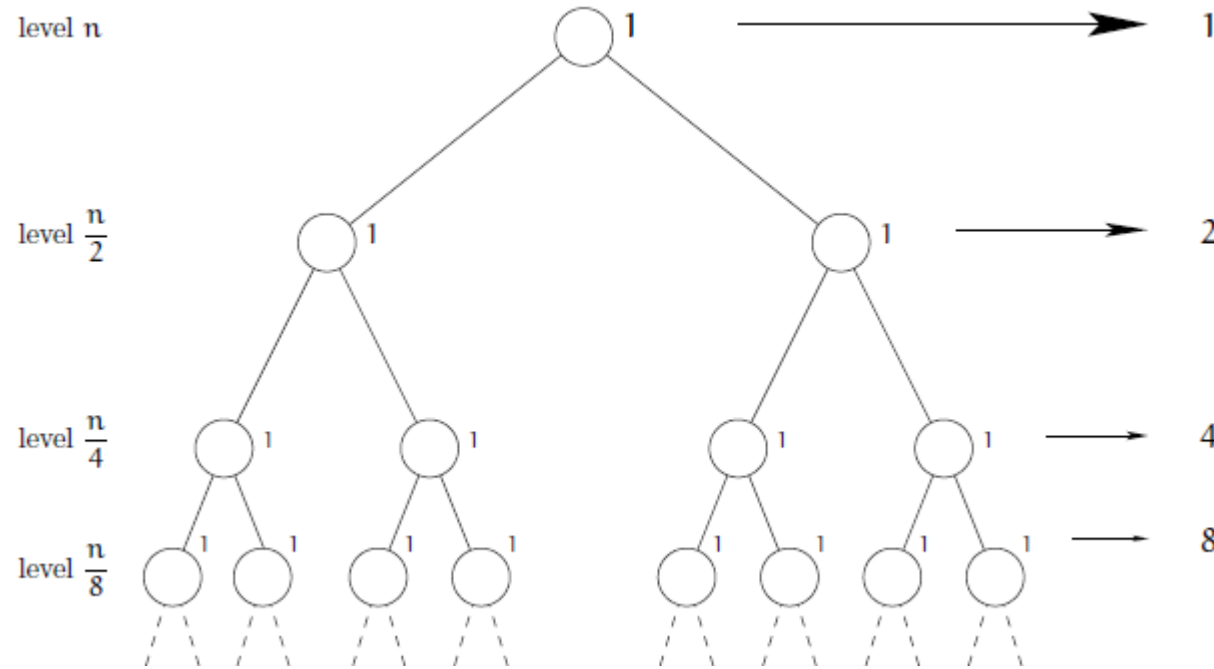
$$= 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \dots + 8 + 4 + 2 + 1$$

$$= 2^k T(n-k) + \sum_{k=0}^{\infty} 2^{k-1}$$

$$= 2^k T(0) + 2^k - 1 = 2^n + 2^n - 1 = 2^{n+1} - 1$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$

$$T(1) = \Theta(1)$$



$$T(n) = \underbrace{1 + 2 + 4 + 8 + \dots}_{(\lg n + 1)}$$

$$= 1 + 2 + 4 + 8 + \dots + \frac{n}{2} + n$$

$$= \sum_{i=0}^{\lg n} \frac{n}{2^i} = n \left(\sum_{i=0}^{\lg n} \frac{1}{2^i} \right) \leq n \underbrace{\left(\sum_{i=0}^{\infty} \frac{1}{2^i} \right)}_2 = 2n$$

$$T(n) = \Theta(n)$$

Hızlı Sıralama

- Worst Case

worst case

$$T(n) = T(k) + T(n-k) + \alpha n$$

$$T(n) = T(1) + T(n-1) + \alpha \cdot n$$

$$T(n) = T(n-1) + T(1) + \alpha \cdot n$$

$$= T(1) + T(n-2) + \alpha(n-1) + T(1) + \alpha \cdot n$$

$$T(n) = T(n-2) + 2T(1) + \alpha(n-1 + n)$$

$T(n-2)$ yeniden yazılabilirse

$$T(n) = T(1) + T(n-3) + \alpha(n-2) + 2T(1) + \alpha(n-1 + n)$$

$$T(n) = T(n-3) + 3T(1) + \alpha(n-2 + n-1 + n)$$

$$T(n) = T(1) + \underline{T(n-4)} + \alpha \cdot (n-3) + 3T(1) + \alpha(n-2 + n-1 + n)$$

$$= T(n-4) + 4T(1) + \alpha[(n-3) + (n-2) + (n-1) + n]$$

Aynı biçimde devam edilerek i . adım için

$$T(n) = T(n-i) + iT(1) + \alpha[(n-i+1) + (n-i+2) + \dots + (n-1) + n]$$

$$T(n) = T(n-i) + iT(1) + \alpha \left[\sum_{j=0}^{i-1} n-j \right]$$

Burada dikkat edilirse i değeri en fazla $(n-1)$ 'e kadar gidebilir. Çünkü $(n-i)$ değeri 1'den büyük olamaz.

$T(n) = T(n-i) + i T(1) \dots$ O halde denklemi yeniden yazacak olursak ($i = n-1$ yazarak)

$$T(n) = T(n-(n-1)) + (n-1)T(1) + \alpha \sum_{j=0}^{n-1-1} n-j$$

$$T(n) = T(1) + (n-1)T(1) + \alpha \left[\sum_{j=0}^{n-2} (n-j) \right]$$

$$= nT(1) + \alpha \left[n^2 - 2n - \frac{1}{2}n^2 + \frac{3}{2}n - 1 \right]$$

$$T(n) = T(1) + (n-1)T(1) + \alpha \left[\sum_{j=0}^{n-2} n - \sum_{j=0}^{n-2} j \right]$$

$$= \overbrace{T(1) + (n-1)T(1)} + \alpha \left[n(n-2) - \frac{(n-2)(n-1)}{2} \right]$$

$$= \overbrace{nT(1)}^{\text{Doğrusal}} + \underbrace{\alpha}_{\text{Sabit}} \left[\overbrace{\frac{1}{2}n^2 - \frac{1}{2}n - 1}^{\text{Kareli}} \right]$$

HızlıSıralama

- Best Case

$$T(n) = T(k) + T(n-k)$$

$$k = \frac{n}{2}, \quad n-k = n - \frac{n}{2} = \frac{n}{2}$$

"Best Case"

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + \alpha n = 2 \cdot T\left(\frac{n}{2}\right) + \alpha \cdot n \quad T\left(\frac{n}{2}\right) = 2 \cdot T\left(\frac{n}{4}\right) + \alpha \cdot \frac{n}{2}$$

$$T(n) = 2 \cdot \left[2 \cdot T\left(\frac{n}{4}\right) + \alpha \cdot \frac{n}{2} \right] + \alpha \cdot n = 2^2 T\left(\frac{n}{4}\right) + 2 \alpha \cdot n$$

$$\left\{ \text{Burada } T\left(\frac{n}{4}\right) = 2 \cdot T\left(\frac{n}{8}\right) + \alpha \frac{n}{4} \right\}$$

yerine yazılırsa

$$T(n) = 2^2 \left[2 \cdot T\left(\frac{n}{8}\right) + \alpha \frac{n}{4} \right] + 2 \cdot \alpha n$$

$$= 2^3 T\left(\frac{n}{8}\right) + 3 \alpha \cdot n$$

"Best Case"

aynı biçimde devam edilirse, k . adımda

$$T(k) = 2^k T\left(\frac{n}{2^k}\right) + k \cdot \alpha n$$

$T(1)$ olması için rekursiyon ancak $n=2^k$ ya kadar gidi.

Diğer durumlarda $\frac{n}{2^k} < 1$ olur.

Bu durumda $n = 2^k$ olacaktır. $\log_2 n = k$

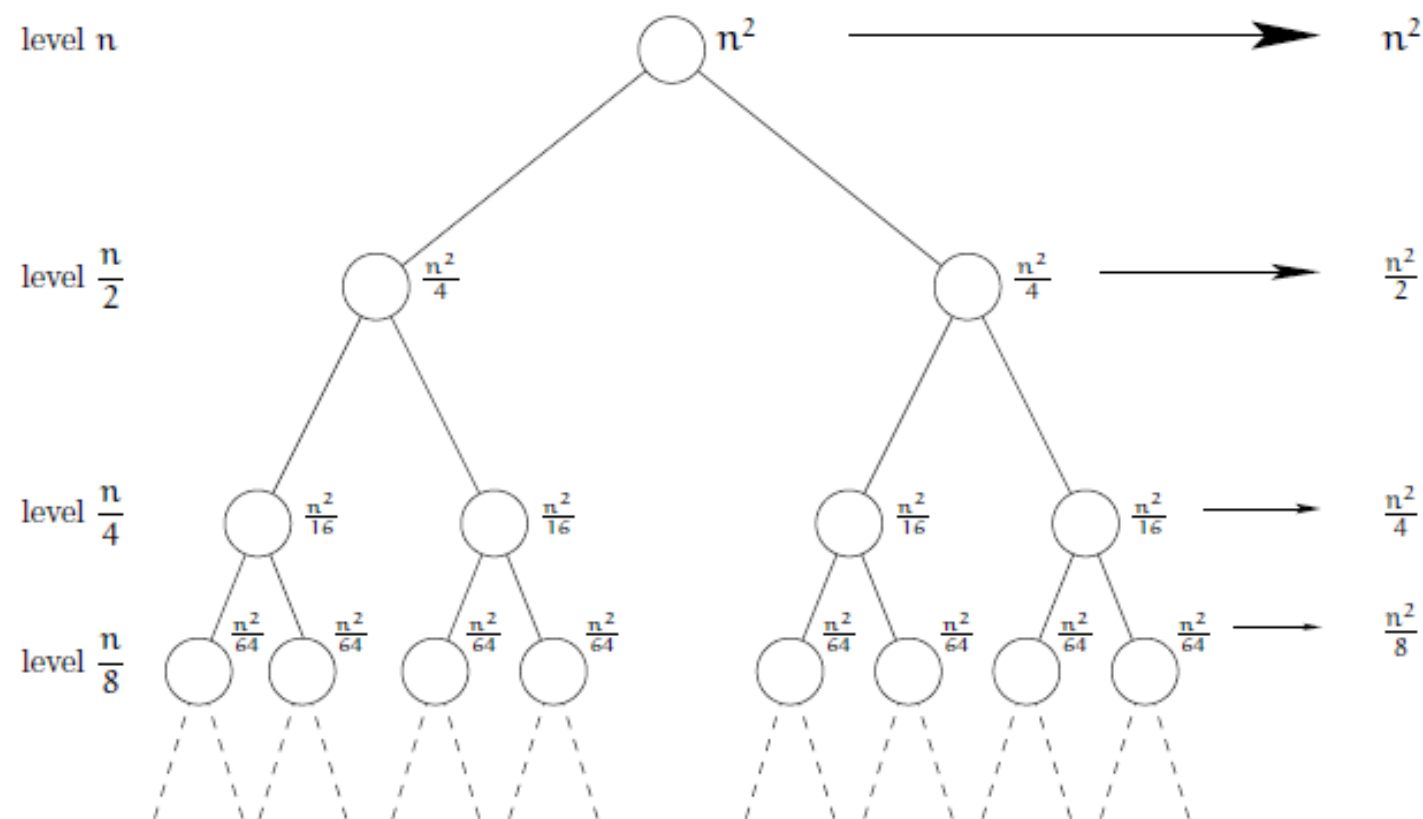
Denklemleri bu yeni duruma göre yeniden yazarsak

$$T(n) = 2^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n \cdot \alpha \cdot n$$

$$= n T(1) + \alpha \cdot n \cdot \log n \rightarrow O(n \cdot \log n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

recursion tree



$$n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \frac{n^2}{8} + \dots \leq n^2 \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n^2$$

$$T(n) = \Theta(n^2)$$

$$T(n) = T(n-1) + \frac{1}{n}$$

$$T(n) = T(n-1) + \frac{1}{n}$$

$$= T(n-2) + \frac{1}{n-1} + \frac{1}{n}$$

$$= T(n-3) + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}$$

...

$$= T(1) + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}$$

$$= T(1) - 1 + \underbrace{1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}}_{\Theta(\lg n)}$$

$$= O(1) + \Theta(\lg n)$$

$$= \Theta(\lg n)$$

$$T(n) = aT(n/b) + cn^d \quad (\text{for } n > 1)$$

$$a \geq 1, b \geq 2, c > 0 \text{ and } d \geq 0$$

$T(n)$ is $\Theta(n^d)$ if $a < b^d$

$T(n)$ is $\Theta(n^d \log n)$ if $a = b^d$

$T(n)$ is $\Theta(n^{\log_b a})$ if $a > b^d$.

$$T(n) = aT(n/b) + cn^d \quad (\text{for } n > 1)$$

$$\begin{aligned} T(n) &\text{ is } \Theta(n^d) \text{ if } a < b^d \\ T(n) &\text{ is } \Theta(n^d \log n) \text{ if } a = b^d \\ T(n) &\text{ is } \Theta(n^{\log_b a}) \text{ if } a > b^d. \end{aligned}$$

$$a = 2 \text{ and } b = 4, \text{ and } \log_4(2) = 1/2.$$

$$T(n) = 2T(n/4) + 1.$$

$$T(n) = \Theta(n^{1/2})$$

$$T(n) = 2T(n/4) + \sqrt{n}.$$

$$\sqrt{n} = \Theta(n^{1/2})$$

$$T(n) = \Theta(n^{1/2} \log_2(n)).$$

$$T(n) = 2T(n/4) + n$$

$$T(n) = \Theta(n)$$

$$T(n) = 2T(n/4) + n^2.$$

$$T(n) = \Theta(n^2)$$

$$T(n) = aT(n/b) + cn^d \quad (\text{for } n > 1)$$

$$T(n) \text{ is } \Theta(n^d) \text{ if } a < b^d$$

$$T(n) \text{ is } \Theta(n^d \log n) \text{ if } a = b^d$$

$$T(n) \text{ is } \Theta(n^{\log_b a}) \text{ if } a > b^d.$$

$$\textbf{a. } T(n) = 2T(n/2) + n^4.$$

$$\textbf{b. } T(n) = T(7n/10) + n.$$

$$T(n) = \Theta(n^4)$$

$$T(n) = \Theta(n).$$

$$T(n) = aT(n/b) + cn^d \quad (\text{for } n > 1)$$

$$T(n) \text{ is } \Theta(n^d) \text{ if } a < b^d$$

$$T(n) \text{ is } \Theta(n^d \log n) \text{ if } a = b^d$$

$$T(n) \text{ is } \Theta(n^{\log_b a}) \text{ if } a > b^d.$$

$$c. T(n) = 16T(n/4) + n^2.$$

$$d. T(n) = 7T(n/3) + n^2$$

$$T(n) = \Theta(n^2 \lg n)$$

$$T(n) = \Theta(n^2)$$

$$T(n) = aT(n/b) + cn^d \quad (\text{for } n > 1)$$

$$T(n) \text{ is } \Theta(n^d) \text{ if } a < b^d$$

$$T(n) \text{ is } \Theta(n^d \log n) \text{ if } a = b^d$$

$$T(n) \text{ is } \Theta(n^{\log_b a}) \text{ if } a > b^d.$$

$$T(n) = 7T(n/2) + n^2.$$

$$2T(n/4) + \sqrt{n}.$$

$$T(n) = \Theta(n^{\lg 7}).$$

$$T(n) = \Theta(\sqrt{n} \lg n)$$

Theorem 1 (Master Theorem, *Let $a \geq 1$ and $b > 1$ be constants, let $k = \lg_b a$, and let $f(n)$ be a positive function. Let*

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(1) = \Theta(1)$$

where $\frac{n}{b}$ is interpreted either as $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$. Then $T(n)$ can be bounded asymptotically as follows.

Case 1 *If $f(n) = O(n^{k-\epsilon})$ for some positive constant ϵ then $T(n) = \Theta(n^k)$.*

Case 2 *If $f(n) = \Theta(n^k)$ then $T(n) = \Theta(n^k \lg n)$.*

Case 3 *If both*

- 1. $f(n) = \Omega(n^{k+\epsilon})$ for some positive constant ϵ , and*
- 2. $a.f\left(\frac{n}{b}\right) \leq c.f(n)$ for some constant c such that $0 < c < 1$ and for all sufficiently large n ,*

then $T(n) = \Theta(f(n))$.

Problem $T(n) = 3T\left(\frac{n}{4}\right) + n \lg n$

$$a = 3, b = 4, \quad \log_b a = \log_4 3 = 0.79$$

$$n \lg n = \Omega(n^{\log_4 3 + \epsilon})$$

$$0 < c < 1 \text{ and } 3\frac{n}{4} \lg \frac{n}{4} \leq cn \lg n \quad c = \frac{3}{4}, \quad \text{Case 3}$$

$$T(n) = \Theta(n \lg n)$$

1. $f(n) = \Omega(n^{k+\epsilon})$ for some positive constant ϵ , and
2. $a.f\left(\frac{n}{b}\right) \leq c.f(n)$ for some constant c such that $0 < c < 1$ and for all sufficiently large n ,

then $T(n) = \Theta(f(n))$.

Problem $T(n) = 2T\left(\frac{n}{2}\right) + n \lg n$

$$a = 2 \text{ and } b = 2, \quad \log_b a = \log_2 2 = 1 \quad n^1 = n$$

$$n \lg n = O(n^{1-\epsilon}) \quad \text{No}$$

$$n \lg n = \Theta(n^1)? \quad \text{No}$$

$$n \lg n = \Omega(n^{1+\epsilon}) \quad \epsilon > 0? \quad \text{No}$$

$$T(n) = \Theta(n \lg^2 n).$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n^3$$

Böl Yönet yaklaşımı ile sıralama(Örnek:Merge Sort)

- Sıralanması istenen kümeler önce iki alt kümeye ayrılır.
- Fonksiyon kendini sol ve sağ alt küme için iki kez çağırır.
- Parçalama işlemine altkümedeki eleman sayısı bir oluncaya kadar devam edilir.
- Daha sonra rekürsif çağrılar geriye doğru çağırana dönmeye başlar ve bu sırada altkümeler elemanları sıralı olacak biçimde birleştirilir.

Birleştirmeli Sırlama (Merge Sort)

- Böl-Yönet yaklaşımli sıralama
- Her bir altdizide bir eleman kalıncaya kadar sırasız diziyi ikiye böl
- Alt problem çözümlerini birleştir:
 - Altdizilerin ilk elemanlarını karşılaştır.
 - En küçük elemanı kaldır ve sonuç diziyeye koy
 - Bütün elemanlar sonuç diziyeye konuncaya kadar işlemleri tekrarla

37	23	6	89	15	12	2	19
----	----	---	----	----	----	---	----

Birleştirme (Merge Sort) Algoritması

MERGE_SORT(A, p, r)

if $p < r$

then $q \leftarrow \lfloor (p+r) / 2 \rfloor$

MERGE_SORT(A, p, q)

MERGE_SORT(A, q+1, r)

MERGE(A, p, q, r)

Merge Algorithm

MERGE(A, p, q, r)

$n_1 \leftarrow q - p + 1$

$n_2 \leftarrow r - q$

// Create arrays L[1.. n_1 +1] and R[1.. n_2 +1]

for i $\leftarrow$ 1 **to** n_1

do L[i] $\leftarrow$ A[p + i - 1]

for j $\leftarrow$ 1 **to** n_2

do R[j] $\leftarrow$ A[q + j]

L[$n_1 + 1$] $\leftarrow \infty$; R[$n_2 + 1$] $\leftarrow \infty$

i $\leftarrow$ 1; j $\leftarrow$ 1

for k $\leftarrow$ p **to** r

do if L[i] $\leq$ R[j]

then A[k] $\leftarrow$ L[i]

 i $\leftarrow$ i + 1

else A[k] $\leftarrow$ R[j]

 j $\leftarrow$ j + 1

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98	23
----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98	23
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98	23
----	----

23

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98	23
----	----

23	98
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

14

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

14	23
----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

14	23	45
----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

98

23

45

14

23	98
----	----

14	45
----	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

23	98
----	----

14	45
----	----

14	23	45	98
----	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

23	98
----	----

14	45
----	----

14	23	45	98
----	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

23	98
----	----

14	45
----	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

23	98
----	----

14	45
----	----

6

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

23	98
----	----

14	45
----	----

6	67
---	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

14	23	45	98
----	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33
---	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42
---	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14
---	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23
---	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33
---	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42
---	----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45
---	----	----	----	----	----

Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

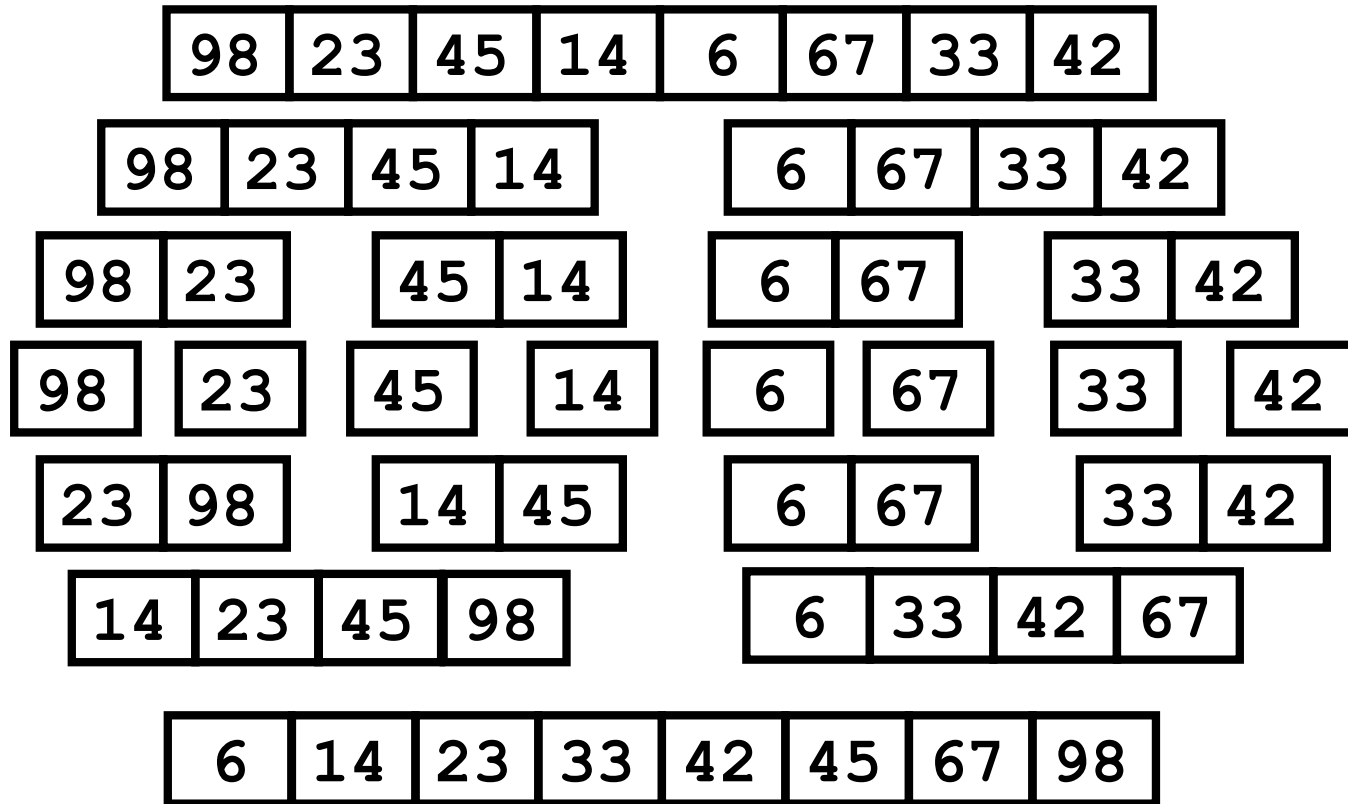
14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45	67
---	----	----	----	----	----	----

Merge

Örnek



Merge

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----

98	23	45	14
----	----	----	----

6	67	33	42
---	----	----	----

98	23
----	----

45	14
----	----

6	67
---	----

33	42
----	----

98

23

45

14

6

67

33

42

23	98
----	----

14	45
----	----

6	67
---	----

33	42
----	----

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45	67	98
---	----	----	----	----	----	----	----

Örnek

98	23	45	14	6	67	33	42
----	----	----	----	---	----	----	----



6	14	23	33	42	45	67	98
---	----	----	----	----	----	----	----

Analiz

- Her rekürsif çağrıda diziyi iki parçaya bölüyoruz. Oluşan ağacın yüksekliği $O(\log n)$ olacaktır.
- i derinlikli düğümde yapılan işlemler $O(n)$ dir.
 - $n/2^i$ elemanlı 2^i parçalama ve birleştirme yaparız.
 - 2^{i+1} recursif çağrı yapılır.
- Toplam $O(n \log n)$ işlem yaparız.

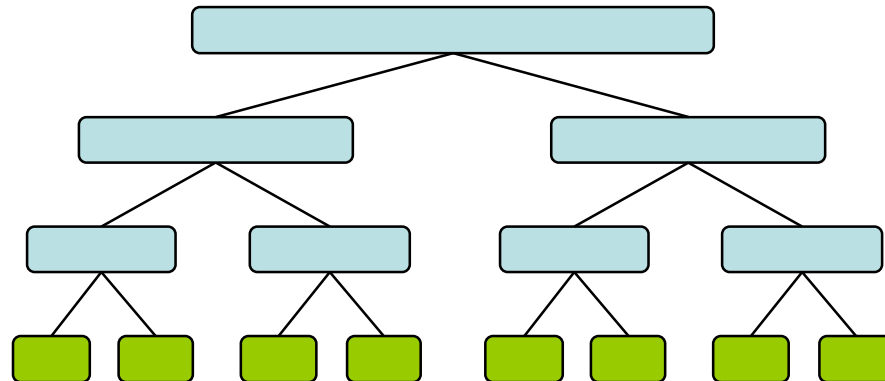
depth #seqs size

$$0 \qquad 1 \qquad n$$

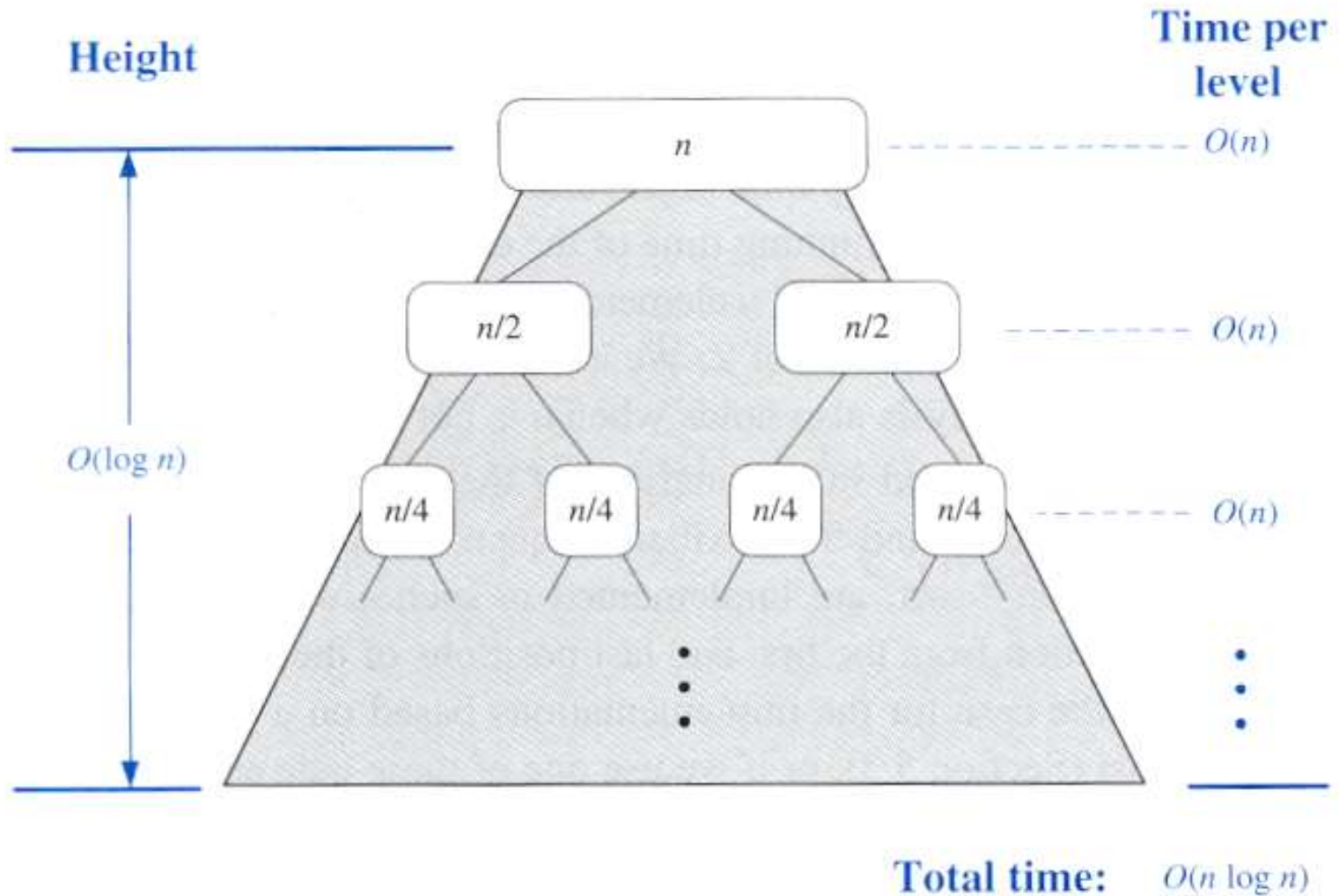
1 2 $n/2$

$$i \qquad 2^i \qquad n/2^i$$

... ..



Analiz



Algoritma Analizi

Büyüme hızlarının karşılaştırılması
(Özet)

$\approx, \preceq, \prec, \succeq$, and $\succ$

Properties of the relations:

$$f(n) \approx g(n) \Leftrightarrow f(n) = \Theta(g(n))$$

$$f(n) \preceq g(n) \Leftrightarrow f(n) = O(g(n))$$

$$f(n) \prec g(n) \Leftrightarrow f(n) = o(g(n))$$

$$f(n) \succeq g(n) \Leftrightarrow f(n) = \Omega(g(n))$$

$$f(n) \succ g(n) \Leftrightarrow f(n) = \omega(g(n))$$

1. Reflexivity: $f(n) \approx f(n)$, $f(n) \preceq f(n)$, $f(n) \succeq f(n)$.

2. Symmetry: $f(n) \approx g(n) \Leftrightarrow g(n) \approx f(n)$.

3. Transitivity:

$$f(n) \approx g(n) \text{ and } g(n) \approx h(n) \Rightarrow f(n) \approx h(n)$$

$$f(n) \preceq g(n) \text{ and } g(n) \preceq h(n) \Rightarrow f(n) \preceq h(n)$$

$$f(n) \prec g(n) \text{ and } g(n) \prec h(n) \Rightarrow f(n) \prec h(n)$$

$$f(n) \succeq g(n) \text{ and } g(n) \succeq h(n) \Rightarrow f(n) \succeq h(n)$$

$$f(n) \succ g(n) \text{ and } g(n) \succ h(n) \Rightarrow f(n) \succ h(n).$$

4. Transpose symmetry:

$$f(n) \succeq g(n) \Leftrightarrow g(n) \preceq f(n)$$

$$f(n) \succ g(n) \Leftrightarrow g(n) \prec f(n).$$

$$5. f(n) \prec g(n) \Rightarrow f(n) \preceq g(n)$$

$$f(n) \preceq g(n) \not\Rightarrow f(n) \prec g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \succeq g(n)$$

$$f(n) \succeq g(n) \not\Rightarrow f(n) \succ g(n)$$

$$6. f(n) \approx g(n) \Rightarrow f(n) \not\prec g(n)$$

$$f(n) \approx g(n) \Rightarrow f(n) \not\succ g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\approx g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\succeq g(n)$$

$$f(n) \prec g(n) \Rightarrow f(n) \not\preceq g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\approx g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\prec g(n)$$

$$f(n) \succ g(n) \Rightarrow f(n) \not\preceq g(n)$$

$$7. f(n) \approx g(n) \Leftrightarrow f(n) \preceq g(n) \text{ and } f(n) \succeq g(n)$$

$$8. f(n) \prec g(n) \text{ and } f(n) \succ g(n) \text{ mümkün değil}$$

$$9. \text{ Let } f(n) = f_1(n) \pm f_2(n) \pm f_3(n) \pm \dots \pm f_k(n). \text{ Let}$$

$$f_1(n) \succ f_2(n)$$

$$f_1(n) \succ f_3(n)$$

...

$$f_1(n) \succ f_k(n)$$

Then $f(n) \approx f_1(n)$.

$$10. \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L \Rightarrow f(n) \approx g(n)$$

$$f(n) \approx g(n) \not\Rightarrow \lim_{n \rightarrow \infty} \frac{f(n)}{q(n)} \text{ exists.}$$

$$f(n) = \frac{1}{2}n^2 - 3n. \qquad f(n) \approx n^2.$$

$$0 \leq c_1.n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2.n^2$$

$$0 \leq c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

$$0 \leq c_1$$

$$c_1 \leq \frac{1}{2} - \frac{3}{n}$$

$$\frac{1}{2} - \frac{3}{n} \leq c_2$$

$$c_1 > 0 \quad n'_0 = 1 \quad c_2 = 1.$$

$$\frac{1}{2} - \frac{3}{7} = \frac{7}{14} - \frac{6}{14} = \frac{1}{14}.$$

$$c_1 = \frac{1}{14} \text{ and } n''_0 = 7.$$

$$n_0 = \max\{n'_0, n''_0\} = 7.$$

$$n_0 = 7, c_1 = \frac{1}{14}, c_2 = 1$$

Hangisi doğrudur?

$$2^{n+1} \approx 2^n$$

$$2^{2n} \approx 2^n$$

$$2^{n+1} = 2 \cdot 2^n$$

$$c \cdot 2^n \approx 2^n.$$

$$2^{n+1} \approx 2^n$$

Which of the following are true:

$$\frac{1}{n^2} \prec \frac{1}{n} \quad \text{Doğru}$$

$$2^{\frac{1}{n^2}} \prec 2^{\frac{1}{n}} \quad \text{eşit}$$

$$\lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n^2}}}{2^{\frac{1}{n}}} \right) = \lim_{n \rightarrow \infty} \left(2^{\frac{1}{n^2} - \frac{1}{n}} \right) = 1 \text{ because}$$

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n^2} - \frac{1}{n} \right) = \lim_{n \rightarrow \infty} \left(\frac{1-n}{n^2} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n} - 1}{n} \right) = 0$$

Hangisi doğrudur?

$$\text{Doğru} \quad \frac{1}{n} \prec 1 - \frac{1}{n}$$

$$\text{eşit} \quad 2^{\frac{1}{n}} \prec 2^{1-\frac{1}{n}}$$

$$\lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n}}{1 - \frac{1}{n}} \right) = \lim_{n \rightarrow \infty} \left(\frac{\frac{1}{n}}{\frac{n-1}{n}} \right) = \lim_{n \rightarrow \infty} \frac{1}{n-1} = 0$$

$$\lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n}}}{2^{1-\frac{1}{n}}} \right) = \lim_{n \rightarrow \infty} \left(\frac{2^{\frac{1}{n}}}{2^{\frac{2}{n}}} \right) = \text{const}$$

$$2^{2n} = 2^n \cdot 2^n$$

$$2^{2n} \approx 2^n$$

not true.

$$2^n \cdot 2^n \leq c_2 \cdot 2^n \Leftrightarrow 2^n \leq c_2$$

Hangisi doğrudur ?

$$f(n) \approx g(n) \Rightarrow a^{f(n)} \approx a^{g(n)}$$

$$a > 1 \quad f(n) \preceq g(n) \Rightarrow a^{f(n)} \preceq a^{g(n)}$$

$$f(n) \prec g(n) \Rightarrow a^{f(n)} \prec a^{g(n)}$$

Üçü de yanlış

$$2n \approx n \text{ and } 2^{2n} \not\approx 2^n.$$

$$\frac{1}{n} \prec 1 - \frac{1}{n} \text{ but } 2^{\frac{1}{n}} \not\prec 2^{1-\frac{1}{n}}.$$

$$\frac{1}{n^2} \prec \frac{1}{n} \text{ but } 2^{\frac{1}{n^2}} \not\prec 2^{\frac{1}{n}}.$$

$$\begin{array}{lcl}
 a^{f(n)} \approx a^{g(n)} \Rightarrow f(n) \approx g(n) & \text{Doğru} & \\
 a > 1. \quad a^{f(n)} \preceq a^{g(n)} \Rightarrow f(n) \preceq g(n) & \text{Doğru} & \longrightarrow \\
 a^{f(n)} \prec a^{g(n)} \Rightarrow f(n) \prec g(n) & \text{Yanlış} & \\
 2^n \prec 2^{2n} \text{ but } n \not\prec 2n. & &
 \end{array}$$

$$\begin{array}{l}
 g(n) = \frac{1}{n}. \\
 2^{1-\frac{1}{n}} \approx 2^{\frac{1}{n}}, \\
 2^{1-\frac{1}{n}} \preceq 2^{\frac{1}{n}}, \text{ but } \frac{1}{n} \prec 1 - \frac{1}{n}, \\
 1 - \frac{1}{n} \not\prec \frac{1}{n}. \\
 2^{\frac{1}{n^2}} \preceq 2^{\frac{1}{n}} \\
 \frac{1}{n^2} \not\prec \frac{1}{n}.
 \end{array}$$

$$a^n \prec n! \prec n^n$$

$$\lim_{n \rightarrow \infty} \left(\frac{n.(n-1).(n-2) \dots 2.1}{\underbrace{a.a.a \dots a.a}_{n \text{ times}}} \right) = \infty$$

$$\lim_{n \rightarrow \infty} \left(\frac{n.(n-1).(n-2) \dots 2.1}{\underbrace{n.n.n \dots n.n}_{n \text{ times}}} \right) = 0$$

a , k and ϵ be any constants, $k > 0$, $a > 1$, and $\epsilon > 0$.

$$(\log_a n)^k \prec n^\epsilon \quad \text{Doğru}$$

$$\lim_{n \rightarrow \infty} \frac{n^\epsilon}{(\log_a n)^k} =$$

$$b \leftarrow \frac{\epsilon}{k} \quad \text{olsun}$$

$$\lim_{n \rightarrow \infty} \frac{(n^b)^k}{(\log_a n)^k} =$$

$$\lim_{n \rightarrow \infty} \left(\frac{n^b}{\log_a n} \right)^k =$$

k positive

$$\lim_{n \rightarrow \infty} \frac{n^b}{\log_a n} = \quad \text{l'Hôpital's rule uygulansın}$$

$$\lim_{n \rightarrow \infty} \frac{bn^{b-1}}{\left(\frac{1}{\ln a}\right) \left(\frac{1}{n}\right)} = \lim_{n \rightarrow \infty} (\ln a) b n^b = \infty$$

a and ϵ be any constants, $a > 1$ and $\epsilon > 0$.

$$n^\epsilon \prec a^n \quad \text{Doğru}$$

Her iki tarafın a tabanına göre logaritmasını alalım

$$\epsilon \cdot \log_a n \prec n$$

$$\epsilon \cdot \log_a n \prec n$$

Definition (log-star function)

$$\lg^{(i)} n = \begin{cases} n, & \text{if } i = 0 \\ \lg(\lg^{(i-1)} n), & \text{if } i > 0 \text{ and } \lg^{(i-1)} n > 0 \\ \text{undefined}, & \text{if } i > 0 \text{ and } \lg^{(i-1)} n < 0 \text{ or } \lg^{(i-1)} n \text{ is undefined} \end{cases}$$

$$\lg^* n = \min \{i \geq 0 \mid \lg^{(i)} n \leq 1\}$$

$$\lg^* 2 = 1, \text{ since } \lg^{(0)} 2 = 2 \text{ and } \lg^{(1)} 2 = \lg(\lg^{(0)} 2) = \lg(2) = 1$$

$$\lg^* 3 = 2, \text{ since } \lg^{(0)} 3 = 3 \text{ and } \lg(\lg^{(0)} 3) = \lg(\lg 3) = 0.6644 \dots$$

$$\lg^* 4 = 2$$

$$\lg^* 5 = 3$$

...

$$\lg^* 16 = 3$$

$$\lg^* 17 = 4$$

...

$$\lg^* 65536 = 4$$

$$\lg^* 65537 = 5$$

...

$$\lg^* 2^{65536} = 5$$

$$\lg^* (2^{65536} + 1) = 6$$

...

Her gerçel sayı t, ikililerden oluşan bir kule ile temsil edilebilir:

$$t = 2^{2^{2^{\dots^{2^s}}}}$$

Burada s, $1 < s \leq 2$ koşulunu sağlayan bir gerçel sayıdır. Kulenin yüksekliği, bu dizideki eleman sayısıdır.

<i>number</i>	<i>its tower of twos</i>	<i>the height of the tower</i>
2	2	1
3	$2^{1.5849625007\dots}$	2
4	2^2	2
5	$2^{2^{1.2153232957\dots}}$	3
16	2^{2^2}	3
17	$2^{2^{2^{1.0223362884\dots}}}$	4
65536	$2^{2^{2^2}}$	4
65537	$2^{2^{2^{2^{1.000000051642167\dots}}}}}$	5

Bunu göz önünde bulundurarak, $\lg n$ 'nin n adet ikili kulenin yüksekliği olduğunu görmek çok kolaydır.

ÖRNEKLER

$$2^{2^{n+1}} \succ 2^{2^n} \quad 2^{2^{n+1}} = 2^{2 \cdot 2^n} = 2^{2^n} \times 2^{2^n}.$$

$$(n+1)! \succ n!$$

$$(n+1)! = (n+1) \times n!$$

$$2^{2^n} \succ (n+1)! \quad \text{Her iki tarafın logaritması alınırsa}$$

$$2^n \lg((n+1)!)$$

$$\lg((n+1)!) \approx (n+1) \lg(n+1)$$

$$(n+1) \lg(n+1) \approx n \lg n.$$

$$2^n \succ n \lg n,$$

$$2^{2^n} \succ (n+1)!$$

$$e^n \succ n \cdot 2^n$$

$$\lim_{n \rightarrow \infty} \frac{n \cdot 2^n}{e^n} = \lim_{n \rightarrow \infty} \frac{n}{\frac{e^n}{2^n}} = \lim_{n \rightarrow \infty} \frac{n}{\left(\frac{e}{2}\right)^n} = 0$$

$$n \cdot 2^n \succ 2^n$$

$$2^n \succ \left(\frac{3}{2}\right)^n \quad \lim_{n \rightarrow \infty} \frac{\left(\frac{3}{2}\right)^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{3}{4}\right)^n = 0$$

$$\left(\frac{3}{2}\right)^n \succ n^{\lg(\lg n)} \quad \text{Heriki tarafın logaritması alınır}$$

$$(\lg n)^{\lg n} \approx n^{\lg(\lg n)}$$

Heriki tarafın logaritması alınır

$$n \cdot \lg\left(\frac{3}{2}\right) \succ \lg n \cdot \lg(\lg n)$$

$$(\lg n) \cdot (\lg(\lg n)) \approx (\lg(\lg n)) \cdot (\lg n)$$

$$n \succ \lg^2 n$$

$$n \succ \lg n \cdot \lg(\lg n)$$

$$\lg^2 n \succ \lg n \cdot \lg(\lg n)$$

$$(\lg n)^{\lg n} \succ (\lg n)!$$

$$m = \lg n \quad \text{olsun} \quad m^m \succ m!$$

$$(\lg n)! \succ n^3$$

Heriki tarafın logaritması alınır

$$m = \lg n \quad \text{olsun} \quad \lg((\lg n)!) \quad 3 \cdot \lg n$$

$$\lg(m!)$$

$$m \lg m, \succ 3 \cdot \lg n$$

$$\lim_{n \rightarrow \infty} \frac{3 \cdot \lg n}{(\lg n) \cdot (\lg(\lg n))} = \lim_{n \rightarrow \infty} \frac{3}{\lg(\lg n)} = 0$$

$$\lg((\lg n)!) \approx (\lg n) \cdot (\lg(\lg n)).$$

$$n^3 \succ n^2.$$

$$n^2 \approx 4^{\lg n}$$

$$n^2 \succ n \lg n.$$

$$\lg n! \approx n \lg n$$

$$n \lg n \succ n.$$

$$n \approx 2^{\lg n}$$

$$n \succ (\sqrt{2})^{\lg n}$$

$$(\sqrt{2})^{\lg n} = 2^{\frac{1}{2} \lg n} = 2^{\lg \sqrt{n}} = \sqrt{n}$$

$$n \succ \sqrt{n}.$$

$$(\sqrt{2})^{\lg n} \succ 2^{\sqrt{2} \lg n}$$

$$\lg n \succ \sqrt{\lg n},$$

$$\frac{1}{2} \cdot \lg n \succ \sqrt{2} \cdot \sqrt{\lg n} = \sqrt{2 \lg n}.$$

$$2^{\frac{1}{2} \cdot \lg n} \succ 2^{\sqrt{2} \lg n}$$

$$(\sqrt{2})^{\lg n} \succ 2^{\sqrt{2} \lg n}.$$

$$2^{\sqrt{2} \lg n} \succ \lg^2 n.$$

Heriki tarafta logaritma alalım

$$\sqrt{2 \lg n} \quad \lg (\lg^2 n) = 2 \cdot \lg (\lg n) \\ m = \lg n \text{ olsun}$$

$$\sqrt{2m} = \sqrt{2} \sqrt{m} = \sqrt{2} \cdot m^{\frac{1}{2}} \quad 2 \lg m.$$

$$m^{\frac{1}{2}} \succ \lg m$$

$$\sqrt{2} \cdot m^{\frac{1}{2}} \succ 2 \lg m,$$

$$\sqrt{2m} \succ 2 \lg m.$$

$$\sqrt{2 \lg n} \succ \lg (\lg^2 n).$$

$$\lg^2 n \succ \ln n.$$

$$\ln n \succ \sqrt{\lg n}.$$

$$\sqrt{\lg n} \succ \ln \ln n$$

$$\ln \ln n \succ 2^{\lg^* n}$$

$$\ln n = \frac{\lg n}{\lg e}.$$

$$\sqrt{\frac{\ln n}{\ln 2}} \succ \ln \ln n$$

$\ln \ln n \approx \lg \lg n$ olduğundan
yeniden aşağıdaki gibi yazılabilir

$$m = \ln n \text{ olsun}$$

$$\lg \lg n \succ 2^{\lg^* n}$$

Heriki tarafın logaritmasını alalım

$$\frac{1}{\sqrt{\ln 2}} \cdot \sqrt{m} \succ \ln m$$

$$\lg \lg \lg n \succ \lg^* n.$$

$$2^{\lg^* n} \succ \lg^* n$$

$$\lg^* n \approx \lg^* (\lg n)$$

$f(n)$ artan bir fonksiyon ise

Hatırlatma: n sayısı 2'ler kulesi olsun.

n ile $\lg n$ 'inkule yüksekikleri arasındaki fark 1'dir.

$$2^{f(n)} \succ f(n)$$

$$\lg^* (\lg n) = (\lg^* n) - 1$$

$$1 \approx n^{\frac{1}{\lg n}}$$

$$\lg^* n \succ \lg (\lg^* n)$$

$$n^{\frac{1}{\lg n}} =: 2$$

$$\lg \left(n^{\frac{1}{\lg n}} \right) = \frac{1}{\lg n} \cdot \lg n = 1$$

$$\lg (\lg^* n) \succ 1$$

k bir sabit ve $0 < k < 1$ ise

$$1 + k + k^2 + k^3 + \dots + k^n = \Theta(1)$$

$$1 + k + k^2 + k^3 + \dots + k^n < \sum_{t=0}^{\infty} k^t = \frac{1}{1-k} = \Theta(1)$$

Genelleme:

$$1 + k + k^2 + k^3 + \dots + k^n = \begin{cases} \Theta(1), & \text{if } 0 < k < 1 \\ \Theta(n), & \text{if } k = 1 \\ \Theta(k^n), & \text{if } k > 1 \end{cases}$$

Algoritma Analizi ve Tasarımı

Dinamik Programlama

Dinamik Programlama

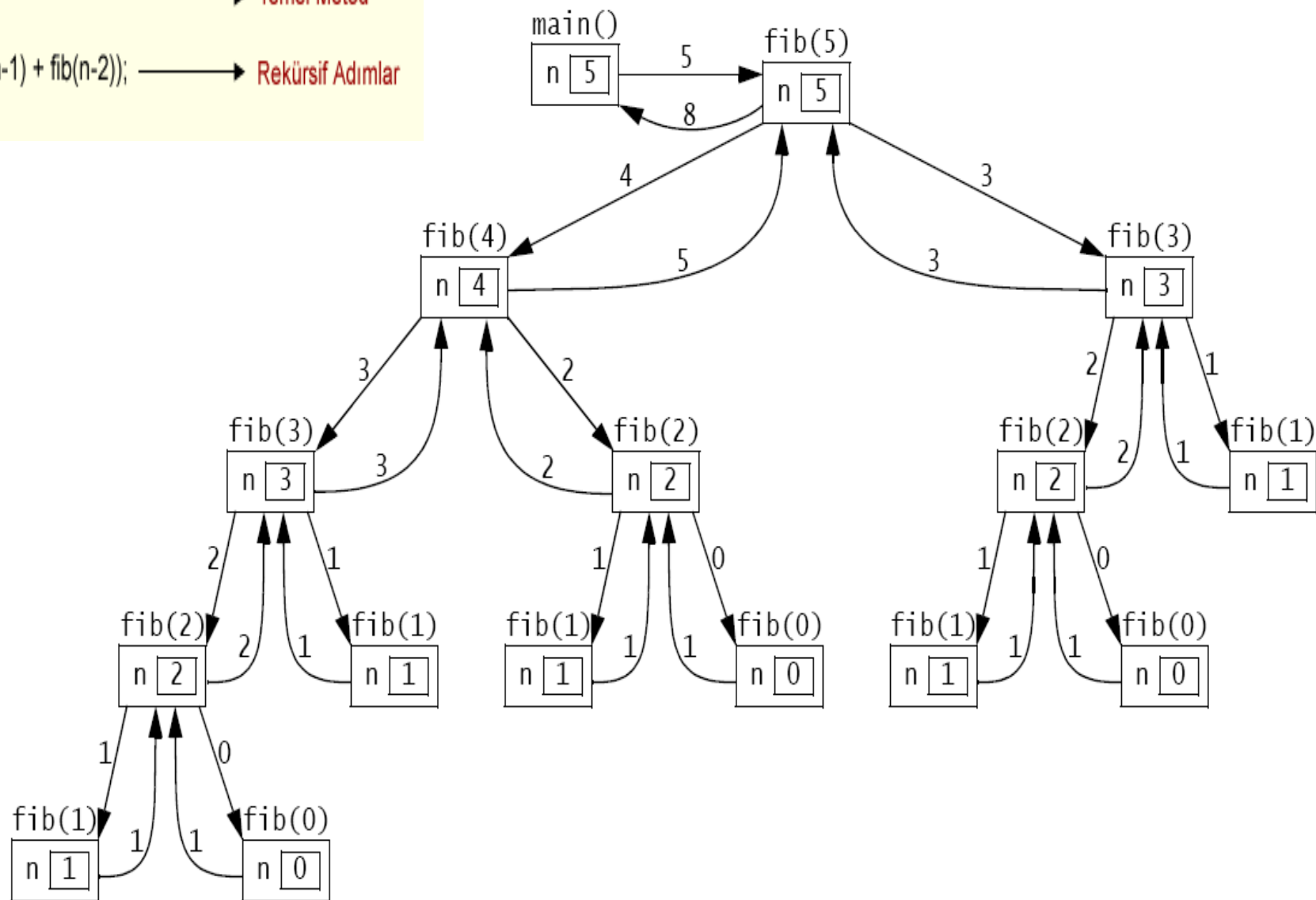
- Algoritma tasarım tekniği-Divide and conquer
- (Böl ve Yönet)
 - Problem bağımsız alprogramlar halinde bölünür
 - Altprogramlar rekürsif olarak çözülür
 - Orjinal problemi çözmek için çözümler birleştirilir

```

int fib(int n) {
  if (n <= 1)
    return n;
  else
    return (fib(n-1) + fib(n-2));
}

```

→ Temel Metod
 → Rekürsif Adımlar



```
1  public static int fib(int n) {  
2      if (n < 2) {  
3          return n;  
4      }  
5      int[] f = new int[n];  
6      f[0] = 0;  
7      f[1] = 1;  
8      for (int i=2; i<n; i++) {  
9          f[i] = f[i-1] + f[i-2];  
10     }  
11     return f[n-1] + f[n-2];  
12 }
```

Dinamik Programlama

- Alt problemler bağımsızken uygulanabilir.
 - Altproblemler daha küçük altpromlarla da paylaşılır.

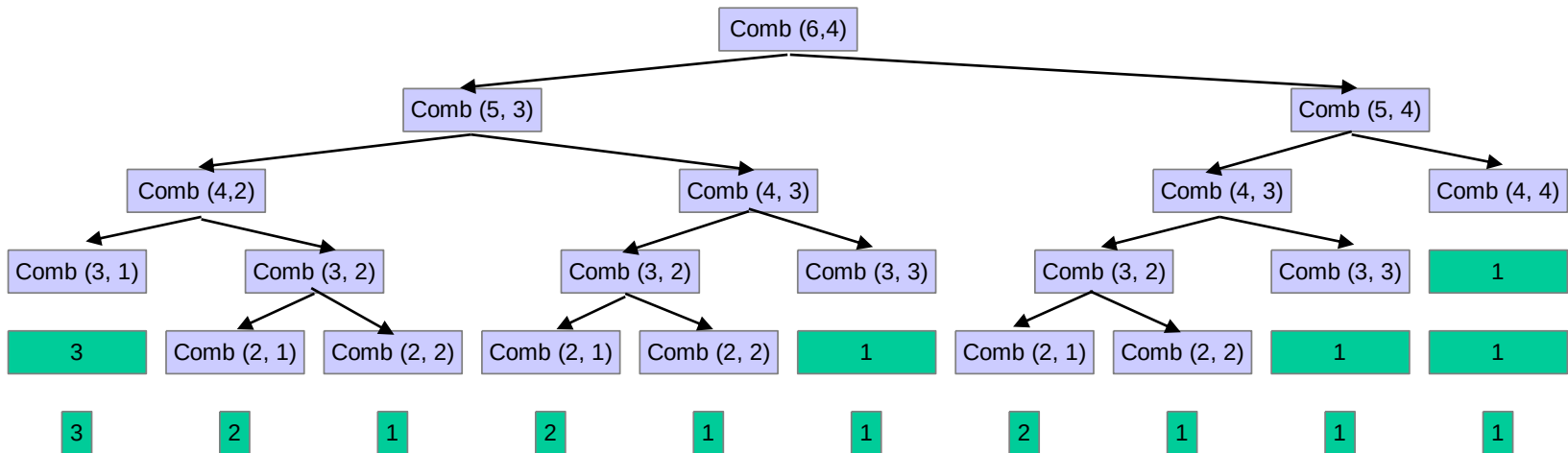
Combinations:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\binom{n}{1} = 1 \qquad \binom{n}{n} = 1$$

- Böl ve Yönet yaklaşımı ortak alt problemleri tekrar tekrar çözecektir
- Dinamik programlama her alt problemi sadece bir kez çözer ve cevabı bir tabloda saklar

Kombinasyon Problemi



$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

Recurrence: $C(n,k) = C(n-1,k) + C(n-1,k-1)$ for $n > k > 0$
 $C(n,0) = 1, C(n,n) = 1$ for $n \geq 0$

		k 							
		0	1	2	3	4	5	6	7
n 	0	1							
	1	1	1						
	2	1	2	1					
	3	1	3	3	1				
	4	1	4	6	4	1			
	5	1	5	10	10	5	1		
	6	1	6	15	20	15	6	1	
	7	1	7	21	35	35	21	7	1
	8	1	8	28	56	70	56	28	8
	9	1	9	36	84	126	126	84	36
	10	1	10	45	120	210	252	210	120

```
for  $i \leftarrow 0$  to  $n$  do  
    for  $j \leftarrow 0$  to  $\min(i, k)$  do  
        if  $j = 0$  or  $j = i$   
             $C[i, j] \leftarrow 1$   
        else  $C[i, j] \leftarrow C[i - 1, j - 1] + C[i - 1, j]$   
return  $C[n, k]$ 
```

Time efficiency: $\Theta(nk)$

Space efficiency: $\Theta(nk)$

```

// A Dynamic Programming based solution to compute nCr % p
#include <bits/stdc++.h>
using namespace std;
// Returns nCr % p
int nCrModp(int n, int r, int p)
{
    // Optimization for the cases when r is large
    if (r > n - r)
        r = n - r;

    // The array C is going to store last row of pascal triangle at the end. And last entry of last row is nCr
    int C[r + 1];
    memset(C, 0, sizeof(C));

    C[0] = 1; // Top row of Pascal Triangle

    // One by constructs remaining rows of Pascal Triangle from top to bottom
    for (int i = 1; i <= n; i++) {
        // Fill entries of current row using previous row values
        for (int j = min(i, r); j > 0; j--)
            //  $nC_j = (n-1)C_j + (n-1)C_{j-1}$ ;
             $C[j] = (C[j] + C[j - 1]) \% p$ ;
    }
    return C[r];
}

// Driver program
int main()
{
    int n = 10, r = 2, p = 13;
    cout << "Value of nCr % p is " << nCrModp(n, r, p);
    return 0;
}

```

Dinamik Programlama

- **Optimizasyon problemleri**

- En iyi çözümü elde etmek için bir dizi seçenek yapılmalıdır
- En uygun değere sahip bir çözüm bulunur (minimum veya maksimum)
- Optimum değere götüren birçok çözüm olabilir
- Hedef: En iyi çözümü bulmak

Dinamik Programlama Algoritması

1. Optimal bir çözümün yapısını karakterize edin
2. Optimal bir çözümün değerini tekrar tekrar tanımlayın
3. En uygun çözümün değerini aşağıdan yukarıya doğru hesaplayın
4. Hesaplanan bilgilerden optimal bir çözüm oluşturun

Matris-Zinciri Çarpımı

Problem: verilen bir dizi $\langle A_1, A_2, \dots, A_n \rangle$, matrisi için aşağıdaki çarpımın hesaplanması.

$$A_1 \cdot A_2 \cdots A_n$$

- Matris uyumluluğu:

$$C = A \cdot B$$

$$\text{col}_A = \text{row}_B$$

$$\text{row}_C = \text{row}_A$$

$$\text{col}_C = \text{col}_B$$

$$C = A_1 \cdot A_2 \cdots A_i \cdot A_{i+1} \cdots A_n$$

$$\text{col}_i = \text{row}_{i+1}$$

$$\text{row}_C = \text{row}_{A_1}$$

$$\text{col}_C = \text{col}_{A_n}$$

MATRIX-MULTIPLY(A, B)

if $\text{columns}[A] \neq \text{rows}[B]$

then error "incompatible dimensions"

else for $i \leftarrow 1$ to $\text{rows}[A]$

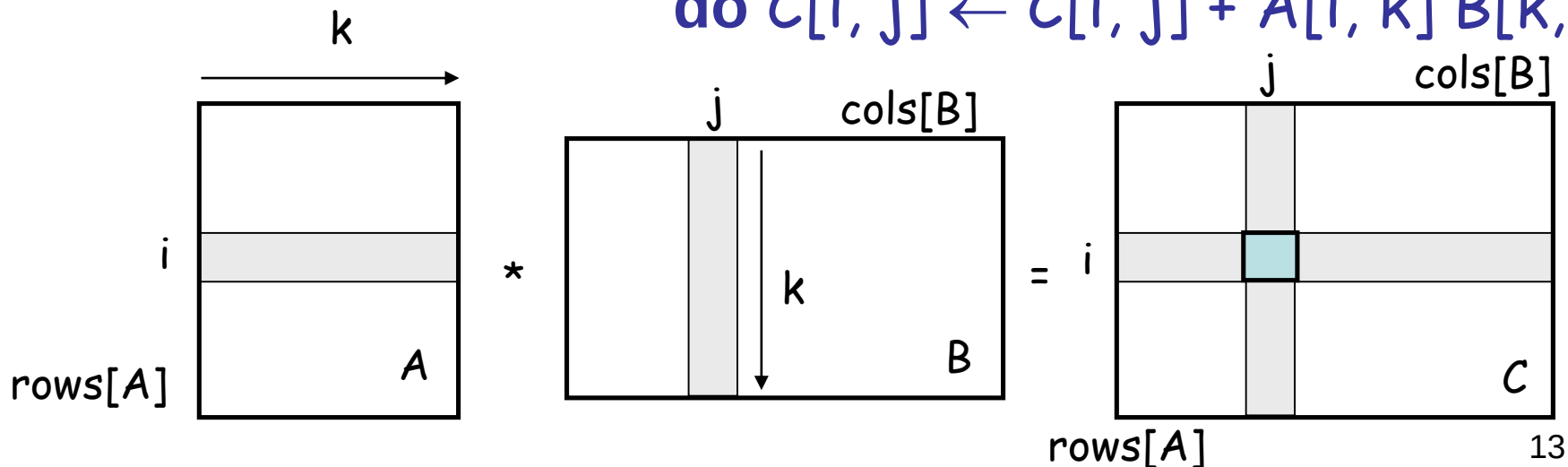
do for $j \leftarrow 1$ to $\text{columns}[B]$

do $C[i, j] = 0$

for $k \leftarrow 1$ to $\text{columns}[A]$

do $C[i, j] \leftarrow C[i, j] + A[i, k] B[k, j]$

$\text{rows}[A] \cdot \text{cols}[A] \cdot \text{cols}[B]$
multiplications



Matris-ZinciriÇarpımı

- Matrisleri hangi sırayla çarpmalıyız?

$$A_1 \cdot A_2 \cdots A_n$$

- Matrislerin çarpılacağı sırayı almak için ürünü parantez içine alalım:

$$\begin{aligned}\text{Şöyleki: } A_1 \cdot A_2 \cdot A_3 &= ((A_1 \cdot A_2) \cdot A_3) \\ &= (A_1 \cdot (A_2 \cdot A_3))\end{aligned}$$

- Bu siparişlerden hangisini seçmeliyiz?
- Matrisleri çarpma düzenimiz, çarpımı değerlendirme maliyeti üzerinde önemlidir.

Örnek

$$A_1 \cdot A_2 \cdot A_3$$

- A_1 : 10×100
- A_2 : 100×5
- A_3 : 5×50

1. $((A_1 \cdot A_2) \cdot A_3)$: $A_1 \cdot A_2 = 10 \times 100 \times 5 = 5,000$ (10×5)
 $((A_1 \cdot A_2) \cdot A_3) = 10 \times 5 \times 50 = 2,500$

Total: 7,500 skaler çarpım

2. $(A_1 \cdot (A_2 \cdot A_3))$: $A_2 \cdot A_3 = 100 \times 5 \times 50 = 25,000$ (100×50)
 $(A_1 \cdot (A_2 \cdot A_3)) = 10 \times 100 \times 50 = 50,000$

Total: 75,000 skaler çarpım

Matris-ZinciriÇarpımı Problemi

- Verilen bir matris zincirinde $\langle A_1, A_2, \dots, A_n \rangle$, burada A_i , $p_{i-1} \times p_i$ boyutlarındadır. Skaler çarpım sayısını minimuma indirecek şekilde $A_1 \cdot A_2 \cdots A_n$ çarpımını tam parantezlersek.

$$\begin{array}{ccccccc} A_1 & \cdot & A_2 & \cdots & A_i & \cdot & A_{i+1} & \cdots & A_n \\ p_0 \times p_1 & & p_1 \times p_2 & & p_{i-1} \times p_i & & p_i \times p_{i+1} & & p_{n-1} \times p_n \end{array}$$

Olası parentezleme sayısı nedir?

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k). \quad f(n) = \frac{1}{n} \binom{2n-2}{n-1}.$$

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796,

4 adet matrisin çarpımı için 5; 10 adet matris için 4862 farklı çarpım düzeni vardır.

$$f(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi} n^{1.5}}.$$

$$f(n) = \Omega\left(\frac{4^n}{n^{1.5}}\right).$$

$$\Omega(4^n/n^{3/2})$$

1. Optimal parantezleme yapısı

- Gösterim:

$$A_{i\dots j} = A_i A_{i+1} \cdots A_j, i \leq j$$

- Diyelim ki $A_{i\dots j}$ için bir optimal parentezleme ifadeyi, A_k ve A_{k+1} , arasında bölsün.
- Burada $i \leq k < j$

$$\begin{aligned} A_{i\dots j} &= A_i A_{i+1} \cdots A_j \\ &= A_i A_{i+1} \cdots A_k A_{k+1} \cdots A_j \\ &= A_{i\dots k} A_{k+1\dots j} \end{aligned}$$

Optimal altproblem

$$A_{i\dots j} = A_{i\dots k} A_{k+1\dots j}$$

- $A_{i\dots k}$ kesinlikle optimal parentezleme olmalı
- $A_{i\dots k}$, için daha az maliyetli bir parentezleme olsaydı, Bunu $A_{i\dots j}$ 'nin parantez içindeki yerine koyabilir ve optimumdan daha düşük maliyetli bir parantez oluşturabilirdik. Bu da bir çelişki olacaktır.
- **Matris zinciri çarpımının bir örneğine en uygun çözüm, içinde alt sorunlara en uygun çözümleri içerir**

2. Özyineli bir çözüm

- Altproblem:

Minimum parantezleme maliyeti belirlenir.

$$A_{i\dots j} = A_i A_{i+1} \cdots A_j \quad \text{for } 1 \leq i \leq j \leq n$$

- $m[i, j] = A_{i\dots j}$ 'yi hesaplamak için gereken minimum çarpma sayısı
- Ana problem ($A_{1..n}$): $m[1, n]$
 - $i = j$: $A_{i\dots i} = A_i \Rightarrow m[i, i] = 0$, for $i = 1, 2, \dots, n$

2. Özyineli bir çözüm

- Parantezleme altproblemini ele aldığımızda

$$A_{i...j} = A_i A_{i+1} \cdots A_j \quad \text{for } 1 \leq i \leq j \leq n$$

$$= \underbrace{A_{i...k}}_{m[i, k]} \underbrace{A_{k+1...j}}_{m[k+1, j]} \quad \text{for } i \leq k < j$$

$p_{i-1} p_k p_j$

- Eğer optimal parantezleme çarpımı aşağıdaki gibi bölse $A_i A_{i+1} \cdots A_j$ k , ($i \leq k < j$) arasında

$$m[i, j] = \underbrace{m[i, k]} + \underbrace{m[k+1, j]} + \underbrace{p_{i-1} p_k p_j}$$

$A_{i...k}$ hesabı için
minimum çarpma
sayısı

$A_{k+1...j}$ hesabı için
minimum çarpma
sayısı

$A_{i...k} A_{k...j}$ hesabı için
minimum çarpma
sayısı

2. Özyineli bir çözüm

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$$

- k değerini bilmiyoruz.
 - k için $j - i$ muhtemel değer var $k: k = i, i+1, \dots, j-1$
- $A_i A_{i+1} \dots A_j$ 'yi Parantezleme maliyetini minimuma indirme problemi şu hale gelecektir.

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

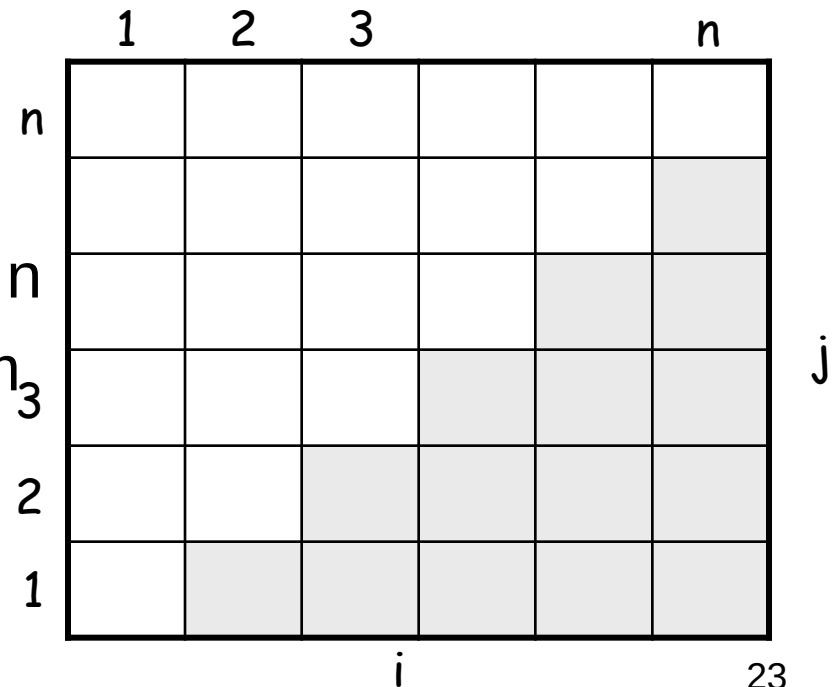
3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Optimal çözümü rekürsif olarak hesaplamak üstel zaman maliyeti alır!
- Alt problem sayısı?

$$\Rightarrow \Theta(n^2)$$

- $A_{i...j}$ 'yi parentezle $1 \leq i \leq j \leq n$
- Her i ve j seçimi 1 problem



3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Bu $m[1..n, 1..n]$ tablosunu nasıl dolduracağız?
 - $m[i, j]$ hesabında hangi tablo girişleri kullanılacaktır?

$$A_{i...j} = A_{i...k} A_{k+1...j}$$

- Alt problemlerin boyutu orijinal boyuttan küçük
- **Idea:** m 'yi artan uzunluktaki problemleri çözecek şekilde doldurun

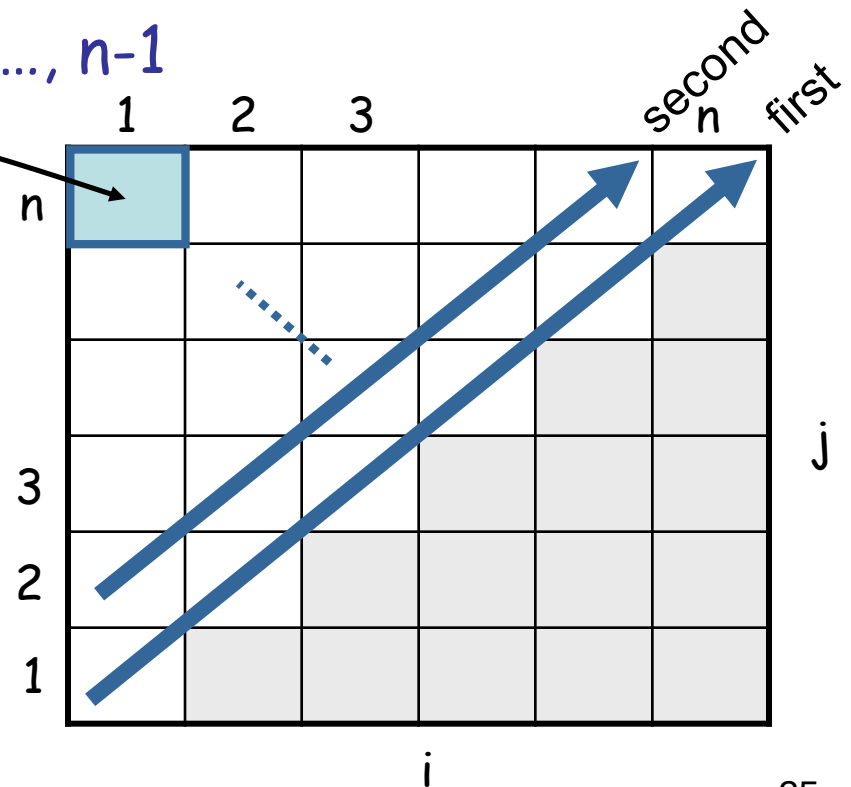
3. Optimal maliyet

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Length = 1: $i = j, i = 1, 2, \dots, n$
- Length = 2: $j = i + 1, i = 1, 2, \dots, n-1$

$m[1, n]$:problemin
optimal çözümü

Satırlar aşağıdan yukarıya ve
soldan sağa hesaplanır.



Örnek: $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1p_2p_5 & k = 2 \\ m[2, 3] + m[4, 5] + p_1p_3p_5 & k = 3 \\ m[2, 4] + m[5, 5] + p_1p_4p_5 & k = 4 \end{cases}$$

	1	2	3	4	5	6
6						
5						
4						
3						
2						
1						

i

$m[i, j]$ değerleri sadece
önceden hesaplanmış
değerlere bağlıdır.

Örnek $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

$A_1 \cdot A_2 \cdot A_3$ çarpımını hesapla:

- A_1 : 10×100 ($p_0 \times p_1$)
- A_2 : 100×5 ($p_1 \times p_2$)
- A_3 : 5×50 ($p_2 \times p_3$)

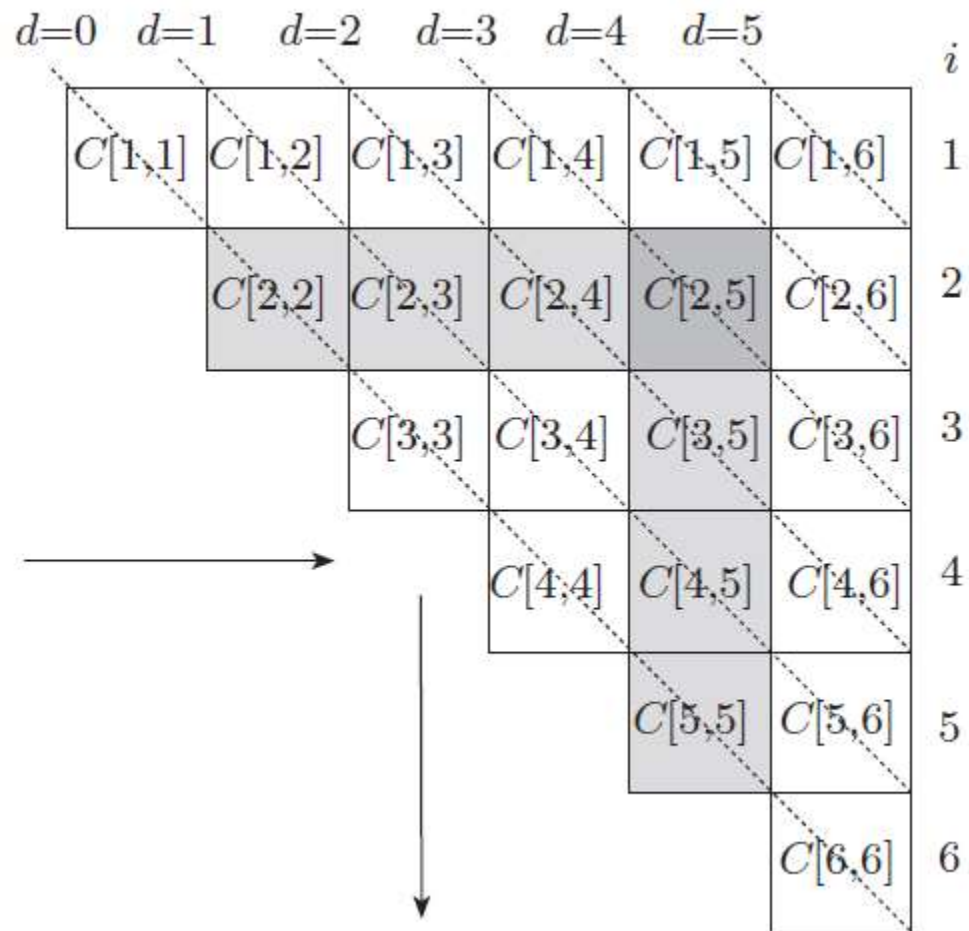
$m[i, i] = 0$ for $i = 1, 2, 3$

$$\begin{aligned} m[1, 2] &= m[1, 1] + m[2, 2] + p_0p_1p_2 & (A_1A_2) \\ &= 0 + 0 + 10 * 100 * 5 = 5,000 \end{aligned}$$

$$\begin{aligned} m[2, 3] &= m[2, 2] + m[3, 3] + p_1p_2p_3 & (A_2A_3) \\ &= 0 + 0 + 100 * 5 * 50 = 25,000 \end{aligned}$$

$$m[1, 3] = \min \begin{cases} m[1, 1] + m[2, 3] + p_0p_1p_3 = 75,000 & (A_1(A_2A_3)) \\ m[1, 2] + m[3, 3] + p_0p_2p_3 = 7,500 & ((A_1A_2)A_3) \end{cases}$$

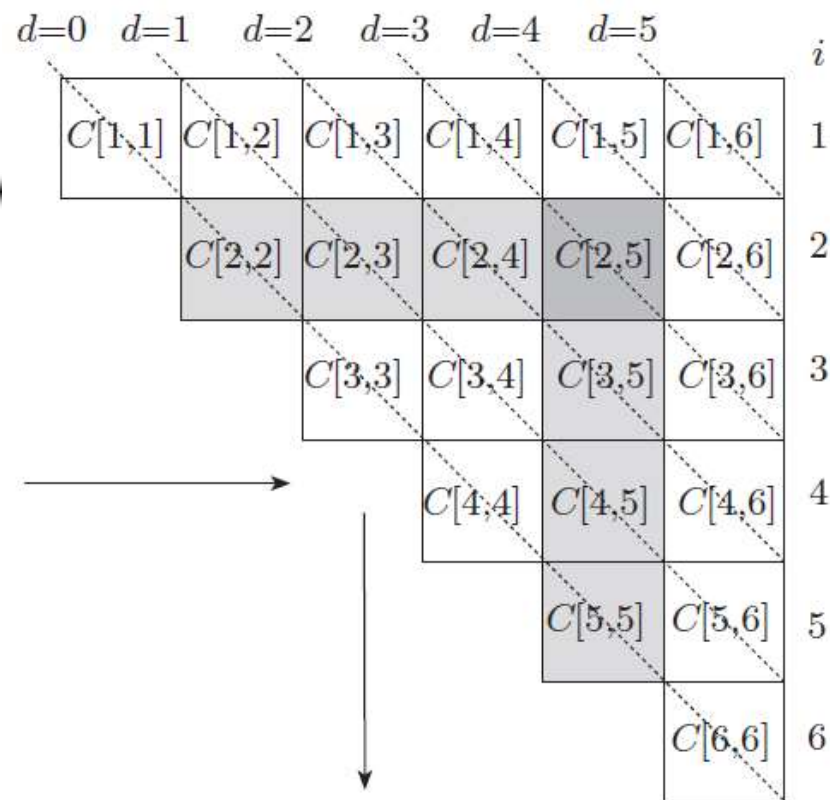
	1	2	3
3	² 7500	² 25000	0
2	¹ 5000	0	
1	0		



```

1. for  $i \leftarrow 1$  to  $n$ 
2.    $C[i, i] \leftarrow 0$ 
3. end for
4. for  $d \leftarrow 1$  to  $n - 1$ 
5.   for  $i \leftarrow 1$  to  $n - d$ 
6.      $j \leftarrow i + d$ 
7.     comment: The next three lines compute  $C[i, j]$ 
8.      $C[i, j] \leftarrow \infty$ 
9.     for  $k \leftarrow i + 1$  to  $j$ 
10.       $C[i, j] \leftarrow \min\{C[i, j], C[i, k - 1] + C[k, j] + r[i]r[k]r[j + 1]\}$ 
11.    end for
12.  end for
13. end for
14. return  $C[1, n]$ 

```



Matrix-Chain-Order(p)

```
MATRIX-CHAIN-ORDER(p)
1  n ← length[p] − 1
2  for i ← 1 to n
3      do m[i, i] ← 0
4  for l ← 2 to n           ▷ l is the chain length.
5      do for i ← 1 to n − l + 1
6          do j ← i + l − 1
7              m[i, j] ← ∞
8              for k ← i to j − 1
9                  do q ← m[i, k] + m[k + 1, j] + pi−1pkpj
10                 if q < m[i, j]
11                     then m[i, j] ← q
12                     s[i, j] ← k
13  return m and s
```

$O(N^3)$

4. Optimal Çözümü Yapılandır

- Benzer bir matriste k 'nin optimal değerlerini saklayalım
- $s[i, j] = k$;
- $k, A_{i..j}$ zincirini A_k ve A_{k+1} biçiminde çarpanlara ayırır.

	1	2	3		n
n					
			k		
3					
2					
1					

4. Optimal Çözümü Yapılandır

- $s[1, n]$, $A_{1..n}$ çarpımının tamamı ile ilişkilidir.
- Sonuç matris çarpımı $k=s[1, n]$ 'de bölünecektir.

$$A_{1..n} = A_{1..s[1, n]} \cdot A_{s[1, n]+1..n}$$

- Her bir alt çarpım için, optimum parantezlemeye karşılık gelen k değerini bulunur.

	1	2	3		n	
n						
3						
2						
1						

4. Optimal Çözümü Yapılandır

- $s[i, j] = A_i A_{i+1} \dots A_j$ zincirini A_k and A_{k+1} arasında çarpanlara ayıran k değeri

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

i

j

- $s[1, n] = 3 \Rightarrow A_{1..6} = A_{1..3} A_{4..6}$
- $s[1, 3] = 1 \Rightarrow A_{1..3} = A_{1..1} A_{2..3}$
- $s[4, 6] = 5 \Rightarrow A_{4..6} = A_{4..5} A_{6..6}$

4. Optimal Çözümü Yapılandır

PRINT-OPT-PARENS(s, i, j)

if $i = j$

then print " A_i "

else print "("

PRINT-OPT-PARENS($s, i, s[i, j]$)

PRINT-OPT-PARENS($s, s[i, j] + 1, j$)

print ")"

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					
	i					
						j

Örnek: $A_1 \cdot \cdot \cdot A_6$

$((A_1 (A_2 A_3)) ((A_4 A_5) A_6))$

PRINT-OPT-PARENS(s, i, j)

if $i = j$

then print " A_i "

else print "("

PRINT-OPT-PARENS($s, i, s[i, j]$)

PRINT-OPT-PARENS($s, s[i, j] + 1, j$)

print ")"

P-O-P($s, 1, 6$) $s[1, 6] = 3$

$i = 1, j = 6$ "(" P-O-P ($s, 1, 3$) $s[1, 3] = 1$

$i = 1, j = 3$ "(" P-O-P($s, 1, 1$) $\Rightarrow "A_1"$

P-O-P($s, 2, 3$) $s[2, 3] = 2$

$i = 2, j = 3$ "(" P-O-P ($s, 2, 2$) $\Rightarrow "A_2"$

P-O-P ($s, 3, 3$) $\Rightarrow "A_3"$

)"

)"

...

$s[1..6, 1..6]$

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

i

j

Memoization

- Tipik dinamik programlama yaklaşımının verimliliği ile yukarıdan aşağıya yaklaşım
- Her bir alt problemin çözümü için bir tabloya giriş sağlama
- **memoize** verimsiz özyinelemeli algoritmayı hatırlamak
- Bir alt problemle ilk karşılaşıldığında çözümü hesaplanır ve bu tabloda saklanır
- Alt probleme sürekli “çağrılar” ve sadece o anki değere bakın

Alg.: MEMOIZED-MATRIX-CHAIN(p)

1. $n \leftarrow \text{length}[p] - 1$

2. **for** $i \leftarrow 1$ **to** n

3. **do for** $j \leftarrow i$ **to** n

4. **do** $m[i, j] \leftarrow \infty$

} m tablosunu, $m[i, j]$ değerlerinin hesaplanıp hesaplanmadığını gösteren büyük değerlerle başlatın

5. **return** LOOKUP-CHAIN(p, 1, n) ← Top-down yaklaşım

Alg.: LOOKUP-CHAIN(p, i, j)

Running time is $O(n^3)$

1. **if** $m[i, j] < \infty$
2. **then return** $m[i, j]$
3. **if** $i = j$
4. **then** $m[i, j] \leftarrow 0$
5. **else for** $k \leftarrow i$ **to** $j - 1$
6. **do** $q \leftarrow$ LOOKUP-CHAIN(p, i, k) +
 LOOKUP-CHAIN($p, k+1, j$) + $p_{i-1}p_kp_j$
7. **if** $q < m[i, j]$
8. **then** $m[i, j] \leftarrow q$
9. **return** $m[i, j]$

Dinamik Programlama ↗ Hafıza

- Dinamik programlamanın kaydedilmiş algoritmalara göre avantajları
- Özyineleme için ek yük yok, tabloyu korumak için daha az ek yük
- Zaman veya alan gereksinimlerini azaltmak için tablo erişimine ilişkin normal desen kullanılabilir
- Kaydedilmiş algoritmaların dinamik programlamaya göre avantajları
- Bazı alt problemlerin çözülmesine gerek yoktur

Dinamik Programlamanın Bileşenleri

- Optimal AltProblem
 - Bir soruna optimal çözüm, alt sorunlara optimal çözümü içerir
 - Ana probleme optimal çözüm, optimal alt problem çözümlerden **aşağıdan yukarıya** doğru bir şekilde oluşturulur.
- Çakışan Alt Problemler(overlapping subproblems)
 - Eğer bir rekürsif algoritma aynı alt problemi tekrar tekrar çözme durumu

Optimal altyapının parametreleri

- Ana problem için optimal çözümde kaç alt problem kullanılmaktadır?
 - Matris çarpımı: İki altproblem ($A_{i..k}$, $A_{k+1..j}$)
- Optimum bir çözümde hangi alt sorunların kullanılacağını belirlemek için kaç seçeneğimiz var?
 - Matris çarpımı: **j - i seçenek k için (çarpımı ayırmak)**

Optimal altyapının parametreleri

- Sezgisel olarak, bir dinamik programlama algoritmasının çalışma süresi iki faktöre bağlı:
 - Toplam altproblem sayısı
 - Her alt problem için kaç seçeneğe bakıyoruz?
- Matrix multiplication:
 - $\Theta(n^2)$ altproblem ($1 \leq i \leq j \leq n$)
 - En fazla $n-1$ seçim var

$\Theta(n^3)$ tamamı

Matris-Zinciri Çarpımı

Örnek

Matrix-Chain Multiplication

- $A_1A_2A_3A_4$.
- $(A_1(A_2(A_3A_4)))$
- $(A_1((A_2A_3)A_4))$
- $((A_1A_2)(A_3A_4))$
- $((A_1(A_2A_3))A_4)$
- $((A_1A_2)A_3)A_4$

$$f(n) = \sum_{k=1}^{n-1} f(k)f(n-k).$$

$$f(n) = \frac{1}{n} \binom{2n-2}{n-1}.$$

İki matrisin çarpımı

➤ **Input:** $A_{p \times q}$ ve $B_{q \times r}$ matrisleri

➤ **Output:** $A \cdot B$ matrislerinin çarpımı: $C_{p \times r}$

Algorithm Matrix-Multiply($A_{p \times q}$, $B_{q \times r}$)

```
{  for  $i := 1$  to  $p$  do
    for  $j := 1$  to  $r$  do
        {   $C[i, j] := 0$ ;
            for  $k := 1$  to  $q$  do
                 $C[i, j] := C[i, j] + A[i, k] \cdot B[k, j]$ ;
            }
        return  $C$ ;
}
```

➤ **Skaler çarpma sayısı** = pqr

Örnek

- $A_{10 \times 100}$, $B_{100 \times 5}$, and $C_{5 \times 50}$
- There are 2 ways to parenthesize
 - $((AB)C) = D_{10 \times 5} \cdot C_{5 \times 50}$
 - $AB \Rightarrow 10 \cdot 100 \cdot 5 = 5,000$ scalar multiplications,
 - $DC \Rightarrow 10 \cdot 5 \cdot 50 = 2,500$ scalar multiplications,
 - Total scalar multiplication = 7,500
 - $(A(BC)) = A_{10 \times 100} \cdot E_{100 \times 50}$
 - $BC \Rightarrow 100 \cdot 5 \cdot 50 = 25,000$ scalar multiplications,
 - $AE \Rightarrow 10 \cdot 100 \cdot 50 = 50,000$ scalar multiplications,
 - Total scalar multiplication = 75,000

Dinamik Programlama

➤ Step 2: A Recursif çözüm)

- $A_i A_{i+1} \dots A_j$ çarpımının rekürsif tanımı:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j. \end{cases}$$

- Optimal çözümün yapılandırılması:
- $s[i, j] = k$; *optimal bölümlleme*.
- $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$

➤ Step 3: Optimal maliyet

- Zincir uzunluğu=1 için hesapla, Daha sonra zincir uzunluğu $l=2, 3, \dots$ için optimal maliyeti hesapla
- bottom-up

Dinamik Programlama

- **Input:** Array $p[0 \dots n]$
- **Result:** Minimum maliyet tablosu: s

Algorithm MatrixChainOrder($p[]$, n)

```
{  
    for  $i := 1$  to  $n$  do  
         $m[i, i] := 0$ ;  
    for  $l := 2$  to  $n$  do  
        for  $i := 1$  to  $n-l+1$  do  
            {  $j := i+l-1$ ;  
               $m[i, j] := \infty$ ;  
              for  $k := i$  to  $j-1$  do  
                  {  $q := m[i, k] + m[k+1, j] + p[i-1] p[k] p[j]$ ;  
                    if ( $q < m[i, j]$ ) then  
                        {  $m[i, j] := q$ ;  
                           $s[i, j] := k$ ;  
                        }  
                  }  
            }  
        }  
    return  $m$  and  $s$   
}
```

Örnek

➤ $O(n^3)$ time ve $O(n^2)$ space.

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

i\j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Örnek

- Chains of length 2 are easy, as there is no minimization required, so
- $m[i, i+1] = p_{i-1}p_i p_{i+1}$
- $m[1, 2] = 10 \times 20 \times 5 = 1000$
- $m[2, 3] = 20 \times 5 \times 15 = 1500$
- $m[3, 4] = 5 \times 15 \times 50 = 3750$
- $m[4, 5] = 15 \times 50 \times 10 = 7500$
- $m[5, 6] = 50 \times 10 \times 15 = 7500$

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

i\j	1	2	3	4	5	6
1	0	1000				
2		0	1500			
3			0	3750		
4				0	7500	
5					0	7500
6						0

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

- $m[1,3] = \min\{(m[1,1] + m[2,3] + p_0 p_1 p_3), (m[1,2] + m[3,3] + p_0 p_2 p_3)\}$
 $= \min\{(0 + 1500 + 10 \times 20 \times 15), (1000 + 0 + 10 \times 5 \times 15)\}$
 $= \min\{4500, 1750\} = 1750$
- $m[2,4] = \min\{(m[2,2] + m[3,4] + p_1 p_2 p_4), (m[2,3] + m[4,4] + p_1 p_3 p_4)\}$
 $= \min\{(0 + 3750 + 20 \times 5 \times 50), (1500 + 0 + 20 \times 15 \times 50)\}$
 $= \min\{8750, 16500\} = 8750$
- $m[3,5] = \min\{(m[3,3] + m[4,5] + p_2 p_3 p_5), (m[3,4] + m[5,5] + p_2 p_4 p_5)\}$
 $= \min\{(0 + 7500 + 5 \times 15 \times 10), (3750 + 0 + 5 \times 50 \times 10)\}$
 $= \min\{8250, 6250\} = 6250$
- $m[4,6] = \min\{(m[4,4] + m[5,6] + p_3 p_4 p_6), (m[4,5] + m[6,6] + p_3 p_5 p_6)\}$
 $= \min\{(0 + 7500 + 15 \times 50 \times 15), (7500 + 0 + 15 \times 10 \times 15)\}$
 $= \min\{18750, 9750\} = 9750$

i\j	1	2	3	4	5	6
1	0	1000	1750			
2		0	1500	8750		
3			0	3750	6250	
4				0	7500	9750
5					0	7500
6						0

Matrix	A ₁	A ₂	A ₃	A ₄	A ₅	A ₆
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

- $m[1,4] = \min\{(m[1,1]+m[2,4]+p_0p_1p_4), (m[1,2]+m[3,4]+p_0p_2p_4), (m[1,3]+m[4,4]+p_0p_3p_4)\}$
 $= \min\{(0+8750+10 \times 20 \times 50), (1000+3750+10 \times 5 \times 50), (1750+0+10 \times 15 \times 50)\}$
 $= \min \{ 18750, 7250, 9250 \} = 7250$
- $m[2,5] = \min\{(m[2,2]+m[3,5]+p_1p_2p_5), (m[2,3]+m[4,5]+p_1p_3p_5), (m[2,4]+m[5,5]+p_1p_4p_5)\}$
 $= \min\{(0+6250+20 \times 5 \times 10), (1500+7500+20 \times 15 \times 10), (8750+0+20 \times 50 \times 10)\}$
 $= \min \{ 7250, 12000, 18750 \} = 7250$
- $m[3,6] = \min\{(m[3,3]+m[4,6]+p_2p_3p_6), (m[3,4]+m[5,6]+p_2p_4p_6), (m[3,5]+m[6,6]+p_2p_5p_6)\}$
 $= \min\{(0+9750+5 \times 15 \times 15), (3750+7500+5 \times 50 \times 15), (6250+0+5 \times 10 \times 15)\}$
 $= \min \{ 10875, 15000, 7000 \} = 7000$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250		
2		0	1500	8750	7250	
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

- $m[1,5] = \min\{(m[1,1]+m[2,5]+p_0p_1p_5), (m[1,2]+m[3,5]+p_0p_2p_5),$
 $(m[1,3]+m[4,5]+p_0p_3p_5), (m[1,4]+m[5,5]+p_0p_4p_5)\}$
 $= \min\{(0+7250+10 \times 20 \times 10), (1000+6250+10 \times 5 \times 10),$
 $(1750+7500+10 \times 15 \times 10), (7250+0+10 \times 50 \times 10)\}$
 $= \min \{ 9250, 7750, 10750, 12250 \} = 7750$
- $m[2,6] = \min\{(m[2,2]+m[3,6]+p_1p_2p_6), (m[2,3]+m[4,6]+p_1p_3p_6),$
 $(m[2,4]+m[5,6]+p_1p_4p_6), (m[2,5]+m[6,6]+p_1p_5p_6)\}$
 $= \min\{(0+7000+20 \times 5 \times 15), (1500+9750+20 \times 15 \times 15),$
 $(8750+7500+20 \times 50 \times 15), (7250+0+20 \times 10 \times 15)\}$
 $= \min \{ 8500, 15750, 31,250, 10250 \} = 8500$
- $m[1,6] = \min\{(m[1,1]+m[2,6]+p_0p_1p_6), (m[1,2]+m[3,6]+p_0p_2p_6),$
 $(m[1,3]+m[4,6]+p_0p_3p_6), (m[1,4]+m[5,6]+p_0p_4p_6),$
 $(m[1,5]+m[6,6]+p_0p_5p_6)\}$
 $= \min\{(11500, 8750, 13750, 22250, 9250 \} = 8750$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250	7750	8750
2		0	1500	8750	7250	8500
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Dinamik Programlama

- Zaman maliyeti: $O(n^3)$ Bellek maliyeti: $O(n^2)$
- **Step 4: Optimal çözümü yakalayalım**
 - minimum-maliyet tablosu: $s(m, n)$.
 - $s[i, j] = k$ değeri $A_i A_{i+1} \dots A_j$ için minimum parantezleme değeri olsun.
 - Aşağıdaki procedure optimal parantezlemeyi sunar.

Algorithm PrintOptimalPerens(s, i, j)

```
{  if (i=j) then
    Print "A"i;
  else
    {  Print "(";
      PrintOptimalPerens( $s, i, s[i, j]$ );
      PrintOptimalPerens( $s, s[i, j]+1, j$ );
      Print ")";
    }
}
```

- Cevap: 8750 skaler çarpma
- Peki bu değer matrislerin hangi sırada çarpılması ile elde edilebilir?
- İlk parantezleme $k=2$ olarak uygulanır. $s[1,6]=2$.

$$A = (A_1 A_2)(A_3 A_4 A_5 A_6)$$

$$s[1,6]=2.$$

$$A_{i..j} = A_{i..k} A_{(k+1)..j} \quad s[i, j] = k.$$

- Then from the values of k : optimal parantezleme noktası

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

➤ ?

i\j	1	2	3	4	5	6
1	1	1	2	2	2	2
2		2	2	2	2	2
3			3	3	4	5
4				4	4	5
5					5	5
6						6

➤ $s[3,6] = 5$, $A_{3..6} = A_{3..5}A_{6..6}$ ve $s[3,5] = 4$,

$$A = ((A_1A_2)((A_3A_4)A_5)A_6).$$

➤ 8750 çarpma olduğu doğrulanabilir.